

ETH Zürich
Department of Computer Science

Cryptography Foundations

Ueli Maurer

Spring 2018

Contents

Preface	ix
1 Introduction	1
1.1 What Cryptography is About	1
1.1.1 The Role of Cryptography in Information Security	1
1.1.2 Cryptography – Science of Paradoxes	2
1.2 The Need for a Rigorous Mathematical Treatment	3
1.3 Some Terminology	4
1.3.1 Cryptographic Schemes and Protocols	4
1.3.2 Requirements	4
1.3.3 A Discussion of the Efficiency Requirement	5
1.4 Towards Defining Security	5
1.4.1 Security of Schemes vs. Security of Applications	6
1.4.2 Attack-based vs. “Ideal-world” Security Definitions	6
1.4.3 The Adversary – A Hypothetical Entity	7
1.4.4 Information-theoretic vs. Computational Security	8
1.4.5 Modularity and Composability	8
1.4.6 The Role of Assumptions	9
1.5 Computational Hardness and Reductions	9
1.5.1 Hardness Implications and Weakening Assumptions	10
1.5.2 Computational Reductions	10
1.6 Categories of Cryptography Research	11
2 Some Cryptographic Schemes, Protocols, and Security Definitions	12
2.1 Introduction	12
2.1.1 Purpose of this Chapter	12
2.1.2 The Story of this Chapter	13

2.2 Symmetric Encryption	13
2.2.1 Concept and Definition	13
2.2.2 Towards Defining Security of Symmetric Encryption	15
2.2.3 The One-Time Pad	17
2.2.4 The Traditional Security Proof for the One-Time Pad	18
2.2.5 Two “Security Problems” of the One-Time Pad	19
2.2.6 Computational Security: Additive Stream Ciphers	19
2.2.7 Addressing the Three Problems	20
2.2.8 Pseudo-randomness: The Concept	21
2.2.9 Block Ciphers, AES, and Pseudo-random Functions	22
2.3 Three Types of Computational Problems in Cryptography	23
2.3.1 Games	23
2.3.2 Distinction Problems	24
2.3.3 Bit-guessing Problems	26
2.3.4 The Relation between Distinction and Bit-guessing Problems	26
2.4 Constructive Cryptography: A First Example	27
2.4.1 The Basic Paradigm	27
2.4.2 The One-time Pad Example	28
2.4.3 The Simulator	30
2.4.4 Computational Security	30
2.4.5 Composability	31
2.5 Message Authentication Codes	31
2.5.1 The Concept	31
2.5.2 Security Definition	32
2.5.3 The CBC-MAC	32
2.5.4 Combining Encryption and Authentication	32
2.5.5 Encryption and Authentication with a Single Key	34
2.6 Key Agreement	34
2.6.1 The Diffie-Hellman Key-Agreement Protocol	35
2.6.2 The Mechanical Analog	36
2.6.3 Generalization to Arbitrary Cyclic Groups	36
2.6.4 From a Random Group Element to a Uniform Key	37
2.6.5 Towards Defining Security	38
2.6.6 Underlying Computational Problems	39
2.7 Public-key Encryption and the RSA System	40
2.7.1 Concept and Definition	40

2.7.2	Security Definitions for PKE	41	3.7	Discrete Computational Problems	70
2.7.3	Using Public-key Encryption	42	3.7.1	Discrete Games	70
2.7.4	Using the Diffie-Hellman Protocol as a PKE-Scheme	43	3.7.2	Discrete Distinguishers	71
2.7.5	Trapdoor One-way Permutations	43			
2.7.6	The RSA Trapdoor One-way Permutation	44	4	Computational Problems and Reductions	72
2.7.7	Public-key encryption Using a TOWP	46	4.1	Preliminaries	72
2.8	Digital Signatures	47	4.1.1	Decision Problems	72
2.8.1	Concept and Definition	48	4.1.2	Search Problems	73
2.8.2	Security Definition	48	4.1.3	Beyond Worst-case Analyses	74
2.8.3	An RSA-based Signature Scheme	49	4.1.4	Problems Relevant in Cryptography	74
2.8.4	Collision-Resistant Hash Functions	51	4.2	Example 1: Security of the RSA LSB	75
2.8.5	Hash-then-Sign	52	4.3	Example 2: Guessing the LSB of a Discrete Logarithm	77
2.9	Wrapping up this Chapter	52	4.3.1	Introduction	77
3	Discrete Systems	55	4.3.2	The Problem Statement	78
3.1	Introduction	55	4.3.3	Idea 1: Using an LSB-oracle to Compute the DL	79
3.1.1	Reactive Systems	55	4.3.4	Idea 2: From Worst-Case to Average Case	80
3.1.2	Descriptions vs. Mathematical Types	56	4.3.5	Idea 3: Restriction to the Initial Segment	81
3.2	Deterministic Discrete Systems	57	4.3.6	Idea 4: Performance Amplification for Worst-Case Solvers	82
3.2.1	Definition	57	4.3.7	Making Idea 2 Work: Randomizing the Instance	82
3.2.2	Parallel Composition	58	4.3.8	Idea 5: Changing the Instance Distribution	82
3.2.3	Interfaces, and Resources as Discrete Systems	59	4.3.9	Putting Things Together	83
3.3	Environments and Transcripts	60	4.3.10	Summary	83
3.4	Converters and Constructing Systems from Systems	61	4.3.11	Some Remarks	84
3.4.1	Introduction	61	4.4	Abstract Computational Problems and Reductions	84
3.4.2	Definition of Deterministic Converters	61	4.4.1	Introduction	84
3.4.3	Filters	62	4.4.2	Preliminaries	85
3.4.4	Cascading Systems	62	4.4.3	Problems, Solvers, and Performance	85
3.4.5	Combining the Outputs of Systems	63	4.4.4	Upper Bounds on Performance	86
3.4.6	Connecting a Converter to an Interface	63	4.4.5	The Reduction Concept	86
3.5	Probabilistic Systems	64	4.4.6	Interpretations of Reductions	87
3.6	The Behavior of Discrete Systems	65	4.4.7	Complexity-Theoretic Interpretation	87
3.6.1	A First Step: Channels in Communication Theory	65	4.4.8	Composition of Reductions	88
3.6.2	Defining the Behavior of Discrete Systems	67	4.4.9	Generalized Reductions	88
3.6.3	Equivalence of Probabilistic Systems	68	4.4.10	Worst-Case Problems	89
3.6.4	Cumulative Description of the Behavior	69	4.5	Basic Types of Computational Problems	90
3.6.5	Computing the Transcript Distribution	69	4.5.1	Games and Multi-games	90
			4.5.2	Distinction Problems	91

4.5.3	Bit-Guessing Problems	92
4.6	Discrete Computational Problems	92
4.7	Basic Reduction Types	93
4.7.1	Multiple Independent Instantiation and Cloning	93
4.7.2	Reductions by a Converter	94
4.7.3	Reductions by Multiple Instantiation	94
4.8	Some Reduction Statements for Games	94
4.8.1	Performance Amplification for Games by Repetition	94
4.8.2	Cloning Game Systems	96
4.8.3	Random Self-Reduction of Games	96
4.8.4	Combining Clonability and Random Self-Reduction	97
4.9	Hardness Amplification for Games	98
4.9.1	Combining Several Games	98
4.9.2	The Goal of Hardness Amplification	98
4.9.3	A Lemma on Multi-Argument $[0, 1]$ -Valued Functions	99
4.9.4	Hardness Amplification for Two Instances	101
4.9.5	Hardness Amplification for Many Instances	103
4.10	Relating Games and Distinction Problems	104
4.10.1	Game Equivalence	105
4.10.2	Distinguishing $(\mathcal{X}, \mathcal{Y})$ -Systems	106
4.10.3	Relating Game Winning and Distinguishing	107
4.11	Proving Indistinguishability	108
4.11.1	Conditional Equivalence	108
4.11.2	From Conditional Equivalence to Indistinguishability	109
4.11.3	Example: The URP-URF “Switching Lemma”	111
5	Constructive Cryptography	112
5.1	The Construction Paradigm	112
5.1.1	Modular Construction of Complex Systems	112
5.1.2	Abstract Constructions	113
5.1.3	Composition	114
5.2	Resource Specifications, Constructions, and Relaxations	114
5.2.1	Resource Specifications	114
5.2.2	Constructions	115
5.2.3	Relaxations	115
5.3	Constructions for Discrete Systems	117
5.3.1	Interface Types	117

5.3.2	Constructions	117
5.3.3	Resources vs. Converters	117
5.3.4	ϵ -Relaxation	118
5.3.5	Reduction-based Relaxations	119
5.3.6	$*$ -Relaxations	119
5.3.7	A new Look at the Simulation Paradigm	120
5.3.8	Game-Relaxation	120
5.3.9	Substitution Relaxation	121
5.4	Example: Authentication Amplification	121
5.5	Parameterized Resources and Constructions	122

6 Randomness Expansion 123

6.1	The Randomness Expansion Problem	123
6.1.1	Introduction	123
6.1.2	Example: k -wise Independent Random Variables	124
6.2	Domain Extension of Uniform Random Functions	124
6.2.1	Motivation	124
6.2.2	Variable Input-Length URFs	125
6.2.3	CBC-MAC as a Randomness Expander	125
6.2.4	A Construction Based on the Collision-Game for a Compression Function	127
6.2.5	Using a Collision-Resistant Hash Function	129
6.2.6	δ -Almost Universal Hash Functions	129
6.2.7	URF Domain Extension Using a δ -AUH	130

7 Constructing Shared Secret Keys 132

7.1	The Key Agreement Problem	132
7.2	Information-Theoretic Key-Agreement	133
7.2.1	Impossibility	133
7.2.2	Possibility	137
7.2.3	Privacy Amplification	138
7.3	Key-Agreement in the Random Oracle Model	141
7.3.1	Definition of a Random Oracle	141
7.3.2	Key-Agreement Using A TOWP in the ROM	142
7.3.3	Impossibility of Implementing a Random Oracle	142

A 1

A.1	Probability Theory Basics	1
-----	-------------------------------------	---

A.1.1	Probability Spaces	1
A.1.2	Random Variables	2
A.1.3	Expected Value and Variance	4
A.2	Information Theory Basics	4
A.3	Number Theory and Algebra Basics	7
A.3.1	Euclid's Extended GCD-Algorithm	8
A.3.2	Inverses Modulo m	10
A.3.3	The Chinese Remainder Theorem	10
A.3.4	Groups	11
A.3.5	Fast Exponentiation	13

Preface

Cryptography, known as a fascinating but difficult field, is a mathematical science providing the foundations and tools for constructing secure information systems.

The cryptography literature consists of a large body of proposed schemes and protocols, attacks against such schemes, security definitions, and security proofs. This course focuses on foundations of cryptography, i.e., on aspects of a general theory underlying the field. Our goal is to modularize statements and proofs so that the individual modules are simple, easy to understand, and reusable in different contexts, and so that they fit into a general theoretical framework. This is achieved by taking an abstract approach. We identify general principles and patterns that occur in different places in the literature on cryptography and try to unify them in a concise treatment.

This course builds on several other courses in the Computer Science curriculum at ETH (or taught elsewhere in a similar form). We need a good understanding of *Probability Theory*, basic algebraic concepts (e.g. groups) from *Discrete Mathematics*, and a basic understanding of algorithms, complexity, and reductions from *Theoretical Computer Science*. Moreover, some basic cryptographic schemes were discussed in *Information Security*. Despite these prerequisites, the course will be mostly self-contained, but certain topics that were treated in previous courses will be discussed quite rapidly, with references to the Appendix or to other lecture notes.

This course has benefitted enormously from discussions with many present and former postdocs and Ph.D. students. I would like to thank Divesh Aggarwal, Joel Alwen, Christian Badertscher, Fabio Banfi, Sandro Coretti, Grégory Demay, Peter Gaži, Martin Hirt, Thomas Holenstein, Daniel Jost, Chen-Da Liu Zhang, Christoph Lucas, Christian Matt, Marta Mularczyk, Krzysztof Pietrzak, Christopher Portmann, Dominik Raub, Pavel Raykov, Renato Renner, Gregor Seiler, Björn Tackmann, Stefano Tessaro, Zuzana Trubini, Daniel Tschudi, Stefan Wolf, and Vassilis Zikas for many helpful discussions and input to the course.

The lecture notes will be made available as the course evolves. They contain only few figures; many figures accompanying the text will be drawn on the blackboard. It is highly recommended to take notes (especially figures) during the course. The lectures, the exercises, and the copies of slides handed out

constitute an integral part of this course, also for the exam.

The course does not make use of textbooks, but we encourage the students to consult several books to get a more detailed and sometimes complementary view on the various topics. Some useful books on cryptography, each with a different focus, are listed below:

- O. Goldreich, *Foundations of Cryptography, Basic Tools*, Cambridge University Press, 2001
- J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, Chapman & Hall, 2015.
- W. Mao, *Modern Cryptography – Theory and Practice*, Prentice Hall, 2004.
- A.J. Menezes, P.C. van Oorschot, and S.A. Vanstone, *Handbook of Applied Cryptography*, Boca Raton: CRC Press, 1997.
- D.R. Stinson, *Cryptography – Theory and Practice*, CRC Press, 1995.
- N. P. Smart, *Cryptography Made Simple*, Springer Verlag, 2016.

Zurich, February 2018

Ueli Maurer

Chapter 1

Introduction

1.1 What Cryptography is About

1.1.1 The Role of Cryptography in Information Security

Cryptography¹ provides the core foundations for building secure information systems. It is an interdisciplinary mathematical science drawing on several fields, most importantly on computer science (theoretical computer science, complexity theory, algorithms, hardware, software), mathematics (algebra, number theory, probability theory, information theory, combinatorics, statistics, etc.), and physics (e.g. quantum physics, noise models, side-channel analysis, etc.).

To understand the mission of cryptography within information security, we observe that the field of information security can be classified coarsely into the following two sub-areas:

- **Correct system implementation.** Most of the security problems reported in the media relate to system failures and vulnerabilities exploited by an entity, often called intruder or adversary. The problem of building a secure system is hence, in principle, the same as more generally in software design, consisting of:
 - *specification* of the system,
 - *implementation* of the specification, ideally with a proof that the implementation meets the specification.

This long-studied problem is intrinsically hard since the complexity of many systems (e.g. an operating system) is too high to be really fully

¹One sometimes uses the term *cryptography* only for the science of *designing* cryptographic mechanisms, *cryptanalysis* for the science of *breaking* cryptographic systems, and *cryptology* as the term comprising both.

specified. The lack of a clear specification and/or the lack of an implementation provably meeting the specification are at the core of today's security problems.

- **Virtual system emulation.** This can be seen as the mission of cryptography. The goal is to construct, by a cryptographic protocol, a system that behaves like a virtual system, in a world where such a system does not actually exist. An example of such virtual system is a secure communication channel which is emulated (using encryption) in a world where all communication channels are insecure (e.g. based on Internet communication). Another example of such virtual system is a voting functionality to which every voter can send his/her vote and which tallies the votes and announces the result to the public. In a world where one does not trust a single computer system because it could be ill-designed or badly managed, cryptography can nevertheless emulate such a virtual voting system.

1.1.2 Cryptography – Science of Paradoxes

One can see cryptography as a *science of paradoxes*. It is hard to find another scientific discipline that has produced so many paradoxical discoveries.

Public-key cryptography is an apparently paradoxical concept. How should it be possible that two entities can communicate securely, without sharing a secret key initially, only by communication over a channel accessible to an adversary? Similarly, many of the achievements of modern cryptography are quite paradoxical:

- **Digital signatures.** A user can issue a bit-string as a digital signature for a certain digital contract such that everyone can verify the signature, yet nobody can forge a signature.
- **Interactive proofs and zero-knowledge protocols.** One can prove a mathematical statement (or the claim that one knows a certain secret value) without leaking any information about the proof (or the secret).
- **Secure multi-party computation.** An illustrative example of this general paradigm is the so-called millionaires' problem: Two millionaires want to find out who is richer, without telling each other anything about how much they own, except what is implied by who is richer. They want to achieve this solely by talking to each other or, in our terminology, by a protocol between the two. While this example is only a curiosity, many similar protocols have important potential applications, for example in e-voting. Block-chains can be seen as a special instantiation of secure multi-party computation.
- **Computing on encrypted data.** A server can perform computations on data stored on the server even though the data is encrypted and the server

does not know the secret key. This can be possible due to a homomorphic property of the encryption.

- **Security against adversaries with infinite computing power.** In many cryptographic settings one can achieve security against an adversary with infinite computing power. In other words, even trying all keys does not help to break the system.
- **Anonymous digital cash.** A bank can issue bit-strings as digital bank notes or coins. Forging coins is impossible. The digital money is anonymous in the sense that, although the bank issues the money to a known customer, it cannot recognize a coin when it is deposited and hence cannot trace where a customer spent the money.

1.2 The Need for a Rigorous Mathematical Treatment

A main problem in research and practice in information security, and more specifically in cryptography, is that *one cannot test security*. One can only *prove* security, which requires meaningful definitions of security in the first place.

Many real-world cryptographic security protocols, like TLS, were designed in a quite ad-hoc manner, and they have a long history of being broken by novel attacks, against which fixes were subsequently proposed. This cycle of breaks and fixes must be interrupted by a rigorous security protocol design methodology that achieves provable security.

Protocol design flaws (e.g. that the order of encryption and authentication is wrong) are flaws in the protocol specification and are hence present in any (even correct) implementation.² The reason for protocol design flaws is the lack of understanding of what exactly cryptographic schemes achieve and how they can be composed in a sound manner.

In this course we take a rigorous approach to designing cryptographic protocols. The goal is to precisely specify the security goals a protocol must achieve and to design a protocol that can be mathematically proven to be secure, usually based on a list of well-defined assumptions, like the hardness of a certain computational problem. This approach should be contrasted with the approach often taken in practice where a protocol design is based on heuristics and the security goals are not even precisely specified, let alone the security proved. Security statements are mathematical statements and require a proof.

A rigorous approach must answer the following three questions:

- How can we *define* security of a cryptographic scheme (e.g. encryption)?

²In this course we do not consider *implementation bugs* (e.g. the well-known buffer overflow problem), which are due to bad software design and programming methodologies.

- How can we *prove* the security of a cryptographic scheme (or protocol)?
- How can we *use* a secure scheme in a larger context?

1.3 Some Terminology

1.3.1 Cryptographic Schemes and Protocols

We discuss a few important cryptographic terms and their relation.

A *cryptographic scheme* consists of one or several functions. For example, a symmetric encryption scheme consists of a pair of functions, an encryption function and a decryption function. Other types of cryptographic schemes are message authentication schemes, hash functions, public-key encryption schemes, digital signature schemes, commitment schemes, etc.

A *cryptographic protocol* for a given set of parties consists of, for each party, a precisely specified behavior in the interaction with the other parties, for example in the form of a program that must be executed by the party. A cryptographic protocol typically makes use of several cryptographic schemes in a well-defined manner. For example, a secure communication protocol for two parties, Alice and Bob, specifies, typically, how Alice uses an encryption function and a message authentication function to transform a message before sending it over the Internet. Other examples are e-voting protocols, anonymous e-cash protocols, and so-called secure multi-party computation protocols.

The purpose of applying a cryptographic protocol is to achieve an *application goal*, for example to obtain a secure communication channel. The security of a protocol can only be judged relative to a declared (application) goal and relative to a specification of which parties must be tolerated to be dishonest.

1.3.2 Requirements

Cryptographic schemes and protocols need to satisfy the following three types of requirements:

- **Correctness:** The specification of what must be achieved, not considering an adversary. For example, an encryption scheme is correct if encryption followed by decryption results in the initial message.
- **Security:** The specification of what must be achieved when considering an adversary. For example, the adversary must not obtain any information about a message (which needs to be precisely defined).
- **Practicality:** The scheme must be usable. The most fundamental practicality requirement is **efficiency**: The functions (of a scheme) must be computable sufficiently quickly on a given computer.

Correctness is often included as a condition in the definition of a cryptographic scheme since it does not make sense to consider incorrect schemes.

In contrast, security is often considered as a separate definition, not included in the basic definition of a scheme. The reason is that one can define security in several different ways, which can all be meaningful. One can then compare different security definitions for a given scheme.

1.3.3 A Discussion of the Efficiency Requirement

For the most part of this course, practicality is obvious or implicit. For example, it is obvious how the function can efficiently be computed, or they are even specified in terms of an efficient algorithm. Practicality will rarely be an explicit topic in this course.

This is in contrast to much of the cryptographic literature, where often the efficiency condition is part of a definition. For example, a cryptographic scheme is defined to consist of *efficient algorithms* (as opposed to functions). This approach has little benefit but two significant drawbacks:

- It requires a mathematical definition of an algorithm, including a specific computational model. The typical model used in the literature is that of a (not exactly defined) Turing machine.
- It requires a mathematical definition of *efficiency*, i.e., a clear distinction between an algorithm that is efficient and one that is not, and such a distinction can only be meaningfully made in asymptotic terms, for instance by defining efficient as polynomial-time.

It is important to distinguish between the concept of a function and the concept of an algorithm for computing the function. For the most part of this course, we consider functions rather than algorithms.

Let us explain the significance of the separation between function and algorithm. In many definitions one finds in the literature, the practicality condition is included as the requirement that the involved functions must be *efficiently computable*. However, this efficient computability condition is not exactly what is required (in a mathematical sense) since one actually needs an efficient algorithm for computing the function, not just that such an algorithm exists (possibly without knowing one explicitly). This is one of many examples that illustrates the subtleties and difficulties in developing a meaningful and precise mathematical theory of cryptography.

1.4 Towards Defining Security

In this section we discuss a number of conceptual aspects arising when trying to define security.

1.4.1 Security of Schemes vs. Security of Applications

Most security definitions in the literature are security definitions for *schemes*, for example for an encryption scheme or a digital signature scheme. Such a definition is useful as the target which the designer of a cryptographic scheme has to meet. The definition serves as the interface between the designers of such schemes and the designers of cryptographic protocols making use of such schemes.

However, what one is actually interested in is that the *application* is secure, for example that transmitted messages are inaccessible for an adversary. It is generally not clear whether and how the security definitions for the *schemes* used imply the security of the application. This obviously depends on the particular way in which the scheme is used in the application and does not depend on the scheme (and its security definition) alone. For example, whether transmitting encrypted messages is secure generally depends on whether the transmission channel is authenticated or not.

In fact, for a given type of scheme there are usually several different security definitions, which shows that the required security definitions for the schemes used in an application may depend on the concrete application. Often, such security definitions can be compared in the sense that one definition is strictly stronger than another one, or that they are (mathematically) equivalent. Some of the definitions proposed in the literature may turn out to be useless (for applications) while other definitions may turn out to be too strong.

In this course we put strong emphasis on defining and proving the security of applications and understanding exactly which security definition of an underlying cryptographic scheme is required.

1.4.2 Attack-based vs. “Ideal-world” Security Definitions

Roughly speaking, one can distinguish the following approaches to making precise security definitions in cryptography.

- **Attack-based (or game-based) security definitions.** One defines an *attack game* capturing what an adversary might try to achieve (e.g. performing a so-called chosen-ciphertext attack and trying to guess some bit of a confidential message). Security means that no adversary from a specific class of considered adversaries (see Section 1.4.3) can win this game with substantial probability.
- **Ideal-world security definitions.** One defines an ideal system which is secure by definition, i.e., where an adversary is by definition limited appropriately (for example the adversary can, by definition, not receive the confidential messages). The actual protocol runs in the so-called real world. Security means that whatever the adversary can achieve in the real world, where the protocol is run, she can also achieve in the ideal world where

the ideal system is used, thus showing that the real world is not worse (for the honest parties) than the ideal world.

Constructive cryptography is a specific instantiation of this ideal-world paradigm. A main advantage of constructive security statements is that they can be naturally composed.

Attack-based security definitions focus on the adversary. Often it is not explicitly specified what the honest entities achieve, only what the adversary *can not* achieve. It is often difficult to guarantee that all relevant attacks have been included in the model, in particular because the relevant attacks might depend on the context where a system is used. One can criticize an attack-based security definition by exhibiting a cryptographic scheme satisfying the definition as well as an application in which using the scheme is insecure.

1.4.3 The Adversary – A Hypothetical Entity

The *adversary*³ is a hypothetical entity introduced to define security. Security is defined via a thought experiment showing that such a hypothetical adversary, with certain assumed capabilities, is not able to do harm to a system (or, more precisely, to do harm to the honest entities using the system).

From a mathematical perspective, the adversary stands for a universal quantifier ($\forall$) used to state that for all elements in a certain set (the set of all considered adversaries), a certain statement is true.

More generally, every entity or party in a protocol can be considered as being potentially dishonest. This is often captured in cryptography by considering a (central) adversary who can *corrupt* parties. But more generally one wants to also capture the fact that the entities can have conflicting goals and that there can therefore be several independent misbehaving entities. However, in this course we concentrate mostly on the setting with a single dishonest party.

For any security consideration to be meaningful, one must specify the assumptions about the hypothetical adversary. Such an adversary specification includes at least the following parameters:

- **Computing power** (see Section 1.4.4 below).
- **Memory capacity.** One usually allows the adversary unbounded memory, except in the so-called bounded-storage model.
- **Attack capability.** An adversary may be able to perform certain attacks, for instance a so-called chosen-message attack in which he can choose a plaintext and obtain the corresponding ciphertext (for the secret key actually in use).

³The adversary is sometimes also called attacker, opponent, eavesdropper, cryptanalyst, or Eve or Mallory. Sometimes a distinction is made between Mallory, an actively cheating adversary, and Eve who only intercepts the communication without actively cheating. For us, Eve is the general adversary.

An important principle was stated by A. Kerckhoffs (1835-1903):

Kerckhoffs' principle: "A cryptographic system should be designed to be secure when the adversary knows all details of the system, except for the values explicitly declared to be secret (e.g. secret keys)."

1.4.4 Information-theoretic vs. Computational Security

The adversary's computing power can be assumed to be bounded or unbounded:

- **Infinite computing power.** A system secure without any assumption on the adversary's computing power is called *information-theoretically secure* or sometimes *unconditionally secure*.
- **Bounded computing power.** One specifies an upper bound on the computing power, in some model of computation, and considers the time required to break the system, using the fastest algorithm. A system secure under a reasonable assumption on the adversary's computing power is called *computationally secure*.

Information-theoretic security is of course more desirable than computational security, but it is usually more difficult to achieve and the practicality of such systems is often quite limited. (It may come as a surprise that such systems exist at all.) Therefore, most practical systems are not information-theoretically secure, and one must resort to computational security, based on some intractability assumption (e.g. that factoring large integers is hard). However, it is important to note that none of these intractability assumptions has been proved, and hence no cryptosystem has been proven to be computationally secure.

1.4.5 Modularity and Composability

We want to understand cryptography as a constructive discipline. A central paradigm in any constructive discipline is the decomposition of a complex system into simpler component systems or modules, each of which may consist of yet simpler modules, and so on. This paradigm, sometimes called step-wise refinement, is useful only if the composition of modules is well-defined and preserves the relevant properties of the modules. For example, in software design, the composition operation must preserve correctness of the modules, i.e., a system consisting of correct modules must itself be correct.

In the context of cryptography, composition should mean that a protocol proved secure in isolation should remain secure if plugged into higher-level protocols or if run in parallel with other protocols. This can only be the case if the security definitions are carefully chosen.

For attack-based security definitions, the composition property is often unclear, or does not hold, or at best is highly non-trivial. One consequence of the lack of composability is that proposed security protocols are often not well-motivated, sometimes unnecessarily complex, and sometimes insecure.

1.4.6 The Role of Assumptions

The security of every cryptographic system depends on certain assumptions about the physical world. Three obvious assumptions, which are usually not stated explicitly, are:

- **Randomness exists.** This assumption is needed, for example, for the generation of secret keys.⁴
- **Independence exists.** Observations at different locations can be independent. For example, the outcome of flipping a coin is assumed to be independent of anything an adversary can observe.
- **The laws of physics are correct,** i.e., non-physical phenomena, such as reading somebody's mind, do not exist.

More generally, we make the assumption that our understanding of physics is sufficiently correct. Some security proofs are based explicitly on *physical assumptions*, for instance that quantum theory is correct, that certain devices are tamper-resistant, or that a communication channel is subject to at least a minimal noise level.

Other crucial assumptions made explicitly in cryptography are:

- **Trustworthiness** of certain entities.
- **Computational hardness** of certain computational problems.

The goal of cautious cryptographic design is to make these assumptions both explicit and as weak as possible. Reducing the required assumptions and trust requirements for achieving a certain security objective is a major theme of research in cryptography.

1.5 Computational Hardness and Reductions

What is common to all security definitions in the computational sense, attack-based or constructive, is that security is defined relative to the computational hardness of a certain computational problem.

Informally, a problem is hard if no feasible algorithm (for an appropriate notion of feasibility) can solve the problem with success probability more than

⁴Quantum theory was the first physical theory in which randomness is an inherent feature.

a negligible value (for an appropriate notion of negligible). In other words, any algorithm with a non-negligible success probability is infeasible.⁵

1.5.1 Hardness Implications and Weakening Assumptions

Since actual hardness proofs for computational problems of interest and for relevant computational models remain unachievable for the foreseeable future, possibly forever, one is usually interested in proving *implications* of the form:

If problem P is hard, then problem Q is hard.

Such a statement is of interest if it appears (*a priori*) more credible that P is hard than that Q is hard. Such implications can be composed.

A relevant research goal in cryptography is to prove the hardness of breaking a certain cryptographic scheme under an apparently very weak assumption. One therefore tries to prove hardness implications, as explained above, where P is a computational problem whose hardness is much more credible than the hardness of breaking the considered cryptographic scheme.

To weaken cryptographic assumptions is both a fundamental challenge in the theory of cryptography and at the same time a major goal for the cautious and prudent design of practical cryptographic systems.

1.5.2 Computational Reductions

Hardness implication results can be proved by so-called *reductions*, a concept we now briefly discuss. A more thorough treatment will be given in Chapter 4.

Consider two computational problems, P and Q , and the following statements A and B :

$$\begin{aligned} A &= \text{"}P \text{ is computationally hard"} \\ B &= \text{"}Q \text{ is computationally hard"} \end{aligned}$$

A typical setting is that A is an assumption and B should be proved under this assumption. In other words, one wants to prove the implication

$$A \implies B.$$

Such a statement is usually proved using that fact that $A \implies B$ is logically equivalent to

$$\neg B \implies \neg A.$$

⁵In the cryptographic literature, generally no distinction is made between the concept of efficiency in the sense of practicality (for the users) and the notion of feasibility, implying the notion of infeasibility (for an adversary), and both are called *efficient* and instantiated in the same concrete manner, namely as polynomial-time computable.

Here $\neg B \implies \neg A$ can be read as follows: If there exists a feasible algorithm for solving Q , then there also exists a feasible algorithm for solving P . This proof is usually carried out in a constructive manner, by exhibiting a feasible algorithm R , called a *reduction*, which can invoke any algorithm solving Q , and thereby solve P .

The relevance of a reduction is crucially determined by two factors:

- the loss (or gain) in success probability incurred by R , and
- the loss in efficiency incurred by R .

For example, whether R invokes the given algorithm for Q a billion times or only once makes a significant difference for the strength of the corresponding hardness implication statement.

1.6 Categories of Cryptography Research

The goals and tasks of research in cryptography can be roughly classified into the following four categories:

- **Propose cryptographic schemes and protocols**, for example a new public-key cryptosystem. Often, the claim that the cryptosystem is secure corresponds to making a new computational assumption.
- **Security proofs** for particular schemes or protocols.
- **Theoretical foundations**. A goal in itself is to develop a general theory of security statements and reductions. This course presents basic elements of such a theory.
- **Cryptanalysis** is concerned with proving that a certain assumption is false, (e.g. by exhibiting an efficient algorithm for breaking a proposed cryptographic scheme).

Chapter 2

Some Cryptographic Schemes, Protocols, and Security Definitions

2.1 Introduction

2.1.1 Purpose of this Chapter

The purpose of this chapter is to set the stage for the course. We present some basic classical cryptographic schemes and investigate how they can be composed to obtain a cryptographic protocol allowing two parties to communicate securely over an insecure channel or network. We point out the difficulties and subtleties arising if one wants to define security mathematically precisely. A main purpose of the course is to develop the understanding of security definitions and proofs. In summary, the goals of this chapter are

- to present some important basic cryptographic schemes,
- to start discussing how their security could be defined,
- to present some ad-hoc ways of using some of these schemes in a cryptographic protocol,
- to discover various flaws of such naïvely designed protocols,
- to present a well-designed cryptographic protocol for secure communication,
- to start discussing how the security of this protocol can be defined and proved, and
- to introduce several types of computational problems arising in cryptography, namely bit-guessing problems, distinction problems, and games.

It is our goal to challenge (and hit the limits of) an informal treatment of cryptography, thus providing the motivation for a rigorous mathematical treatment.

2.1.2 The Story of this Chapter

In this chapter we explore how two parties, called Alice and Bob, can establish a secure connection between them, similar to what the widely-used and well-known TLS protocol tries to achieve. We discuss this problem in a somewhat ad-hoc manner, in line with how security protocols are often discussed informally in the literature. We proceed step by step, introducing a new scheme whenever we need it to solve a problem that still remains to be solved.

2.2 Symmetric Encryption

Let us begin the story. If we want to transmit information securely, we certainly need to protect the confidentiality, i.e., the message must remain secret from an eavesdropper. This is achieved by using encryption, assuming (for now) that Alice and Bob share a secret key. The confidentiality property is also called secrecy or privacy.¹

2.2.1 Concept and Definition

The model of a *symmetric cryptosystem*, also called a *symmetric encryption scheme*, is shown in Figure 2.1. The message m , often called *plaintext*, is encrypted by a sender Alice, using a *secret key* k , resulting in the *ciphertext* c . The encryption transformation can be probabilistic, i.e., involve fresh random bits for each message to be encrypted.

The ciphertext is transmitted to the receiver Bob over a non-confidential channel to which an adversary has access, either only read access, or read/write access, depending on the assumed model. The secret key is shared between sender and receiver by some secure means, e.g. by sending it before-hand over a secure channel, for instance by use of a trusted courier. The receiver can decrypt the ciphertext, using the secret key, resulting in the plaintext.

Formally, a symmetric cryptosystem can be defined as follows:

Definition 2.1. A *symmetric cryptosystem* (or *symmetric encryption scheme*) for message space $\mathcal{M}$, ciphertext space $\mathcal{C}$, key space $\mathcal{K}$, a probability distribution P_K over $\mathcal{K}$,² and randomness space $\mathcal{R}$, is a pair of functions:

¹We avoid the term privacy because it is generally used specifically to refer to the protection of personal data.

²The key distribution is almost always the uniform distribution.

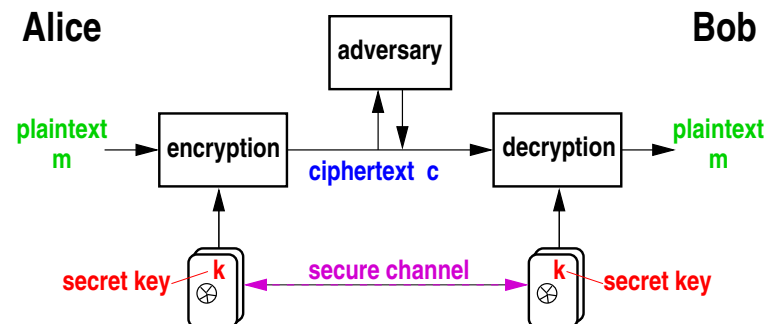


Figure 2.1: Model of a symmetric cryptosystem. A symmetric cryptosystem can be probabilistic, i.e., the encryption transformation can involve random bits generated afresh for each message.

- an encryption function $e: \mathcal{M} \times \mathcal{K} \times \mathcal{R} \rightarrow \mathcal{C}$, and
- a decryption function $d: \mathcal{C} \times \mathcal{K} \rightarrow \mathcal{M} \cup \{\perp\}$,³

such that $d(e(m, k, r), k) = m$ for all $k \in \mathcal{K}$, $m \in \mathcal{M}$, and $r \in \mathcal{R}$ (correctness).

We will always assume that messages, ciphertexts, and keys are bit-strings, i.e., $\mathcal{M}$, $\mathcal{C}$, and $\mathcal{K}$ are subsets of $\{0, 1\}^*$. One could, as an alternative but equivalent definition, define e to be a randomized function, i.e., a conditional probability distribution of the ciphertext, given the message and the key. This would avoid making the randomness space $\mathcal{R}$ explicit but would make the correctness condition slightly more cumbersome to describe.

A deterministic symmetric cryptosystem (i.e., without randomness space $\mathcal{R}$), is also called a *cipher*.

We point out that by such a definition of a cryptographic scheme, even though its intended use might be somehow implicit, it is generally not explicit. There can be different ways of using the scheme, some of which may be secure while others are insecure. For example, one typically may want to encrypt many messages with the same key, but it also makes sense to use any given key only for encrypting a single message, and then to change the key. In the former, multi-message case, a deterministic symmetric cryptosystem is problematic since an adversary can at least recognize when the same message is repeated. This problem does not arise if the application is known to never repeat a message, for example because the messages contain an increasing counter or the time. Another technique for achieving multi-message security is to keep a state in the encryption function between messages.

³The symbol $\perp$ means that no decrypted message is generated, for example because the ciphertext is invalid in the sense that it does, for the given key, not correspond to any message.

As explained in Section 1.3, the above definition does not include the security condition; it only captures a correctness condition, namely that decryption with the correct key must always work. The identity function (i.e., no encryption at all) satisfies the definition but can hardly be considered secure.

Moreover, the definition, as stated, does not include a *practicality* condition, namely that the encryption and decryption functions must be efficiently computable (see Section 1.3.3).

2.2.2 Towards Defining Security of Symmetric Encryption

A necessary requirement for a symmetric cryptosystem to be secure is that the key space be large enough so that an exhaustive key search is infeasible. In practice, a commonly used key length is 128 bits (i.e., $|\mathcal{K}| = 2^{128}$).

An intuitive security definition for an encryption scheme could be that it is infeasible to compute the plaintext when given the ciphertext, without knowledge of the secret key. However, such a security definition is unsatisfactory for two reasons.

- First, it should be infeasible to compute *any* information about the plaintext, even to guess a single bit with success probability significantly greater than what would be possible without seeing the ciphertext.
- Second, this should be true even if the adversary can mount various attacks on the cryptosystem, for instance adaptively choose a plaintext and receive the corresponding ciphertext (under the unknown secret key), as well as choose a ciphertext and receive the corresponding plaintext.

Hence we need a more adequate security definition. A natural approach appears to be to require that no *partial* information should leak. But how can one capture partial information? The goal of the following definition is to capture this.⁴ We first discuss a game-based security definition and then briefly mention the security definition in constructive cryptography. Game-based security means that a certain game, defined between a challenger and an attacker or adversary, cannot be won by any feasible adversary substantially better than what is trivially possible. We define the so-called IND-CPA game which leads to the so-called IND-CPA security notion. Here “CPA” stands for *chosen-plaintext attack* and “IND” stands for indistinguishability.

Definition 2.2. t -message IND-CPA game between an adversary and a challenger:

1. The challenger chooses a secret key k according to the key distribution P_K .

⁴In the literature, the term *semantic security* is used to refer to definitions of the following form: For any binary function of the message, and assuming one is restricted to efficient (say, polynomial-time) computation, then one cannot guess the binary value of that function when given the ciphertext substantially better than when the ciphertext is not given. It has been shown that this security notion is equivalent to the IND-CPA security notion discussed here.

2. The adversary can choose up to t messages and receive their encryptions; for a message m , the challenger returns $e(m, k, r)$ for a fresh and independent randomness value $r \in \mathcal{R}$.
3. The adversary chooses two messages m_0 and m_1 of the same length.⁵
4. The challenger chooses a uniformly random bit b , computes the encryption of m_b , i.e., $c = e(m_b, k, r)$ for fresh and independent randomness value $r \in \mathcal{R}$, and returns c to the adversary.
5. The adversary can again choose messages and receive their encryptions, as in step 2, where the total number of messages (including step 2) is limited to t .
6. The adversary guesses b (i.e., which message was encrypted) by issuing a guess b' .

The adversary’s *advantage* in this game is $2(\Pr(b' = b) - \frac{1}{2})$.

In the literature, the above definition is typically stated as follows:

A symmetric cryptosystem is *IND-CPA secure* if every polynomial-time adversary (algorithm) has negligible⁶ advantage in the above game, where t is not bounded (i.e., only bounded by the adversary’s running time).

We make a few remarks about Definition 2.2:

- Subtracting $\frac{1}{2}$ in the definition of the advantage is justified by the fact that achieving $\Pr(b' = b) = \frac{1}{2}$ is trivial by outputting a random bit b' . The factor 2 serves the purpose of calibrating the advantage so that the maximal value is 1.
- The existence of steps 2 and 5 is also referred to as the availability (to the adversary) of an encryption oracle.
- A *fixed-challenge* version of the security definition is obtained if one takes the maximum over all message pairs (m_0, m_1) rather than allowing the adversary to choose the pair after seeing some ciphertexts.
- The definition can be strengthened by also allowing the adversary to make queries to a decryption oracle (where, of course, the challenge ciphertext is forbidden). This notion is referred to as IND-CCA (chosen-ciphertext attack) and is not discussed further at this point.

Exercise 2.1. Show that the above fixed-challenge version of the security definition is strictly weaker than the general IND-CPA security definition.

It is *a priori* not clear whether IND-CPA security suffices for a particular application, or whether the stronger IND-CCA security is required, or whether one

⁵Why is it necessary to require that the messages are of the same length?

⁶for a suitable negligibility notion

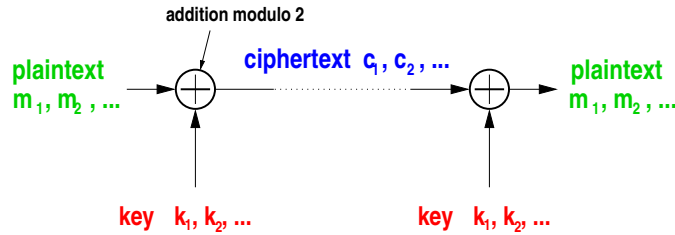


Figure 2.2: The one-time pad (OTP) encryption scheme. The plaintext (consisting of bits $m_1, m_2, \dots$) the key (consisting of bits $k_1, k_2, \dots$) and the ciphertext (consisting of bits $c_1, c_2, \dots$) are bit-strings of equal length, say n bits. The key is a uniformly random bit-string that is used only once. Encryption [decryption] consists of XOR-ing (i.e., adding bit-wise modulo 2) the key to the plaintext [ciphertext].

needs an even stronger definition. (For example, one could allow parts of the secret key to leak to the adversary.) More precisely, it is not clear which security notion is sufficient for which application, in particular for establishing a secure channel between Alice and Bob.

As explained in Section 1.4.2, this is an intrinsic dilemma of game-based security definitions. It may not be clear which definition suffices for the application within a certain protocol. This is a problem for two reasons.

1. We want a security proof for our overall protocol and therefore need to understand which security notion for encryption is required.
2. If we work with an unnecessarily strong security notion, then a scheme achieving that notion may not exist at all, or if one exists, it may be significantly less efficient than one satisfying a weaker notion.

These problems will be addressed by introducing constructive cryptography.

Actually, here is yet another security definition for symmetric encryption. The adversary can choose up to t messages m_i , with $1 \leq i \leq t$, and either always gets the encryption of a fresh random message of the same length as the chosen message, or he gets the real encryption (with the unknown secret key) of the message. (There are also two versions, depending on whether the choice of messages is adaptive or not.) Investigating this security definition and its relation to the IND-CPA game (Definition 2.2) is the subject of an exercise.

2.2.3 The One-Time Pad

In this section we discuss the one-time pad (OTP, see Figure 2.2), the best-known provably secure cryptosystem, as a motivating example. We present the tradi-

tional security statement for OTP encryption and explain three intrinsic problems with this security statement for the one-time pad:

1. In Section 2.2.5, we show that, despite the proof, the OTP is nevertheless insecure, for two different reasons.
2. It is not necessarily clear how one can use one-time pad encryption securely within a larger protocol, i.e., how it can securely be composed with other cryptographic schemes (e.g. authentication).
3. In Section 2.2.6, we argue that the security proof cannot be carried over to a more practical setting where the (long) secret key is replaced by a pseudo-random key generated from a short secret key.

The constructive approach to defining security will solve all these problems.

2.2.4 The Traditional Security Proof for the One-Time Pad

The OTP encryption scheme can be proved to be information-theoretically secure, i.e., unbreakable even for a *computationally unbounded* adversary. How can one formulate and prove this claim?

The usual way to argue about the security of OTP encryption is to show that the plaintext (also called the message) and the ciphertext are statistically independent. This means that the ciphertext gives no information about the plaintext, independent of the available computing power. In other words, if one has to guess the plaintext when given the ciphertext one can just as well ignore the ciphertext.

Proposition 2.1. In one-time pad encryption, for a given message length n , the plaintext and the ciphertext are statistically independent (for all plaintext distributions),⁷ i.e.,

$$P_{CM}(c, m) = P_C(c) \cdot P_M(m)$$

for all m and c .

Exercise 2.2. Prove the following more general statement of which the proposition is a special case. Consider a group G with group operation $\star$ and consider an arbitrary random variable X over G as well as an *independent* random variable U with uniform distribution over G . Then the random variables $U \star X$ and X are statistically independent. The proof also shows that $U \star X$ is uniform, no matter what the distribution of X is.

We note that statistical independence can also be captured by the condition $P_{M|C}(m, c) = P_M(m)$ for all m and c , which states that the distribution of the message does not change if one learns the ciphertext. We also note that the fact that $P_{C|M}(\cdot, m)$ is independent of m can be seen as an information-theoretic

⁷This property is called *perfect secrecy*, a term introduced by Shannon.

version of the IND-CPA security definition: **The ciphertext has the same distribution for all messages.**

In the following two subsections we discuss two reasons why this security statement is not sufficient.

2.2.5 Two “Security Problems” of the One-Time Pad

Despite the above security proof, the OTP appears to be very insecure, at least for two independent reasons.

- First, an **adversary (Eve)** with access to the communication channel can **modify the message in a controlled manner**, even though she gets no information about the transmitted message. The adversary can add (bit-wise modulo 2, i.e., XOR) an arbitrary offset δ to the transmitted message, simply by adding δ to the ciphertext. If OTP encryption were used in a banking application, an attacker could for instance change the bank account number of a money transfer.
- Second, suppose that the OTP is used to encrypt one of the two messages “Yes” or “No”. Since the two messages have different lengths, the ciphertext leaks complete information about the message, at least in a naïve implementation of the OTP. This appears to be in contradiction to the above claim that the ciphertext is independent of the plaintext message. This makes clear that when using OTP encryption, **either one should pad all messages to have the same length**, or one should accept the fact that the length of the message leaks.

2.2.6 Computational Security: Additive Stream Ciphers

To illustrate the third short-coming of the security statement (Proposition 2.1), we consider a variation of OTP encryption used in practice, where the uniformly random key is replaced by a key-stream sequence that is generated deterministically from a short secret key by a so-called key-stream generator. This is called an **additive stream cipher** (see Figure 2.3).

How should one define the security of such a stream cipher? Our goal is that the two security definitions for OTP encryption and stream cipher encryption are similar. **Because the entire key-stream is generated from a short secret key, the system cannot be information-theoretically secure.** One could (theoretically) break the system by trying all possible keys until the correct one is identified as the only key that results in meaningful plaintext after decryption.⁸ Hence we must find a way to formulate what it means for this system to be *computationally secure*.

⁸This requires a sufficient amount of ciphertext and assumes that the plaintext is redundant, i.e., that one can distinguish between meaningful and meaningless messages.

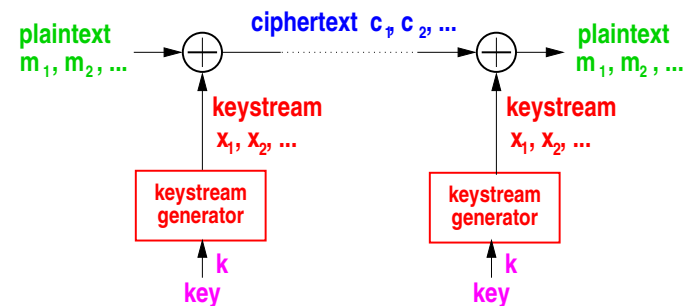


Figure 2.3: Binary additive stream cipher. It is obtained from the OTP encryption by replacing the random key $k_1, k_2, \dots$ by the output $x_1, x_2, \dots$ of a pseudo-random bit generator (also called key-stream generator) with a short key k . This system can be used for encrypting arbitrarily long messages, and also many consecutive messages if sender and receiver use consecutive portions of the key-stream.

2.2.7 Addressing the Three Problems

All three problems mentioned in Section 2.2.3 can be solved by *constructive cryptography* introduced later (see Section 2.4 for a treatment of the OTP example).

The first criticism, namely that there are attacks against the OTP not covered by the security proof, could be eliminated by stating that we assume the **communication channel to be authenticated** (but not necessarily confidential). Then the security proof could be interpreted as stating that OTP encryption “constructs” a secure channel from an authenticated channel (and a shared secret key). The fact that the message length leaks can be solved by simply stating this fact as part of the specification of the secure channel. What “construct” means is a question we will address later.

The second criticism, namely that the statement of Proposition 2.1 does not (necessarily) guarantee composability with other cryptographic schemes, is resolved in constructive cryptography by the fact that resources constructed by a protocol (i.e., a secure channel) can be used as the assumed resources in further construction steps.

The third criticism, namely that the security definition does not extend to computational security, can be solved by defining a computational⁹ variant of the construction notion.

⁹The term “computational” is often used to state that security only holds against computationally bounded adversaries.

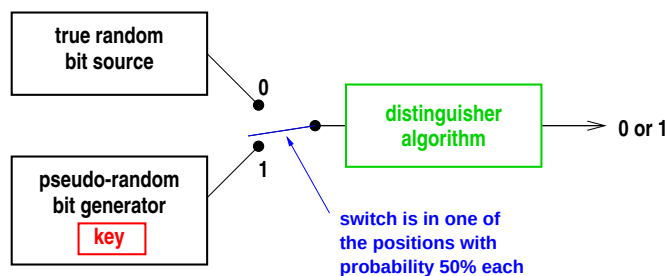


Figure 2.4: Distinguishing the output of a pseudo-random bit generator from a truly random bit generator. This is a special case of a distinction problem (see Section 2.3.2), where the figure appears mirrored, with the distinguisher on the left side.

2.2.8 Pseudo-randomness: The Concept

In practice one often approximates the generation of random bits by a deterministic algorithm, starting from a random *seed* or *key* value. This is called a *pseudo-random bit generator (PRBG)*. There are two reasons for using a PRBG rather than a true random bit generator:

- Implementing a random bit generator on a given hardware platform may be infeasible or impossible.
- The generated random bit sequence must be reproducible at another location. This is a typical problem in cryptography where secret keys should be short. But the problem also occurs in other contexts, for example in scientific simulations where one wants a result to be reproducible.

It is delicate to define pseudo-randomness properly. A bit generator that outputs the constant sequence $000000 \dots 0000$ would probably not be considered a PRBG, nor one that outputs $010101 \dots 0101$. How do we capture the requirement that the output sequence must “look random”? Any fixed list of statistical or other properties (e.g. large period) is arbitrary and falls short of capturing all aspects of “looking random”. The actual definition used in cryptography is based on a very important general paradigm for security definitions, namely *indistinguishability*. A PRBG is secure if it cannot be distinguished from a truly random bit generator by any *feasible* distinguisher algorithm (see Figure 2.4), except with very small (negligible) advantage.

The notions of feasible and negligible are usually defined asymptotically as (some form of) polynomial-time computable and as decreasing faster than the inverse of any polynomial, respectively. We will address this later.

An important concept for generating pseudo-random bits is that of a deterministic function which expands a given random input to a longer pseudo-

random output. Let U_m denote a random variable with uniform distribution over the bit-strings of length m .

Definition 2.3. (Informal) A function $g : \{0, 1\}^k \rightarrow \{0, 1\}^m$ for $m > k$ is called a *pseudo-random generator (PRG)* if no feasible algorithm has a non-negligible advantage in distinguishing $g(U_k)$ and U_m .¹⁰

One generally also needs an efficient algorithm for computing the function, but in line with the explanations of Section 1.3.3, we have not included this condition in the formal definition of a PRG. Actually, what one finds in the literature is the condition that the function g is required to be *efficiently computable*. As pointed out in Section 1.3.3, such a condition is actually not sufficient.

Instead of working with Definition 2.3, which is not yet formal, we will work with the *distinction problem* of distinguishing $g(U_k)$ and U_m . We refer to Section 2.3.2 for the definition of a distinction problem.

An important natural question, which we later answer positively, is the following: *If a PRG is applied iteratively, by using part of the output as the new input, does one obtain a PRG with a greater expansion factor?* For example, if we start from a uniform random n -bit key and use a PRG $g : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ to generate $2n$ pseudo-random bits, then we can reuse half of these bits as a new input to g , resulting in $3n$ pseudo-random bits. This process can be repeated, resulting in an algorithm that starts from a seed value and produces a pseudo-random sequence of arbitrary length, i.e., to obtain a PRBG as previously discussed.

2.2.9 Block Ciphers, AES, and Pseudo-random Functions

An important type of cipher used in practice are so-called *block ciphers*, a stateless encryption function where plaintext and ciphertext are n -bit strings (usually $n = 64$ or $n = 128$).

It is important to point out that a block cipher should not be used as a symmetric cryptosystem, but a block cipher serves as a building block to define an encryption function. Such a particular way of using a block cipher is called a *mode*. Three such modes, which are not discussed in the course notes, are cipher block chaining (CBC), counter mode, and cipher feedback mode.

Definition 2.4. A *block cipher* with block length n and key length m is a function $f : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ such that for every key k , $f(\cdot, k)$ is a bijection (i.e., a permutation) $\{0, 1\}^n \rightarrow \{0, 1\}^n$.

The practicality requirement is that one knows efficient algorithms for computing $f(x, k)$ and the inverse $f^{-1}(\cdot, k)$.

¹⁰To make this definition formal, one would need to consider an asymptotic setting, where g would be a family of functions, k would be a security parameter, and m would be a function of k .

The most prominent example of a block cipher is the so-called *Data Encryption Standard (DES)* with $n = 64$ and $m = 56$, developed in the 70's and standardized by the U.S. government. The currently most widely used block cipher is the so-called *Advanced Encryption Standard (AES)*, standardized by the U.S. government in 2000. It allows for different parameter sizes; the most often used parameters are $n = 128$ and $m = 256$.

When used in the natural mode called electronic codebook mode, a message is encrypted by cutting it into n -bit blocks and encrypting each block separately. This encryption mode has a security problem (see exercises).

A block cipher can also be used in other modes of operation which we do not discuss at this point.

A very strong and natural security requirement can be captured via the problem, parameterized by t , of distinguishing the following two systems (see Section 2.3.2):

- a system that, for a uniformly chosen key K , answers t queries $f(x, K)$ for (adaptively chosen) values of x .
- a system that, for a function $g : \{0, 1\}^n \rightarrow \{0, 1\}^n$ chosen uniformly from the set of functions $\{0, 1\}^n \rightarrow \{0, 1\}^n$, answers t queries $g(x)$ for (adaptively chosen) values of x .

A function $f : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ for which no feasible distinguisher has a non-negligible advantage in this distinction problem is called a *pseudo-random function (PRF)*.

2.3 Three Types of Computational Problems in Cryptography

In this section we discuss three types of computational problems of interest in cryptography, and more generally in computer science. A more formal discussion will follow later.

2.3.1 Games

A game is a generalization of a search problem, where a solver must find a solution to a given problem instance. A game is defined by a system G (containing internal randomness) to which a solver (in the context of games also called a winner) W has access (see Figure 2.5). For the game G a winning condition is defined, shown in Figure 2.5 as a binary output A on the right side of G . The quantity of interest is defined as follows.

Definition 2.5. The winning probability of W for game G is

$$\Gamma^W(G) := \Pr^{WG}(A = 1).$$



Figure 2.5: The winner W connected to the game G , producing a binary output A indicating whether the game is won.



Figure 2.6: The distinction problem of distinguishing S and T . The goal is that D outputs 1 if connected to T and 0 if connected to S .

Examples of games are the collision-finding problem for a hash function and the discrete logarithm problem, both discussed later in this chapter.

2.3.2 Distinction Problems

We have already seen two examples of distinction problems: distinguishing the output of a PRG from a uniform random string, and distinguishing a block cipher (with a secret random key) from a uniform random function.

A distinction problem (see Figure 2.6) is defined by two systems, say S and T . A so-called distinguisher D is connected to one of the systems and produces a binary output Z . The task of a distinguisher D is to distinguish S and T . The goal of a distinguisher is to always output $Z = 1$ if connected to T and to never output $Z = 1$ (i.e., always outputs $Z = 0$) if connected to S . This is captured in the following definition defining how good a distinguisher is. This interpretation will be discussed in the next section.

Definition 2.6. The *advantage* of D in distinguishing S and T is¹¹

$$\Delta^D(S, T) := \Pr^{DT}(Z = 1) - \Pr^{DS}(Z = 1). \quad (2.1)$$

Distinguisher classes. In the cryptographic literature, one usually considers the advantage of the *best* distinguisher, where possibly one considers a restricted

¹¹Note that, as indicated, the two probabilities in this definition are defined in different random experiments, namely the random experiment of selecting D and S independently, and the random experiment of selecting D and T independently.

set $\mathcal{D}$ of distinguishers (e.g. the class of feasibly implementable distinguishers, for some notion of feasibility).

Definition 2.7. The maximal advantage of distinguisher class $\mathcal{D}$ is

$$\Delta^{\mathcal{D}}(S, T) := \sup_{D \in \mathcal{D}} \Delta^D(S, T).$$

Moreover, if the class of *all* distinguishers is considered, then we write

$$\Delta(S, T) := \sup_D \Delta^D(S, T).$$

One can easily show that probabilistic distinguishers are not more powerful than deterministic ones in the sense that best deterministic distinguisher has the same advantage as the best probabilistic distinguisher.

In the literature, the distinguishing advantage of D is often defined as the *absolute value* of $\Delta^D(S, T)$, but is nevertheless written as $\Delta^D(S, T)$.

Distinguishing random variables. The distinguishing concept is perhaps best illustrated for the problem of distinguishing two random variables X_0 and X_1 with range $\mathcal{X}$. As the class $\mathcal{D}$ of distinguishers we can consider the set of all functions $\mathcal{X} \rightarrow \{0, 1\}$.

Definition 2.8. The *statistical distance* of two random variables X and Y over a finite set $\mathcal{X}$, denoted $\delta(X, Y)$ (or sometimes $\|P_X - P_Y\|$), is defined as

$$\delta(X, Y) := \frac{1}{2} \sum_{x \in \mathcal{X}} |P_X(x) - P_Y(x)|.$$

Lemma 2.1. The advantage of the best distinguisher for X and Y is the statistical distance $\delta(X, Y)$ between X and Y , i.e.,

$$\Delta(X, Y) = \delta(X, Y).$$

Proof. Left as an exercise. Note that a distinguisher for such random variables is a (possibly probabilistic) function $\mathcal{X} \rightarrow \{0, 1\}$. $\square$

The following lemma, which has a straight-forward proof, allows to bound the distinguishing advantage between two systems S_0 and S_k in terms of the distinguishing advantages of intermediate (hybrid) systems. The application of this lemma often called a *hybrid argument*.

Lemma 2.2. For any D ,

$$\Delta^D(S_0, S_k) = \Delta^D(S_0, S_1) + \Delta^D(S_1, S_2) + \cdots + \Delta^D(S_{k-1}, S_k).$$

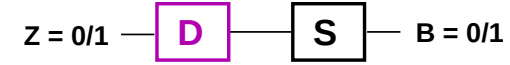


Figure 2.7: The bit-guessing problem (S, B) and a distinguisher D trying to guess the bit B by the bit Z , the goal being $Z = B$.

2.3.3 Bit-guessing Problems

The IND-CPA game discussed above is an example of a specific type of computational problem arising in cryptography, which we call bit-guessing problem.¹²

A bit-guessing problem (see Figure 2.7) consists of a system S and a binary random variable B (correlated with S), which can be thought of being output on the right side of S . Note that, in contrast to a game, the bit B is fixed from the beginning, independently of the interaction on the left side of S .

The task of a distinguisher D , with access to S , is to guess B . Since a random guess yields success probability $\frac{1}{2}$, we want to measure the performance of D only in terms of by how much the success probability exceeds $\frac{1}{2}$. Moreover, we calibrate the advantage so that it is 1 if D is always correct and -1 if it never correct.

Definition 2.9. The *advantage* of D (in guessing bit B) is¹³

$$\Delta^D((S, B)) := 2 \cdot \left(\Pr^{D(S, B)}(Z = B) - \frac{1}{2} \right).$$

2.3.4 The Relation between Distinction and Bit-guessing Problems

Bit-guessing problems are closely related to distinction problems. To demonstrate this, consider two systems S_0 and S_1 and the bit-guessing problem (S_U, U) , where U is an unbiased random bit (independent of anything else).¹⁴ The task of D is hence to guess whether it is connected to S_0 or S_1 when one of the two is randomly selected. Then we have:

Lemma 2.3.

$$\Delta^D((S_U, U)) = \Delta^D(S_0, S_1).$$

¹²The term “game” (often used in the literature) is used here only in the sense of Section 2.3.1. Deciding between only two options is a bit-guessing problem.

¹³Recall that $\Pr^{D(S, B)}$ stands for probabilities in the random experiment of selecting D and the (correlated) pair (S, B) independently.

¹⁴More precisely, the pair (S_U, U) is chosen by first choosing a uniformly random bit U and then, if $U = 0$ (or if $U = 1$), choosing S_0 (or S_1) according to its distribution.

Proof. For any D we have, according to Definition 2.9,

$$\begin{aligned}
\Lambda^D((S_U, U)) &= 2 \cdot \left(\Pr^{D(S_U, U)}(Z = U) - \frac{1}{2} \right) \\
&= 2 \cdot \left(\frac{1}{2} \Pr^{DS_0}(Z = 0) + \frac{1}{2} \Pr^{DS_1}(Z = 1) - \frac{1}{2} \right) \\
&= 2 \cdot \left(\frac{1}{2} (1 - \Pr^{DS_0}(Z = 1)) + \frac{1}{2} \Pr^{DS_1}(Z = 1) - \frac{1}{2} \right) \\
&= \Pr^{DS_1}(Z = 1) - \Pr^{DS_0}(Z = 1) \\
&= \Delta^D(S_0, S_1),
\end{aligned}$$

where we have used Definition 2.6 in the last step. $\square$

Conversely, consider a bit-guessing problem (S, B) . We can consider distinguishers that have access to a system and a bit and the problem of distinguishing the pair (S, B) from the pair (S, U) where the bit U is uniform and independent of S . In the former case the distinguisher sees S and the bit B shown on the right side of the system S . In other words, the distinguisher is given access to S as well as a bit which is either the bit B correlated with S or an independent uniformly random bit U . The following lemma states that if the advantage in guessing bit B when given access to system S is small, then also the advantage in distinguishing the pair (S, B) from the pair (S, U) is small and therefore one can use the bit B more or less like as if it were an independent uniform bit. In other words, a **hard-to-guess bit behaves, in a computational sense, essentially like an independent uniform random bit.**

Lemma 2.4. For any distinguisher D and for a simple mapping¹⁵ $D \mapsto D'$ we have

$$\Delta^D((S, B), (S, U)) = \frac{1}{2} \Delta^{D'}((S, B)).$$

Proof. Left as an exercise. $\square$

2.4 Constructive Cryptography: A First Example

2.4.1 The Basic Paradigm

In the constructive cryptography framework, a cryptographic mechanism (e.g. the one-time pad) is interpreted as a method for *constructing* a certain *resource* (called the *constructed resource*) from certain *assumed resources*. The construction is achieved by each (honest) party attaching a system executing its protocol to its interface of the assumed resource. Such an attached protocol system is called a *converter*.

¹⁵A formal version of this lemma would have to specify this mapping, but here we leave finding it as an exercise.

A resource is a system with interfaces to the involved parties, typically including some honest parties and a (hypothetical) adversary. The constructed resource captures everything that is guaranteed, including what the honest parties can achieve and what an adversary can (and can not) do.

One can think of two worlds, a “real world” where the assumed resources are available and the protocol is executed, and an “ideal world” where the constructed resource is available. One argues that an adversary could, in the ideal world, achieve anything that she could achieve in the real world. This argument involves, as a thought experiment, a *simulator system* which transforms the (constructed) resource in the ideal world into the system in the real world.

We point out that security properties appear in constructive cryptography very differently when compared to a standard treatment of the security of cryptographic schemes, such as encryption. In the constructive approach, the *use* of a scheme is made explicit (which is important) in the converters, and the security of the scheme (for that way of using it) is captured by the fact that a certain construction is achieved. This statement may or may not be equivalent to a certain game-based security definition of a scheme. In the best case, the two types of statements are equivalent, which provides justification for the game-based security definition.

2.4.2 The One-time Pad Example

We need to define the real system and the ideal system for the one-time pad example. We refer to Figure 2.8.

The constructed resource is a secure channel, denoted as SEC (later we also use the symbol $\bullet \longrightarrow \bullet$).¹⁶ A secure channel (for sending a single message from A to B) is a resource system with three interfaces, for a sender A , a receiver B , and an adversary E . A can input a message, B receives the message, and E learns nothing except for the length of the message.

The assumed resources consist of an authenticated channel, denoted as AUT or also as $\bullet \longrightarrow \bullet$, and (in parallel) a shared secret key, denoted as KEY or also as $\bullet \longleftrightarrow \bullet$, which generates a fresh uniform string of a certain length and outputs it to Alice and Bob, while outputting nothing to Eve. The channel AUT takes a message (from a certain message space) at the A -interface and outputs the message both at the B -interface and the E -interface.¹⁷ The real-world system consists of the parallel composition of AUT and KEY (the assumed resource), denoted $[\text{AUT}, \text{KEY}]$ (or, alternatively $[\bullet \longrightarrow \bullet, \bullet \longleftrightarrow \bullet]$), with the converters (protocol systems) otp-enc and otp-dec attached. Here we assume that otp-enc takes as input a message of *variable length*, takes the initial segment of the same length from the key, XORs it to the message, and sends the result over the channel AUT.

¹⁶This notation is explained in the *Information Security* lecture notes. In a nutshell, the $\bullet$ means that the corresponding end-point of the channel is exclusive for the attached entity.

¹⁷In contrast to a secure channel, an authenticated channel is one in which E also learns the message sent by A but, like for a secure channel, E cannot influence the output for B .

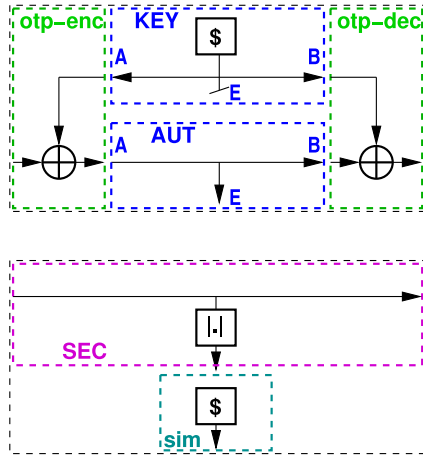


Figure 2.8: The one-time pad (OTP) explained as an example in the constructive cryptography framework. The OTP constructs a secure channel SEC (the constructed resource) from an authenticated channel AUT and a secret key KEY (the assumed resources). There exists a simulator sim which, when attached to interface E of the constructed resource, makes it equivalent to the real-world system with the converters (otp-enc and otp-dec) attached to the assumed resources. Equation (2.2) is an alternative (algebraic) way of describing the systems and stating their equivalence.

In algebraic notation, the real-world system is hence

$$\text{otp-dec}^B \text{ otp-enc}^A [\text{KEY}, \text{AUT}],$$

where the superscript (e.g. A) specifies at which interface the protocol system is connected.

How can we argue that OTP encryption constructs a *secure* channel? After all, in the real-world system, when a message is input, the adversary receives something, namely the ciphertext, while she receives nothing in the ideal world, i.e., in the constructed resource (except for the message length $|m|$, the leakage of which is a feature of the constructed resource). How can we resolve this apparent problem?

2.4.3 The Simulator

Since the ciphertext is statistically independent of the message, we can argue that the adversary is not better off in the ideal world (i.e., when the secure channel is available), compared to the real world where OTP encryption is used. If the secure channel were used, she could *simulate* the ciphertext (she would receive in the real-world system) by herself. More precisely, if a simulator generating a random ciphertext (of appropriate length¹⁸) is attached to the adversary interface of the constructed resource (see Figure 2.8), then the resulting (ideal-world) system is equivalent to the real-world system. Here equivalent means that the entire input-output behavior is exactly the same. In other words, both systems, the systems in the real and the ideal world, would behave in exactly the same manner in any possible environment or application they might be placed in. This means that the real-world system is just as good as the ideal-world system if one assumes that generating a uniform random string is easy for the adversary E .

Written as an equation of systems, the requirement can be stated as

$$\text{otp-dec}^B \text{ otp-enc}^A [\text{KEY}, \text{AUT}] \equiv \text{sim}^E \text{ SEC}. \quad (2.2)$$

2.4.4 Computational Security

Let us now switch to the stream-cipher setting. The security proof for OTP encryption (Section 2.2.4) did not carry over to the stream-cipher setting. In contrast, the above simulation-based security proof does.

The assumed and constructed resources *are the same*, no matter whether information-theoretic or computational security is considered. In the example we consider here, also the same simulator can be used for both settings (OTP and stream-cipher). However, the real-world system with stream-cipher encryption and the ideal-world system with simulator are not identical anymore. But all we want to argue is that they behave essentially identically in any realistic application context, where realistic means that only efficient computations occur. Written as an equation of systems, the requirement can be stated as

$$\text{dec}^B \text{ enc}^A [\text{KEY}, \text{AUT}] \approx \text{sim}^E \text{ SEC},$$

where the notion of systems being *approximately* equivalent is defined via the problem of distinguishing the two systems $\text{dec}^B \text{ enc}^A [\text{KEY}, \text{AUT}]$ and $\text{sim}^E \text{ SEC}$. In asymptotic terms, and informally stated, the construction of SEC from $[\text{KEY}, \text{AUT}]$ is computationally secure if the distinguishing advantage

$$\Delta^D(\text{dec}^B \text{ enc}^A [\text{KEY}, \text{AUT}], \text{sim}^E \text{ SEC})$$

¹⁸The length must be given as an input to the simulator, hence it must be “leaked” by the secure channel.

is negligible for all efficient distinguishers D . However, we will later see ways to capture computational security in a more direct and elegant way.

2.4.5 Composability

An important goal will be that constructions are *composable*. This means, for example, that if one proves an encryption scheme secure, then one should be able to use it in any application that requires a secure communication channel when only an authenticated channel and a secret key is available.

Similarly, when only an insecure channel and a key is available but a certain authentication mechanism constructs an authenticated channel from an insecure channel and a secret key, then the combination of the authentication mechanism and secure encryption should construct a secure channel from an insecure channel and two secret keys.

2.5 Message Authentication Codes

In the previous section we saw that the best encryption scheme can possibly be insecure if the transmitted ciphertexts are not authenticated. In this section we propose a solution: *message authentication*, based on a secret key.

Two relevant questions that will arise are whether the same key used for encryption can also be used for message authentication, and whether the combination of encryption and message authentication is secure if each of the schemes is individually secure (according to some notion).

2.5.1 The Concept

If an adversary can control a communication channel, then she can always delete messages. This cannot be prevented by cryptographic means. But what cryptography can achieve is that a receiver can detect (and ignore) messages *not* originating from a specific sender. This is the goal of message authentication codes (MAC) in a context where sender and receiver share a secret key.

Definition 2.10. A message authentication code (MAC) for message space $\mathcal{M}$, key space $\mathcal{K}$, and tag space $\mathcal{T}$ is a function $f : \mathcal{M} \times \mathcal{K} \rightarrow \mathcal{T}$.

For practicality, one also requires an efficient algorithm for computing the function f .

A MAC is used as follows:

1. The sender computes the MAC for the message and transmits the message together with the (appended) MAC-value.

2. The receiver computes the MAC on the received message and compares it to the received MAC-value. If it is correct, then the message is accepted, otherwise it is rejected.

2.5.2 Security Definition

The game-based security definition for a MAC is obtained by considering the following MAC-forgery game between an adversary and a challenger.

Definition 2.11. *t*-message MAC-forgery game for a MAC f between an adversary and a challenger:

1. The challenger chooses a secret key $k \in \mathcal{K}$ uniformly at random.
2. The adversary can choose up to t messages and receive their MAC-values: For every message m , the challenger returns $f(m, k)$.
3. The adversary chooses a message m' and a MAC-value z . He wins the game if $z = f(m', k)$ and m' was not asked as a query in step 2.

A MAC-function is called *secure* (under a chosen-message attack) if no feasible (adversary) algorithm wins this game with non-negligible probability. One could refine the definition of the above game by also allowing a certain number v of verification queries, where the adversary provides a pair (m, z) and the challenger replies with a binary value indicating whether $f(m, k) = z$. Since such a query can also be done by asking for the MAC $f(m, k)$, this is not a highly relevant generalization.

2.5.3 The CBC-MAC

The so-called CBC-MAC¹⁹ is one of the most widely-used message authentication schemes. The CBC-MAC construction is shown in Figure 2.9 and explained in the figure caption, for the specific choice of instantiating the internal block cipher with AES. A relevant question is whether and why the CBC-MAC is indeed a secure MAC scheme. We will investigate this question later. In a nutshell, we will prove that if AES is replaced by a uniform random function, then the CBC-MAC system is indistinguishable (except with very small advantage) from a uniform random function $\{0, 1\}^* \rightarrow \{0, 1\}^n$, where n is the block length of the block cipher.

2.5.4 Combining Encryption and Authentication

To achieve secure communication over the Internet, one must apply both encryption and a MAC scheme. An important question is how they can be combined in a secure manner. Should one first apply the MAC to the message and

¹⁹CBC stands for cipher block chaining.

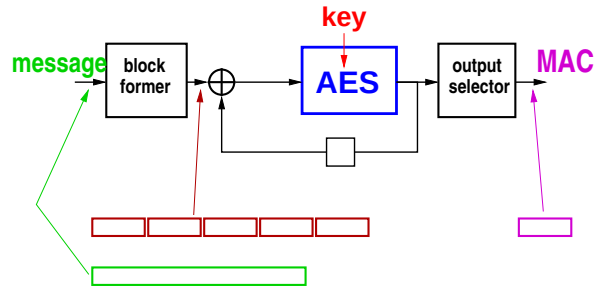


Figure 2.9: CBC-MAC, a message authentication code (MAC) based on a block cipher, for example AES. The message is processed in the block former to obtain a sequence of blocks, for example by appending the encoding of the length and by padding with 0's to obtain a multiple of the block length. Then the sequence of blocks is processed using the PRF (instantiated as AES in the figure). The output selector takes the last output block as the MAC.

then encrypt the result, or should one first encrypt the message and then apply the MAC function to the resulting ciphertext? This is a simple example of the kind of question that comes up when one designs cryptographic protocols.

We briefly describe the construction achieved by applying a MAC-function in the described manner and then discuss how this step can be combined with encryption.

The constructive viewpoint of applying a MAC is illustrated in Figure 2.10. The MAC function is used in the two converters *mac* and *chk*, where *mac* generates the MAC for a given message and appends it to the message, and *chk* verifies the MAC (shown as a function *C* which delivers the message only if the MAC value is correct and otherwise delivers some dummy symbol). This constructs an authenticated channel (not shown in the figure) from an insecure channel *INS* and a secret key *KEY*. Actually, the authenticated channel is weaker than the channel $\bullet \rightarrow$ discussed earlier since the adversary can delete the message, i.e., interrupt the channel. The symbol we will be using for such a channel is $\bullet \rightarrow$. We remark that here *INS* is not yet formally defined and that the claimed construction statement would of course require a proof, which we defer to later.

Now the already discussed construction provided by encryption can be applied, involving a second (independent) key, and resulting in the combined protocol which is often referred to as Encrypt-then-MAC since the message entering the system first gets encrypted and then “MACed”.

Later we will discuss (and prove) why such construction steps can be composed, i.e., why and in which sense the Encrypt-then-MAC paradigm is secure.

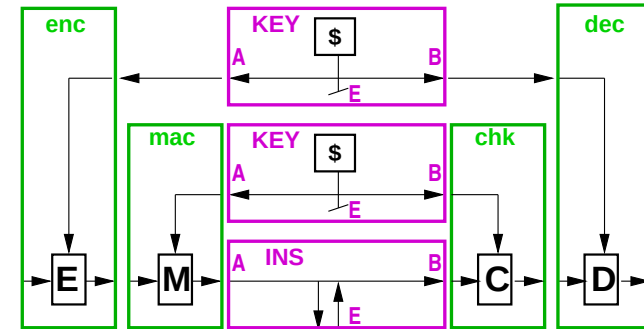


Figure 2.10: The Encrypt-then-MAC paradigm.

2.5.5 Encryption and Authentication with a Single Key

Let us return to a question we asked earlier, namely whether the same key could be used for encryption and message authentication. One could be tempted to argue that an encryption scheme protects the key (as otherwise it would be insecure), and hence re-using the key is safe.

However, the answer to the above question is “no”. One can show (as an exercise) that the secret key can in general not be used for both encryption and for authentication. Such a combined scheme can be insecure even if both involved schemes are individually secure.

A natural idea to avoid the need for sharing two secret keys is to use a PRG to expand a given secret key into a longer bit-string, which can then be split into two keys that can be considered to be independent (in a computational sense). Therefore a single key suffices to perform several tasks that require independent keys. We discuss later how the security of such key expansion can be captured and proved.

2.6 Key Agreement

So far we have ignored the problem of how Alice and Bob obtain a shared secret key in the first place. They could meet physically and exchange a key, or they could send a trusted courier to distribute a secret key. Both these possibilities are highly impractical when Alice and Bob are users or servers on the Internet who wish to establish a secure connection. The Diffie-Hellman protocol provides a remarkable solution.

In this section we do not formally introduce the concept of a key agreement

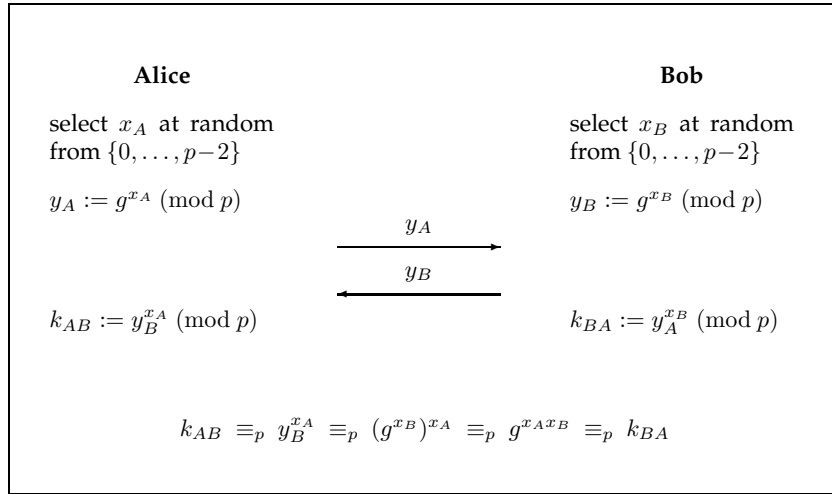


Figure 2.11: The Diffie-Hellman key agreement protocol. The prime p and the generator g are publicly known parameters.

protocol nor discuss a formal security definition.

2.6.1 The Diffie-Hellman Key-Agreement Protocol

In this section we describe the famous key-agreement protocol proposed by Diffie and Hellman in 1976,²⁰ a corner-stone of modern cryptography. It allows two parties, say Alice and Bob, to generate a shared secret key by communicating only over an authenticated (but not confidential) channel to which an adversary has read access. We refer the reader to the Appendix for a repetition of some basic number theory and algebra.²¹

The Diffie-Hellman protocol (see Figure 2.11), as originally proposed²², makes use of exponentiation modulo a large prime p (of, for instance, 2048 bits):

$$x \mapsto g^x \pmod{p}.$$

While $y = g^x \pmod{p}$ can be computed efficiently (see Appendix A.3.5), even if p, g , and x are numbers of several hundred or thousands of digits, computing x

²⁰W. Diffie and M.E. Hellman, New directions in cryptography, *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, 1976.

²¹This material is considered a repetition of material already discussed in the courses *Discrete Mathematics* and *Information Security* (in the Computer Science curriculum at ETH Zurich). It is assumed that the reader is familiar with these schemes and their mathematical foundations.

²²Since then, other versions, for instance based on elliptic curves, have been proposed.

when given p, g , and $y = g^x$ is generally believed to be computationally highly infeasible. This problem is known as (a version of) the *discrete logarithm problem*, which will be discussed later. The prime p and the basis g (e.g. $g = 2$) are public parameters, possibly generated once and for all for all users of the system.

The Diffie-Hellman protocol is shown in Figure 2.11. Alice selects an exponent x_A at random, computes $y_A = g^{x_A}$ modulo p , and sends y_A over an authenticated but otherwise insecure channel to Bob. Bob proceeds analogously, selects an exponent x_B at random, computes $y_B = g^{x_B}$ modulo p , and sends y_B to Alice. Then Alice computes the value

$$k_{AB} := y_B^{x_A} = (g^{x_B})^{x_A} = g^{x_B x_A}$$

modulo p , and Bob computes, analogously,

$$k_{BA} := y_A^{x_B} = (g^{x_A})^{x_B} = g^{x_A x_B}$$

modulo p . The simple but crucial observation is that

$$k_{AB} = k_{BA},$$

due to the commutativity of multiplication (in the exponent). In other words, Alice and Bob arrive at the same shared secret value.

2.6.2 The Mechanical Analog

The Diffie-Hellman protocol can be nicely explained by a mechanical analog, as shown in Figure 2.12. The exponentiation operation (e.g. the operation $x_A \mapsto g^{x_A}$) can be thought of as locking a padlock, an operation that is easy to perform but impossible (in a computational sense) to invert. Note that the padlock in this analog has no key; once locked, it can not be opened anymore. However, a party can remember the open state of the lock (i.e., x_A). Alice and Bob can exchange their locked padlocks (i.e., y_A and y_B), keeping a copy in the open state. Then they can both generate the same configuration, namely the two padlocks interlocked. For the adversary, this is impossible without breaking open one of the two padlocks.

2.6.3 Generalization to Arbitrary Cyclic Groups

The described Diffie-Hellman protocol can be generalized from computation modulo p (i.e., the group $\mathbb{Z}_p^*$) to any cyclic group $G = \langle g \rangle$, generated by a generator g , in which the discrete logarithm problem is computationally hard. The only modifications are that multiplication modulo p is replaced by the group operation of G (and x_A and x_B must hence be chosen from $\{0, \dots, |G| - 1\}$). In practice, one often uses elliptic curves (as the group G) for which the discrete logarithm problem is believed to be even substantially harder than for the group $\mathbb{Z}_p^*$, for comparable group sizes.

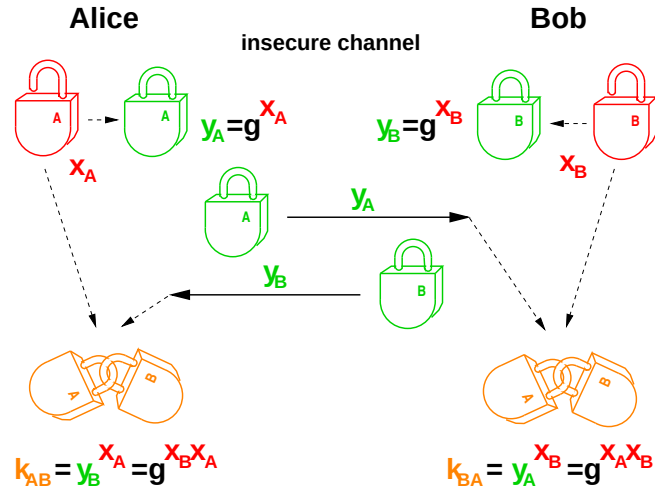


Figure 2.12: Mechanical analog of the Diffie-Hellman protocol.

2.6.4 From a Random Group Element to a Uniform Key

The Diffie-Hellman protocol as described does not yet provide a shared secret key to Alice and Bob because the binary representation of a uniformly random group element is generally not a uniformly random string, actually far from it. For almost all groups, the natural representation is not dense, i.e., only a (small) subset of bitstrings of a certain length actually represent group elements. For the case of $\mathbb{Z}_p^*$, the representation is dense, but one would actually work only in a subgroup of $\mathbb{Z}_p^*$ of size at most $(p-1)/2$, and these elements would not be densely represented.

If one wants to obtain a k -bit secret key, Alice and Bob need to apply a function $\gamma : G \rightarrow \{0, 1\}^k$ mapping group elements to bitstrings. In ad-hoc implementations of the Diffie-Hellman protocol, a fixed function is used, for example a cryptographic hash function. In an implementation designed to be provably secure, the function γ must be a so-called *extractor*, a function which, informally, maps any input with sufficient entropy to an essentially uniform random string. However, an extractor function must be randomized, i.e., it takes as a second input a short uniformly random string, often called *seed*. This seed must be random, but it need not be secret. Hence it can for example be chosen by Alice and sent to Bob (authentically) together with the group element.

Extractors will be discussed later.

2.6.5 Towards Defining Security

A possible security definition could state that it must be computationally infeasible (for an adversary) to compute the shared value $k_{AB} = g^{x_A x_B}$ when given $y_A = g^{x_A}$ and $y_B = g^{x_B}$. This is known as the so-called CDH-problem (see below).

Is this security definition, namely that computing the key is infeasible, sufficient? This raises the more general question of how one can show that a security definition is sufficient or that it is not sufficient? These questions are of crucial importance in cryptography, not just for understanding the Diffie-Hellman protocol. That a security definition is not sufficient can for example be shown by demonstrating an application in which the scheme, even though secure according to the definition, fails nevertheless.

For the Diffie-Hellman protocol, the security definition stating that the key is hard to compute (i.e., the CDH assumption, see below) is not sufficient. Namely, it does not exclude the possibility that part of the secret key can be computed. Is this bad? Yes! One can imagine an application where the generated key is used as a key for one-time pad encryption. Since part of the key is known, part of the message leaks to the adversary.

A stronger security definition could require that the adversary learns essentially nothing about the secret key, in a computational sense. But how could we define “nothing” in a computational sense? One approach is to say that there exists no efficiently computable predicate of the key which can efficiently be guessed better when the protocol execution is observed compared to when the protocol execution cannot be observed. However, such a definition, even if made precise, would still not be sufficient because an application might leak partial information about the key, and in such a case learning anything more about the key should be impossible. For example, even if the adversary learns all but one bit of the key, guessing the last bit should still be hard.

Another natural security definition trying to capture that no information leaks (in a computational sense) is the following, similar to the IND-CPA security definition for symmetric encryption. Given the transcript of the Diffie-Hellman protocol, and assuming that the adversary is also given a value which is either the correct secret value K_{AB} shared by Alice and Bob, or an independent uniformly random group element, she cannot distinguish between the two cases. This is formalized as the DDH-problem in the next section.

This is still not necessarily a satisfactory definition since it is not clear what it really means in an application. A better definition appears to be the following. Executing the protocol should be equivalent (in a computational sense) to an idealized setting where Alice and Bob have access to a random bit-string (the key) to which Eve has no access. This can be formalized in constructive cryptography (see later). It will be possible to show that such a secret key is constructed under the assumption that the DDH-problem is hard (see below).

2.6.6 Underlying Computational Problems

We state three computational problems for a cyclic group G of order $|G| = q$ generated by a generator g (i.e., $G = \langle g \rangle$). As mentioned above, a concrete example is the multiplicative group modulo a large prime p , i.e., $G = \mathbb{Z}_p^*$, with $|G| = q = p - 1$. It is assumed that the group G , the generator g , and the order q are known.

Definition 2.12. The *discrete logarithm (DL)* problem for $G = \langle g \rangle$ is the problem of computing, for a given (uniformly chosen) group element $A \in G$, the exponent $a \in \mathbb{Z}_q$ such that $A = g^a$.

Definition 2.13. The *computational Diffie-Hellman (CDH)* problem for G is the problem of computing, for given (uniformly chosen) group elements $A, B \in G$, the group element C such that $C = g^{ab}$, where $A = g^a$ and $B = g^b$.

Definition 2.14. The *decisional Diffie-Hellman (DDH)* problem for G is the problem of distinguishing, for given elements $A, B, C \in G$, the case where A, B, C are independent and uniformly random in G from the case where A and B are uniformly random and $C = g^{ab}$, where $A = g^a$ and $B = g^b$. (A triple (A, B, C) of the latter type is called a Diffie-Hellman triple.)

The first two problems are games according to Section 2.3 while the last problem is a distinction problem.

For each of these problems one can phrase the assumption that the problem is computationally hard.²³ These assumptions are called the *DL-assumption*, the *CDH-assumption*, and the *DDH-assumption*, respectively.

It is easy to see that the CDH-assumption implies the DL-assumption, and the DDH-assumption implies the CDH-assumption. This is proved by trivial reductions. For example, if one could solve the CDH problem, then one could solve an instance of the DDH problem by simply solving the corresponding CDH problem and checking whether or not the third component of the DDH instance matches.

The CDH assumption is necessary but not sufficient for the Diffie-Hellman protocol to be secure in a meaningful sense. We will see later that the DDH-assumption is necessary and sufficient. However, in the so-called *random oracle model (ROM)*, discussed later, the CDH assumption is sufficient.

Since the DL problem appears more likely to be hard than the CDH problem, and, similarly, since the CDH problem appears more likely to be hard than the DDH problem, it is desirable to prove reductions between these problems. One can actually prove that for essentially all groups the CDH-assumption is equivalent to the DL-assumption, where for solving one instance of the DL problem one needs several calls to an oracle solving the CDH problem.²⁴ In contrast, no reduction from the CDH problem to the DDH problem is known.

²³Note that to define hardness formally (see possibly later), one would need to define an asymptotic family of groups and a specific binary representation of the group elements.

²⁴Ueli Maurer, Towards the equivalence of breaking the Diffie-Hellman protocol and comput-

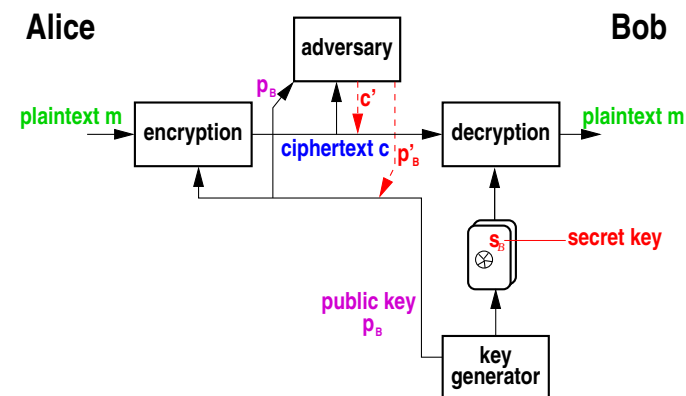


Figure 2.13: Scenario of using public-key encryption. An adversary can try to replace the public key p_B by a fake public key p'_B (which is why p_B must be authenticated) or try to replace the ciphertext c by a fake ciphertext c' (which is why c must be authenticated).

2.7 Public-key Encryption and the RSA System

In this section we take a small detour from our path towards designing a secure communication protocol. This allows us to introduce another basic cryptographic concept, *public-key encryption (PKE)*, as well as the so-called RSA scheme (more precisely, the RSA trapdoor one-way permutation), which will in Section 2.8 also serve as the basis of a *digital signature scheme*.

2.7.1 Concept and Definition

The concept of public-key encryption is shown in Figure 2.13. A user (here Bob) generates a key pair consisting of a *secret key* (also called *private key* or *decryption key*) s_B and the corresponding *public key* (also called *encryption key*) p_B , which he makes public, for example by putting it on his web-page, authenticated by a certificate.²⁵ There is an encryption operation, making use of the public key (but not the secret key) by which any entity (e.g. Alice) can encrypt a message for Bob. Bob (and only he) can decrypt the ciphertext using the secret key.

Informally, a *public-key encryption (PKE) scheme* consists of three efficient algorithms:

- The *key generator* is a probabilistic algorithm which generates a key pair

ing discrete logarithms, *Advances in Cryptology - CRYPTO '94*, Lecture Notes in Computer Science, Springer-Verlag, 1994.

²⁵A certificate is a statement signed by a trusted party, often called a certification authority (CA), which confirms the binding of the public key to a particular entity.

(consisting of a public key and a secret key) according to a **certain probability distribution**.

- The **encryption algorithm** takes as inputs a public key and a plaintext and computes the ciphertext. **Encryption can be probabilistic with fresh randomness for every message.**
- The **decryption algorithm** takes as inputs a secret key and a ciphertext and computes the plaintext.

More formally, we can define a public-key cryptosystem as follows:

Definition 2.15. A **public-key encryption (PKE) scheme**²⁶ for message space $\mathcal{M}$, ciphertext space $\mathcal{C}$, public-key space $\mathcal{P}$, and secret-key space $\mathcal{S}$, consists of:²⁷

- a **key-pair distribution**, a probability distribution over $\mathcal{P} \times \mathcal{S}$,
- a randomized²⁸ **encryption function** $E: \mathcal{M} \times \mathcal{P} \rightarrow \mathcal{C}$,
- a **decryption function** $d: \mathcal{C} \times \mathcal{S} \rightarrow \mathcal{M} \cup \{\perp\}$,²⁹

such that for any message $m \in \mathcal{M}$ and any key pair (p, s) with positive probability, $\Pr(d(E(m, p), s) = m) = 1$ (correctness).

The practicality condition is that efficient algorithms for sampling from the key-pair distribution and for encryption and decryption are given.

2.7.2 Security Definitions for PKE

Informally, security of a PKE-scheme means that it must be computationally infeasible to compute “useful” information about the plaintext for a given ciphertext without knowledge of the secret key.

As for the case of symmetric encryption, “leaking no useful information” can be captured by demanding that an adversary cannot distinguish the ciphertexts of any two messages chosen by her.

One must also specify what type of attack an adversary can perform. In a PKE-scheme, anybody can encrypt messages, hence we do not need to consider an encryption oracle in the definition. But to achieve a high level of security, one can allow the adversary to have access to a decryption oracle. This is summarized in the following bit-guessing problem (see Section 2.3.3), in the literature usually called a game between the adversary and a challenger.

Definition 2.16. **t -ciphertext IND-CCA game for public-key encryption:**

²⁶also called a *public-key cryptosystem* (PKC)

²⁷Note that we are using a capital E for the encryption function because it is a (function-valued) random variable.

²⁸A randomized function is a conditional probability distribution. Here we do not make the randomness needed for key generation and encryption explicit.

²⁹The symbol $\perp$ means that no decrypted message is generated, for example because the ciphertext is invalid in the sense that it does not correspond to any message.

1. The challenger chooses a secret key/public key pair according to the key-pair distribution and sends the public key to the adversary.
2. The adversary can choose up to t ciphertexts and receive (from the challenger) their decryptions under the secret key.
3. The adversary chooses two messages m_0 and m_1 **of the same length**.
4. The challenger chooses a uniformly random bit b , computes the encryption c of m_b , and returns c to the adversary.
5. The adversary can again choose ciphertexts and receive their decryptions, as in step 2, where the total number of ciphertexts (including step 2) is limited to t , and where ciphertext c is excluded.
6. The adversary guesses b (i.e., which message was encrypted) by issuing a guess b' .³⁰

Definition 2.17. If in the above definition no decryption queries are allowed ($t = 0$, i.e., steps 2 and 5 are omitted), then the game is called the **IND-CPA game for public-key encryption**.

Informally, a PKE-scheme is called **IND-CCA secure** (respectively **IND-CPA secure**) if every feasible (adversary) algorithm has only negligible advantage in the respective game.

This IND-CCA security definition seems intuitively very strong, but we do not (yet) know (nor address the question) whether and why this definition is adequate or too strong or too weak in a certain application context. **The answer actually depends on the type of application.**

2.7.3 Using Public-key Encryption

In **order for a public-key cryptosystem to be applicable, the public key must be authenticated. This is achieved by transmitting it over an authenticated channel. Such a channel can, for instance, be obtained by the use of so-called certificates, which will not be discussed in depth in this course.**

Public-key encryption is especially useful in a context with many entities, not just two. For example, B has published his public key authentically, and every entity should be able to send a secret message to B . This is the typical setting when public-key cryptography is used. It corresponds, for example, to a setting where many entities can connect to a server whose public key is publicly known.

Public-key encryption is used primarily for key management,³¹ i.e., the transmitted (encrypted) message is a randomly chosen key, often called a ses-

³⁰Recall from Section 2.3.3 that the adversary’s *advantage* in this game is $2(\Pr(b' = b) - \frac{1}{2})$.

³¹Public-key cryptosystems are also used for various cryptographic protocols (e.g. e-voting protocols) where often a special property (e.g. a homomorphism) of the cryptosystem is exploited.

sion key (or a pre-master secret), that serves as a secret key for symmetric encryption and authentication. One advantage of this approach is the efficiency: symmetric encryption can be implemented two to four orders of magnitude faster than public-key encryption.

Conversely, one can also use a key agreement scheme such as the Diffie-Hellman protocol (combined with symmetric encryption) as a public-key encryption scheme, as described below.

2.7.4 Using the Diffie-Hellman Protocol as a PKE-Scheme

The Diffie-Hellman protocol can be used as a PKE-scheme, as follows. Let the group G and the generator g be fixed and the order $|G| = q$ be known, and let a secure symmetric encryption scheme (e, d) with key space $\mathcal{K}$ (e.g. $\mathcal{K} = \{0, 1\}^k$ for a suitable k) be given. Let γ be a function $G \rightarrow \mathcal{K}$ such that a random element of G is mapped by γ to an essentially random element of $\mathcal{K}$.

- **Key generation** (by Bob): Choose x_B uniformly at random from $\mathbb{Z}_q$. The secret key is x_B , the public key is $y_B = g^{x_B}$.
- **Encryption**: Choose x uniformly at random from $\mathbb{Z}_q$. The ciphertext for a message m is the pair $(g^x, e(m, \gamma(y_B^x), r))$, where r is a uniformly random value from the randomness space of e .
- **Decryption**: left as an exercise.

For the special case where the OTP is used for symmetric encryption (more specifically, the message m is assumed to be represented as a group element and the encryption is simply multiplication in the group with the key, i.e., the ciphertext is $m \cdot k_{AB}$), this scheme is known as the so-called **ElGamal public-key cryptosystem**.

Exercise 2.3. Show that the described PKE-scheme is IND-CPA secure (under the DDH-assumption), but that it is not IND-CCA secure.

2.7.5 Trapdoor One-way Permutations

We now discuss another approach to devising a public-key encryption scheme, based on the following notion.

Definition 2.18. A **trapdoor one-way permutation (TOWP)** for domain $\mathcal{X}$, codomain $\mathcal{Y}$, parameter set $\mathcal{P}$, and trapdoor set $\mathcal{T}$ consists of

- a **parameter-trapdoor distribution**, a probability distribution over $\mathcal{P} \times \mathcal{T}$,
- a function $f: \mathcal{X} \times \mathcal{P} \rightarrow \mathcal{Y}$, and
- a function $g: \mathcal{Y} \times \mathcal{T} \rightarrow \mathcal{X} \cup \{\perp\}$,

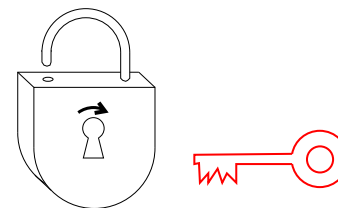


Figure 2.14: Mechanical analog of a trapdoor one-way permutation.

such that for all $x \in \mathcal{X}$ and for all pairs $(p, t) \in \mathcal{P} \times \mathcal{T}$ with positive probability, $g(f(x, p), t) = x$ (correctness).

The practicality condition is that efficient algorithms for sampling from the parameter-trapdoor distribution and for computing f and g are given.

The security requirement is that the following TOWP inversion game must be computationally hard, i.e., no feasible algorithm has non-negligible success probability.

Definition 2.19. **TOWP inversion game** (a game in the sense of Section 2.3.1) between an adversary and a challenger:

1. The challenger generates a pair (p, t) according to the parameter-trapdoor distribution, chooses a uniformly random $x \in \mathcal{X}$, and sends $f(x, p)$ and p to the adversary.
2. The adversary wins the game if she outputs x .

The above description of the game follows the style used in the common cryptographic literature (where typically one directly defines *security* of a TOWP). In a formulation of this game in the sense of Section 2.3.1, one avoids words like challenger and adversary and simply describes the system that internally generates (p, t) according to the parameter-trapdoor distribution, chooses a uniformly random $x \in \mathcal{X}$, and outputs $f(x, p)$ and p . Then it takes an input x' and the game is won if and only if $x' = x$.

Note that the definition of the game does not exclude that partial information about x is computed.

The mechanical analog of a TOWP is shown in Figure 2.14.

2.7.6 The RSA Trapdoor One-way Permutation

This section briefly summarizes the RSA scheme³² already discussed (in the Computer Science curriculum) in the courses *Discrete Mathematics* and *Informa-*

³²R. L. Rivest, A. Shamir, and L. Adleman, A method for obtaining digital signatures and public-key cryptosystems, *Communications of the ACM*, Vol. 21, No. 2, pp. 120–126, 1978.

tion Security, where it was called a public-key encryption scheme. The connection of the RSA TOWP and the related (naïve) public-key encryption scheme will be discussed later.

To understand the RSA TOWP, we need the following simple theorem.

Theorem 2.5. Let G be some finite group (with neutral element 1), and let $e \in \mathbb{Z}$ be a given exponent relatively prime to $|G|$ (i.e. $\gcd(e, |G|) = 1$). The e -th root of $y \in G$, namely $x \in G$ satisfying $x^e = y$, can be computed according to

$$x = y^d,$$

where d is the multiplicative inverse of e modulo $|G|$, i.e.,

$$d \equiv_{|G|} e^{-1}.$$

Proof. We have

$$(x^e)^d = x^{ed} = x^{k \cdot |G| + 1} = \underbrace{(x^{|G|})^k}_{=1} \cdot x = x,$$

where $k = (ed - 1)/|G|$ is an integer. The under-braced term is equal to 1 because of Lagrange's theorem which states that the order of a subgroup of a finite group G divides the group order $|G|$. In particular, the order of every element divides $|G|$. $\square$

When $|G|$ is known, then d satisfying $d \equiv_{|G|} e^{-1}$ can be computed using the extended Euclidean algorithm. No general method is known for computing e -th roots in a group G without knowing its order. This can be exploited to define a TOWP.

Motivated by the Diffie-Hellman protocol also based on modular exponentiation, Rivest, Shamir and Adleman suggested as a possible class of groups the groups $\mathbb{Z}_n^*$, where $n = pq$ is the product of two sufficiently large secret primes p and q . The order of $\mathbb{Z}_n^*$,

$$|\mathbb{Z}_n^*| = \varphi(n) = (p-1)(q-1),$$

can be computed only if the prime factors p and q of n are known.

Exercise 2.4. Show that from n and $\varphi(n)$ one can efficiently compute p and q .

We have seen that

$$\mathbb{Z}_n^* \rightarrow \mathbb{Z}_n^* : x \mapsto x^e \pmod{n},$$

is a permutation and that

$$\mathbb{Z}_n^* \rightarrow \mathbb{Z}_n^* : x \mapsto x^d \pmod{n},$$

is the inverse permutation, where d can be computed according to

$$d = e^{-1} \pmod{\varphi(n)}. \quad (2.3)$$

Exercise 2.5. Show that the RSA function is actually a permutation $\mathbb{Z}_n \rightarrow \mathbb{Z}_n$, making use of the Chinese remainder theorem. There is hence no need to restrict inputs to $\mathbb{Z}_n^*$.

We can hence define the RSA TOWP as follows.

Definition 2.20. RSA trapdoor one-way permutation: The parameter-trapdoor distribution is defined by the following process: A pair of primes, p and q , is chosen according to some distribution, let $n = pq$, then e with $\gcd(e, \varphi(n)) = 1$ is chosen according to some distribution. The parameter is the pair $p = (n, e)$ and the trapdoor is $t = (n, d)$ where d is defined by (2.3). Moreover, $\mathcal{X}$ is a set contained in $\mathbb{Z}_n$ for all possible n . Then f is defined as $f(x, p) = x^e$ in $\mathbb{Z}_n$, and g is defined as $g(y, t) = y^d$ in $\mathbb{Z}_n$.

Note that f and g are efficiently computable. Typically, p and q are chosen such that n has a certain bit-length (e.g. 2048 bits), and quite often p and q are restricted to be of (roughly) equal bit-size. Often, e is chosen to be a small constant, for example $e = 3$, and p and q are chosen to satisfy $\gcd(p-1, e) = 1$ and $\gcd(q-1, e) = 1$.

Let us briefly discuss the security of the RSA TOWP. It is not known whether computing e -th roots modulo n when $\gcd(e, \varphi(n)) = 1$ is easier than factoring n , but it is widely believed that the two problems are computationally about equally hard.³³ Factoring large integers is believed to be computationally infeasible. If no significant breakthrough in factoring is made and if processor speeds keep improving at the same rate as they are now, then a 2048-bit modulus appears to be secure for the next 30 years. However, factoring is known to be efficient on a quantum computer (Shor's polynomial-time algorithm), and RSA and other factoring-based cryptographic schemes will be insecure once sufficiently large quantum computers can be built.

2.7.7 Public-key encryption Using a TOWP

A naïve way to use a TOWP as a PKE-scheme is to let the domain of the TOWP be the message space and to encrypt simply by applying the TOWP, where the parameter of the TOWP is the public key of the PKE-scheme and the parameter/trapdoor pair is the secret key of the PKE-scheme. In terms of mechanical analogs, this corresponds to using the (receiver's) padlock to lock a box, which can then be sent over an insecure (or authenticated) channel. The receiver can decrypt, i.e., remove the lock. If the RSA TOWP is used, the resulting naïve (or plain) version of the public-key cryptosystem is shown in Figure 2.15.

However, it is important to note that naïve RSA encryption, as described above, is deterministic and therefore insecure, for example in applications with

³³One can prove that breaking RSA by a generic algorithm (only performing operations in the ring $\mathbb{Z}_n$) is equivalent to factoring: D. Aggarwal and U. Maurer, Breaking RSA generically is equivalent to factoring, Advances in Cryptology - EUROCRYPT 2009, LNCS, vol. 5479, pp. 36–53, 2009.

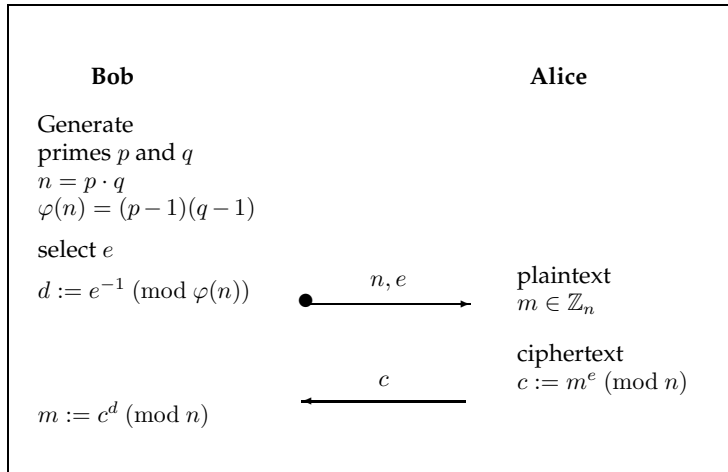


Figure 2.15: The naïve or plain RSA public-key cryptosystem. Bob's public key is the pair (n, e) and his secret key is d . Alice can encrypt a message m , represented as a number in $\mathbb{Z}_n$, by raising it to the e -th power modulo n . Bob decrypts a ciphertext c by raising it to the d -th power. The symbol $\bullet$ indicates that this communication is authenticated.

a small message space because each message in the message space could be tested. Even if the message space is large, the scheme is still not secure. Therefore a reasonable public-key cryptosystem based on RSA is probabilistic, by introducing **appropriate (per-message) randomness**. We might later address the problem of designing a suitable way of using a TOWP as a (secure) PKE-scheme.

2.8 Digital Signatures

We have seen that the Diffie-Hellman protocol is not secure unless the values y_A and y_B are exchanged via authenticated channels. We can not use a MAC scheme to guarantee the authenticity because Alice and Bob do not share a key (yet); generating such a key is the goal of executing the protocol. But one can use digital signatures to authenticate the messages, assuming that Alice and Bob know each other's signature public key. This can be achieved, for example, by certificates.

2.8.1 Concept and Definition

The concept of a digital signature scheme is shown in Figure 2.16. A user (on the left) generates a key pair consisting of a secret signing key (or private key) and the corresponding public signature verification key, which he makes public, for instance by putting it on his web-page, authenticated by a certificate. Knowledge of the secret key enables one to sign messages, while the public key allows one to verify signatures.

Informally, a digital signature scheme consists of three efficient algorithms:

- The **key generator** is a probabilistic algorithm which generates a key pair, consisting of a **signing key** (or secret key or private key) and a **signature verification key** (or public key), according to a certain probability distribution.
- The **signing algorithm** takes as inputs a signing key and a message and computes the signature for the message. **Signing can be probabilistic.**
- The **signature verification algorithm** takes as inputs a signature verification key, a message, and a signature, and outputs a bit (which can be interpreted as "accept" or "reject").

More formally, we can define a digital signature scheme as follows:

Definition 2.21. A **digital signature scheme (DSS)** for message space $\mathcal{M}$, signature set $\mathcal{S}$, verification-key set $\mathcal{V}$, and signing-key set $\mathcal{Z}$, consists of:

- a **key-pair distribution**, a probability distribution over $\mathcal{V} \times \mathcal{Z}$,
- a **signing function**³⁴ $\sigma: \mathcal{M} \times \mathcal{Z} \rightarrow \mathcal{S}$,
- a **verification function** $\tau: \mathcal{M} \times \mathcal{S} \times \mathcal{V} \rightarrow \{0, 1\}$,

such that signatures are correctly verified with probability 1 (correctness), i.e., $\tau(m, \sigma(m, z), v) = 1$ for all (v, z) with positive probability and for all $m \in \mathcal{M}$.

The practicality condition is that efficient algorithms for sampling from the key-pair distribution and for signing and verification are given.

2.8.2 Security Definition

Security of a DSS is defined via the following game (a game in the sense of Section 2.3.1), where, as usual, security means that no feasible (adversary) algorithm has a non-negligible success probability. We describe the game in the usual terminology of the cryptographic literature, and it is immediate to phrase it as a system in the sense of Section 2.3.1.

Definition 2.22. **Signature forgery game for t messages between an adversary and a challenger:**

³⁴This function could also be randomized.

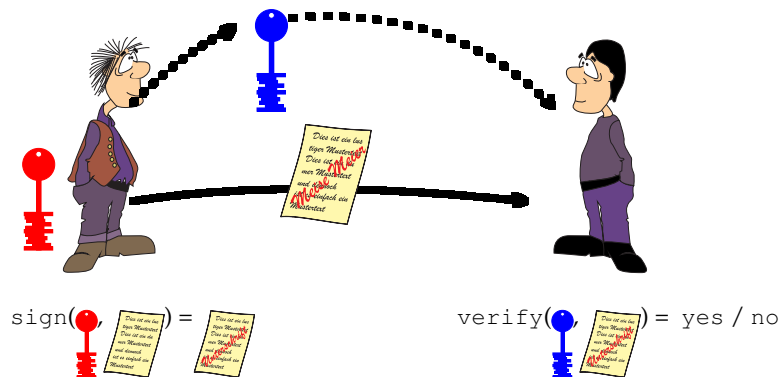


Figure 2.16: Illustration of a digital signature scheme.

1. The challenger samples (v, z) from the key-pair distribution and sends v to the adversary.
2. The adversary can ask at most (arbitrary) messages to be signed by the challenger (i.e., the adversary has access to a signing oracle).
3. The adversary chooses a message m' and a signature s' . He wins the game if s' is a correct signature for m' and m' was not asked as a query in step 2.

2.8.3 An RSA-based Signature Scheme

The RSA scheme (or, more generally, any TOWP) can also be used for generating digital signatures. One exploits the fact that anybody knowing the public key can compute e -th powers, whereas only the owner of the secret key can compute e -th roots.

The naïve approach would hence be to define the signature z for a message m (represented as an element of $\mathbb{Z}_n$) as the e -th root of m , i.e., $s = m^{1/e} \equiv m^d$. However, such a scheme would be insecure. One attack exploits the multiplicative property of RSA (namely, we have $(mm')^d = m^d m'^d$). One can hence forge the signature for the message mm' (the product is modulo n) if one has obtained the signatures for m and m' .

To turn the RSA-TOWP into a secure signature scheme, one must use a carefully designed encoding of the message into an element of $\mathbb{Z}_n$. This will not be discussed here.

Another problem is that one wants to sign long messages, not just messages that can be represented as a number modulo n . A natural approach to encoding messages is therefore to apply a so-called cryptographic hash function h (see Section 2.8.4 below), to the message before applying the d -th power operation: Alice's signature s for a message m is

$$s := (h(m))^d \pmod{n}.$$

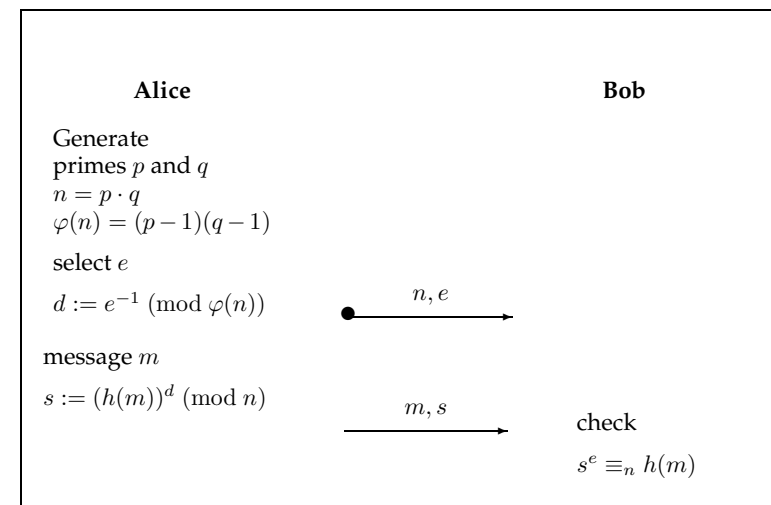


Figure 2.17: The full-domain hash (FDH) digital signature scheme based on the RSA TOWP.

A signature z can be verified by checking the equation

$$s^e \equiv_n h(m).$$

This signature scheme (see Figure 2.17) is sometimes referred to as **full domain hash (FDH)**.

A necessary security requirement is that it must be infeasible to find two messages m and m' with the same hash value, i.e., such that $h(m) = h(m')$. (Such a hash function h is so-called **collision-resistant**, see below). Otherwise an attacker could generate two message hashing to the same value, have one of them signed, and thereby also have a signature for the other message.

However, **collision-resistance is not sufficient for the FDH signature scheme to be secure**. If the hash function were replaced by a purely theoretical model of a truly random function (a so-called *random oracle*) then the security of the FDH signature scheme can be proved. One talks about a security proof in the so-called *random oracle model* (ROM). Unfortunately, one can show that there cannot exist a hash function that “behaves like” a random oracle. This means that a proof in the ROM cannot be carried over into a security proof when a concrete hash function is used, no matter how cleverly it is designed.

2.8.4 Collision-Resistant Hash Functions

Definition 2.23. A hash function is a function $h : D \rightarrow R$ where $|D| \gg |R|$, typically $D = \{0, 1\}^*$ and $R = \{0, 1\}^k$ for some suitable k , e.g. $k = 160$.

Hash functions (of course, efficiently computable ones) have many applications in computer science, including databases. The purpose of a hash function is to assign a (in a certain sense) unique short tag to an arbitrarily long bit-string. In non-cryptographic applications it should be very unlikely that two different bit-strings occurring in the application map to the same tag. In cryptography it is important that the uniqueness is guaranteed even in presence of a malicious adversary trying to provoke such a collision.

Definition 2.24. Collision-finding game for a hash function h between an adversary and a challenger:

1. The adversary can obtain the hash value $h(x)$ for any argument x of his choice (i.e., he has access to an h -hashing oracle).³⁵
2. The adversary outputs a pair (x, x') of messages. She wins the game if $x \neq x'$ and $h(x) = h(x')$.

We make a few remarks about these definitions:

- Note that a typical value z in the range of h has many preimages $x \in D$ with $h(x) = z$. What the definition asks for is only that it be difficult to find two such values, not that they do not exist.
- Definition 2.25 makes no sense for a fixed hash function h (even though one sometimes finds it stated in the literature for a concrete hash function). The reason is that a fixed instance of a computational problem cannot be hard, only a computational problem with a large instance set can be hard. More concretely, for a fixed h , there exists (of course) an algorithm that outputs a collision for h , namely a collision fixed in its code.³⁶ For this reason one often sees the following definition.

Definition 2.25. (Informal.) A parameterized hash function family $\{h_c : c \in \mathcal{C}\}$ (e.g. $\mathcal{C} = \{0, 1\}^s$ for some s) is collision-resistant if no feasible algorithm wins the above collision-finding game for a uniformly random $c \in \mathcal{C}$ (given to the algorithm) with non-negligible success probability.

For a hash function with k -bit output one can find collisions in time $O(2^{k/2})$, due to the so-called birthday paradox. Some well-known hash functions are SHA-1 and MD5. There have been some recent impressive successes in the cryptanalysis of these hash functions, leading to severe potential problems in

³⁵Alternatively, since a description of h is known, this first step can also be eliminated since the adversary can by herself compute hash values.

³⁶This does not mean that anybody knows such an algorithm.

some applications. These attacks reduce the number of operations needed for finding a collision considerably below the $O(2^{k/2})$ required for a brute-force attack. In 2015, the U.S. National Institute of Standards and Technology (NIST) has standardized a hash function under the name SHA-3 after a serious scientific competition for hash function proposals. SHA-3 is generally considered to be secure. Note that all these hash functions are not defined as families and hence are not collision-resistant in the sense of Definition 2.25.

2.8.5 Hash-then-Sign

There are other ways (not discussed here) for obtaining a signature scheme from the RSA-TOWP or from other principles. We do not discuss how to design such a scheme that is provably secure if the RSA-TOWP is secure, and we just mention that such methods appearing in standards are not provably secure.

Such a construction usually allows only to sign messages of a fixed length (for example 256 bits). Assume hence that we are given a secure signature scheme for fixed-length messages. In practice one wants to sign messages of arbitrary length. Again, a natural approach to obtaining a signature scheme for messages of arbitrary length is to simply hash the message and then to apply the basic signature scheme to the hash value. This is known as the *hash-then-sign* principle.

One can prove (as an exercise) that if the basic signature scheme is secure and the hash function is collision-resistant, then the *hash-then-sign* principle yields a secure signature scheme. More precisely, one can show that any algorithm for forging signatures can either be used to find a collision for the hash function or to find a forgery for the basic signature scheme.

Note that the full-domain hash (FDH) signature scheme for a TOWP and the hash-then-sign principle discussed here are different concepts, despite the fact that in both the message is first hashed. The TOWP is not by itself a signature scheme, which is why FDH is generally not provably secure without making additional assumptions.

2.9 Wrapping up this Chapter

In this chapter we have seen a number of fundamental cryptographic concepts as well as some concrete schemes. We have discussed some security definitions for these schemes. For the one-time pad, and also informally for MACs, we have illustrated the constructive viewpoint we will be using later in this course.

The story of the chapter results in a concrete protocol for establishing a secure channel between two entities Alice and Bob, assuming Alice and Bob have an authenticated copy of their communication partner's signature verification key, i.e., we assume the initial availability of single-use authenticated channels

from Alice to Bob and from Bob to Alice. These channels are in practice constructed by the use of **certificates**.

In summary, and informally described, the protocol works as follows. We only consider **the case where a single message is sent securely**, but by appropriate changes one obtains a protocol for sending an arbitrary number of messages.

1. **Alice and Bob execute the Diffie-Hellman protocol**, where the two Diffie-Hellman messages are signed **using the RSA-based FDH signature scheme, for a specific hash function h** . The signatures on the received messages are verified, and **if the verification fails for a party, that party aborts the protocol**.
2. **Alice and Bob each use the resulting key to generate two keys, using a PRG g** .
3. **Alice expands the first of these keys, using the PRBG based on iterated application of g , to obtain a key as long as the message to be transmitted**.
4. Alice uses the **one-time pad** with the generated key to encrypt the message.
5. Alice uses a **MAC scheme, with the second key** from step 2, to compute a MAC **for the ciphertext**.
6. **Alice sends the ciphertext and the MAC** to Bob over an insecure channel.
7. **Bob verifies the received MAC** on the received ciphertext, using the second key. If the check fails, he aborts.
8. **Bob expands the first of the two keys**, using the PRBG.
9. Bob decrypts the ciphertext and outputs the message.

This protocol can be proved to be secure under certain assumptions. **Secure here means that it constructs a secure communication channel according to the constructive cryptography paradigm**. The assumptions are:³⁷

- The **DDH-problem is hard for the group used in the Diffie-Hellman protocol**.
- The **RSA function is a TOWP** (according to Definition 2.18).
- The **function g is a PRG** (according to Definition 2.3).
- The **MAC scheme is secure** (according to Definition 2.10).

Turned around, this means that any distinguisher that could distinguish the “ideal setting” from the “real setting” could be used (by a reduction) to break one of the above assumptions.

³⁷Of course, one also needs to assume that the certification authority works correctly, that the computers on which the protocol is running are not compromised, etc.

However, this only holds in an idealized setting where the hash function h is replaced by a random oracle. Such a proof is called a proof in the random oracle model.

So what remains to be done? The following questions, most of which will be addressed in this course, are now well-motivated. Some of the questions are concrete, and some are of a general conceptual type.

- What kind of mathematical objects are computational problems, methods for solving them, and reductions? And how do we measure how well a method solves a problem?
- Can the assumption that a hash function behaves like a random oracle be made precise, and can there be a hash function with this property? We will actually see that the answer to the latter question is negative, which means that the random oracle model is in a sense not sound.
- How can we prove that the FDH signature scheme is secure if the underlying function (e.g. RSA) is a TOWP (and h is a random oracle)?
- Can one show that RSA is a TOWP, based on the assumption that factoring is hard?
- How can we argue that the DDH problem is indeed hard. Which groups must be excluded because the DDH problem is easy?
- Why is the continued PRG-expansion a secure PRBG?
- How can one construct a PRG from weaker assumptions, for example that a certain function is hard to invert?
- Why is the CBC-MAC scheme secure if the underlying block cipher is secure, and are there better MAC schemes?
- How can we develop a framework in which the individual security statements for the various schemes used can naturally be composed? In other words, how can we provably exclude a failure of our intuition or hidden effects resulting from an unexpected interference between the various schemes used?
- Which other tasks, besides secure communication, can be solved by cryptography?

In order to address these and many other questions, we need to proceed with mathematical precision and rigor.

Chapter 3

Discrete Systems

In this chapter we introduce *discrete systems*, one of the most basic mathematical object considered in cryptography and in other areas of computer science. We discuss the following mathematical types: deterministic discrete systems, probabilistic discrete systems, and their behavior.

3.1 Introduction

3.1.1 Reactive Systems

We consider reactive systems that proceed in rounds, in each round taking an input and producing an output. Essentially every kind of several-round discrete system, in particular a cryptographic system, is of this form. More precisely, an $(\mathcal{X}, \mathcal{Y})$ -system S takes inputs $X_1, X_2, \dots$ (from some set $\mathcal{X}$) and generates, for each new input X_i , an output Y_i (from some set $\mathcal{Y}$).¹ The output Y_i depends (possibly probabilistically) on the current input X_i and on the internal state. A system can be finite, i.e., it can stop answering inputs at a certain point. A special case is that a system answers a fixed number q of input queries. The behavior of a system can be deterministic or probabilistic.

Example 3.1. A system that computes a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ for every new input, i.e., $Y_i = f(X_i)$ for $i \geq 1$, is an $(\mathcal{X}, \mathcal{Y})$ -system.

Definition 3.1. A $\mathcal{Y}$ -source is a system that takes trigger inputs (with a unary alphabet, e.g. $\mathcal{X} = \{\diamond\}$) and produces (for each trigger input) an output in $\mathcal{Y}$ with a certain distribution. A source can be memoryless or have memory.

¹It is not a relevant restriction to consider fixed input and output alphabets. This allows to model also systems where inputs and outputs come from different alphabets for different i , as discussed below.

Example 3.2. A (memoryless) source outputting (when triggered) independent uniform random variables from some domain is sometimes called a *beacon*. Usually the domain is the set of n -bit strings for some n , and such a beacon will be denoted as $\mathbf{B}_n$, where $\mathbf{B}_1$ is often called a *binary symmetric source* or sometimes a *random bit generator*.

Example 3.3. The system that outputs a uniform random n -bit string (once, upon a trigger input) is usually denoted as $\mathbf{U}_n$. (This is a $\{0, 1\}^n$ -source restricted to one query.)

Example 3.4. Consider a system which, for some domain $\mathcal{X}$ and some finite range $\mathcal{Y}$, works as follows. For every fresh input it replies with a fresh uniformly random value, but it replies consistently if a previously given input is repeated. This seems like a precise description of a system, but nevertheless there are two quite different interpretations. First, the outputs can be thought of as being generated on the fly, as just described. Second, if $\mathcal{X}$ is finite, the system can be thought of as consisting of a (once and for all) randomly selected function table embedded in the system.

We point out that in this example we have discussed three different objects, which can all be understood as some kind of system, and in this chapter we want to understand how to think mathematically about them. Do all these objects mean the same thing or do they mean different things? We want to understand which mathematical types are involved and how the three objects relate to each other.

3.1.2 Descriptions vs. Mathematical Types

When defining and studying mathematical objects, one must clearly distinguish between the *mathematical type* of an object and *descriptions* of these objects in a certain description language. The type is unique, but for an object there are generally many different ways to describe them. For example, a graph is a relation (mathematical type), and possible descriptions include the adjacency matrix, neighborhood lists, or the set of vertices and the set of edges. This leads to the natural questions of when two descriptions are equivalent in the sense that they describe the same mathematical object.

Some ways of describing a deterministic $(\mathcal{X}, \mathcal{Y})$ -system are, with increasing level of formality:

- by a textual description of how the system operates.
- by a program (e.g. pseudo-code) with internal state that can be invoked multiple times, each time with the next input.
- by a state space S , an initial state $s_0 \in S$, a next-state function $\phi: \mathcal{X} \times S \rightarrow S$, and an output function $\psi: \mathcal{X} \times S \rightarrow \mathcal{Y}$. This description is to be interpreted as follows. Upon the i th input $x_i \in \mathcal{X}$, the new state is computed as $s_i = \phi(x_i, s_{i-1})$ and the output is computed as $y_i = \psi(x_i, s_{i-1})$.

We now address the question of what the mathematical type of an $(\mathcal{X}, \mathcal{Y})$ -system is. We will first consider deterministic, then probabilistic systems.

3.2 Deterministic Discrete Systems

3.2.1 Definition

Consider a deterministic $(\mathcal{X}, \mathcal{Y})$ -system with input alphabet $\mathcal{X}$ and output alphabet $\mathcal{Y}$. For a first input x_1 , the output y_1 is a function of x_1 . For a second input x_2 , the output y_2 is a function of x_1 and x_2 . For a third input x_3 , the output y_3 is a function of x_1, x_2 and x_3 , and so on. These functions determine how the system operates, independently of how the system is described internally (e.g. the state space). The system is therefore completely characterized by the definition of an output symbol for every input sequence in $\mathcal{X}^* \setminus \{\epsilon\}$, where the empty string ϵ is excluded since the system gives no output before an input is given. Such a system s can hence be understood as a function

$$s: \mathcal{X}^* \setminus \{\epsilon\} \rightarrow \mathcal{Y}.$$

More generally, at a given point of the execution, a system generally accepts only inputs from a *subset* of $\mathcal{X}$. For example, at some point the system takes an input bit (e.g. some kind of selection bit), at another point it accepts as input a bitstring of arbitrary length (e.g. a message), and at yet another point it may accept only a unary (trigger) input. This means that the function s is generally only partially defined, and the input alphabet $\mathcal{X}$ comprises all possible input values that can ever occur.²

Formally, a deterministic discrete $(\mathcal{X}, \mathcal{Y})$ -system is defined as follows:

Definition 3.2. A deterministic discrete $(\mathcal{X}, \mathcal{Y})$ -system s (an $(\mathcal{X}, \mathcal{Y})$ -DDS for short) is a partial function³

$$s: \mathcal{X}^* \setminus \{\epsilon\} \rightarrow \mathcal{Y}$$

with domain (denoted $\text{dom}(s)$) closed under taking prefixes. A system s is called *finite* if $\text{dom}(s) \subseteq \mathcal{X}^n$ for some $n \in \mathbb{N}$.

The interpretation of this definition is that for inputs $x_1, x_2, x_3, \dots$, the outputs $y_1, y_2, y_3, \dots$ are defined by⁴

$$\begin{aligned} y_1 &= s(x_1), \\ y_2 &= s(x_1, x_2), \\ y_3 &= s(x_1, x_2, x_3), \dots \end{aligned}$$

²Without essential loss of generality we can think of $\mathcal{X}$ as being the set of finite-length bitstrings $\{0, 1\}^*$, where often the allowed input alphabet is only a subset of $\{0, 1\}^*$, often a finite subset.

³One can describe a partial function by a fully-defined function with an extra symbol *undef* (for “undefined”) in the codomain. Note, however, that this is only a *description* of the partial function; mathematically they are different objects.

⁴For simplicity we write $s(x_1, \dots, x_k)$ instead of $s((x_1, \dots, x_k))$.

If s is undefined for $(x_1, \dots, x_k)$, we can set

$$y_k = \perp,$$

where the special symbol $\perp$ is not in $\mathcal{Y}$.

One can think of the tree associated with a system s , where the edges are labeled with input values and each node corresponds to an $\mathcal{X}$ -sequence in the domain of s and is labeled with the output value for that input sequence. The degree of a node corresponds to the size of the input alphabet at that point. The input alphabet can also be *empty*; in this case the node is a leaf in the tree and the system is considered to *stop*. The prefix-freeness condition of Definition 3.2 assures that a system can not be undefined for a certain input sequence $(x_1, \dots, x_k)$ but defined for an extension of $(x_1, \dots, x_k)$. A finite system corresponds to a finite-depth tree.

If an input outside of the (currently) allowed domain is given, the system does not “see” this, i.e., it does not change its state. Consequently, if later an allowed input is given, it will reply as if no undefined input had been given. An environment interacting with a system “sees” that a system gives no reply and can, for example, ask a different query.⁵ This is captured in the following definition.

Definition 3.3. Let s be an $(\mathcal{X}, \mathcal{Y})$ -DDS. The corresponding fully defined DDS, denoted $s^\perp$, is the $(\mathcal{X}, \mathcal{Y} \cup \{\perp\})$ -DDS defined as follows:

$$s^\perp(x_1, \dots, x_k) = \begin{cases} s(z_1, \dots, z_m, x_k) & \text{if } s(z_1, \dots, z_m, x_k) \text{ is defined} \\ \perp & \text{else.} \end{cases}$$

where $(z_1, \dots, z_m)$ results from $(x_1, \dots, x_{k-1})$ by deleting all the x_i (starting with x_1), which make s undefined.⁶

3.2.2 Parallel Composition

Given a tuple $(s_1, \dots, s_n)$ of systems, one can consider their parallel composition, a system denoted $[s_1, \dots, s_n]$, where, informally, an input can be given to any of the subsystems s_i and the output is the reply of that subsystem. The other subsystems are not affected by such a query. An input to the system $[s_1, \dots, s_n]$ is a pair (i, r) of an index⁷ $i \in [n]$ selecting a subsystem and an input value $r \in \mathcal{X}_i$ to the selected subsystem. A subsystem s_i only “sees” the inputs with first component i . More formally we have:

⁵Note that an environment need not necessarily “know” in advance whether a certain input is defined.

⁶This can be made precise, for example by a recursive definition. For example, if s is defined for all binary input sequences but $\mathcal{X}$ also contains the symbol 2, then $s^\perp(0, 2, 1, 2, 1) = s(0, 1, 1)$ and $s^\perp(0, 2, 1, 2, 1, 2) = \perp$ because $s(0, 1, 1, 2)$ is undefined.

⁷Let $[n]$ denote the set $\{1, \dots, n\}$.

Definition 3.4. Consider a tuple $(s_1, \dots, s_n)$ of DDS, where s_i is a $(\mathcal{X}_i, \mathcal{Y}_i)$ -DDS. Their parallel composition, denoted $[s_1, \dots, s_n]$, is the DDS with input alphabet

$$\bigcup_{i=1}^n (\{i\} \times \mathcal{X}_i) = \{(i, x) \mid i \in [n], x \in \mathcal{X}_i\}$$

and output alphabet $\mathcal{Y} = \bigcup_{i \in [n]} \mathcal{Y}_i$, defined as follows. If x_k is of the form $x_k = (i_k, r_k)$, then

$$[s_1, \dots, s_n](x_1, \dots, x_k) = s_{i_k}((x_1, \dots, x_k)_{|i_k}),$$

where $(x_1, \dots, x_k)_{|i_k}$ is the sequence $(x_1, \dots, x_k)$ restricted to the second components r_m of the $x_m = (i_m, r_m)$ for which $i_m = i_k$.

For example, $((3, a), (2, b), (1, a), (2, c), (4, a), (2, a))|_2 = (b, c, a)$.

If some of the systems $s_1, \dots, s_n$, are only partially defined, it is easy to see that the fully defined system $[s_1, \dots, s_n]^\perp$ is given by

$$[s_1, \dots, s_n]^\perp = [s_1^\perp, \dots, s_n^\perp].$$

3.2.3 Interfaces, and Resources as Discrete Systems

One can partition the input alphabet $\mathcal{X}$ of a DDS into disjoint alphabets $\mathcal{X}_1, \dots, \mathcal{X}_n$ (with $\mathcal{X}_i \cap \mathcal{X}_j = \{\}$ for $i \neq j$), i.e.,

$$\mathcal{X} = \mathcal{X}_1 \cup \dots \cup \mathcal{X}_n,$$

and interpret the alphabets $\mathcal{X}_1, \dots, \mathcal{X}_n$ as corresponding to n interfaces named $1, \dots, n$. An input $x \in \mathcal{X}_i$ is interpreted as being given at interface i .

Analogously, one can understand a system with several interfaces as a DDS (with a single interface). As an example of systems with many interfaces, recall the concept of a resource introduced informally in the previous chapter. **A resource is a system with interfaces to one or more parties.**

A (deterministic) resource s with interface set $\mathcal{I}$ can be modeled as a DDS with an input alphabet that is the union of mutually disjoint input alphabets for each interface. In other words, an input to s determines both the value x and the interface i to which the value is to be input. A natural choice is to let the input alphabet of s consist of pairs $(i, x) \in \mathcal{I} \times \mathcal{X}$, where the first component $i \in \mathcal{I}$ indicates the interface to which the input is given, and the second component $x \in \mathcal{X}$ denotes the input value (at that interface).

Definition 3.5. A (deterministic) resource s with interface set $\mathcal{I}$, input alphabet $\mathcal{X}$ and output alphabet $\mathcal{Y}$ is a $(\mathcal{I} \times \mathcal{X}, \mathcal{Y})$ -DDS.

Different input alphabets for the different interfaces can be modeled as explained in the previous section. The output is understood to be given at the same interface where the input was given (but this interpretation will be relevant only later).

3.3 Environments and Transcripts

An $(\mathcal{X}, \mathcal{Y})$ -system s is used in an environment e which provides the inputs x_i and receives the outputs y_i of s . This results in a *transcript* of the form $x_1, y_1, x_2, y_2, x_3, y_3, \dots$

Hence, an environment is like a $(\mathcal{Y}, \mathcal{X})$ -system, except that it outputs the first value x_1 before receiving an input, which means that the output is defined also for the empty input string ϵ . Moreover, the environment “sees” if the system does not reply (because the input was undefined), i.e., $\perp$ is in the input alphabet of an environment.

Definition 3.6. A deterministic discrete environment e for an $(\mathcal{X}, \mathcal{Y})$ -DDS, a $(\mathcal{Y}, \mathcal{X})$ -DDE for short, is a function

$$e: (\mathcal{Y} \cup \{\perp\})^* \rightarrow \mathcal{X} \cup \{\perp\},$$

where the symbol $\perp$ means that the environment stops its execution.⁸

The transcript $x_1, y_1, x_2, y_2, x_3, y_3, \dots$ is generated by alternating the invocation of e and of s . Actually, we need to consider the fully defined system $s^\perp$ to capture that the environment “sees” when the system s does not reply because it is undefined:

$$\begin{aligned} x_1 &= e(\epsilon), \\ y_1 &= s^\perp(x_1), \\ x_2 &= e(y_1), \\ y_2 &= s^\perp(x_1, x_2), \\ x_3 &= e(y_1, y_2), \\ y_3 &= s^\perp(x_1, x_2, x_3), \dots \end{aligned}$$

Definition 3.7. The transcript of a system s in environment e , denoted $tr(s, e)$, is the (possibly infinite) sequence $x_1, y_1, x_2, y_2, x_3, y_3, \dots$ defined, for $i \geq 1$, by

$$x_i = e(y_1, \dots, y_{i-1})$$

and

$$y_i = s^\perp(x_1, \dots, x_i).$$

If $e(y_1, \dots, y_{i-1}) = \perp$, the transcript ends with y_{i-1} .

⁸Alternatively, one could consider the environment to be a partially defined function, replacing $\perp$ by “undefined”.

3.4 Converters and Constructing Systems from Systems

3.4.1 Introduction

The idea of using a system to construct another (better) system is universal and is used in many scientific disciplines. In this section we investigate this paradigm for discrete systems.

A discrete $(\mathcal{X}, \mathcal{Y})$ -system s can be used to construct another discrete system, say a $(\mathcal{U}, \mathcal{V})$ -system. For example, when given a system s (e.g. defined by a program) that solves a certain computational problem, then one may want to construct a system t that solves another computational problem. This transformation from s to t is generally called a reduction.

Another very simple example, which however involves probabilistic systems discussed only later, is a system s that outputs a fresh random bit for every (unary) input, where one wants to construct a system t that outputs a uniform ℓ -bit string for every (unary) input. This is trivially achieved as follows: Upon each input, one invokes s ℓ times and outputs the concatenation of these ℓ bits.

3.4.2 Definition of Deterministic Converters

The object which transforms (or converts) an $(\mathcal{X}, \mathcal{Y})$ -system into a $(\mathcal{U}, \mathcal{V})$ -system is called a *converter*. Informally, one can think of a converter, let's call it α ,⁹ as a system with two sides, an outside and an inside. The inside is connected to the given $(\mathcal{X}, \mathcal{Y})$ -system, say s , and the outside of the converter α serves as the interface of the combined system. When an input $u \in \mathcal{U}$ is given (interpreted as being input at the outside), the converter invokes the system at the inside a finite number of times (zero times is allowed) and then returns an output $v \in \mathcal{V}$ at the outside. The resulting system, denoted αs , is a $(\mathcal{U}, \mathcal{V})$ -system.

A bit more precisely, a converter α is a DDS whose input alphabet at the beginning is $\mathcal{U}$. Upon an input $u \in \mathcal{U}$, the converter α produces an output, either of the form (in, x) (intended as input $x \in \mathcal{X}$ for the system s connected at the inside) or of the form (out, v) (interpreted to be output as $v \in \mathcal{V}$ at the outside). In other words, the output alphabet is

$$\{(out, u) | u \in \mathcal{U}\} \cup \{(in, y) | y \in \mathcal{Y}\}$$

and the flag in/out indicates whether the value is meant to be for the inside or the outside. After an output of the form (in, x) , the input alphabet is (a subset of) $\mathcal{Y}$, and the converter α takes an input $y \in \mathcal{Y}$ from the system it is connected to. After an output of the form (out, v) , the defined input alphabet is (a subset of) $\mathcal{U}$. This input value $u \in \mathcal{U}$ is interpreted as being given at the outside.

⁹Converters typically are denoted by Greek letters (e.g. α or β) or short acronyms in small sans-serif font (e.g. `enc` or `dec`).

Definition 3.8. A *deterministic discrete converter* for converting an $(\mathcal{X}, \mathcal{Y})$ -DDS into a $(\mathcal{U}, \mathcal{V})$ -DDS (or a $((\mathcal{U}, \mathcal{V}), (\mathcal{X}, \mathcal{Y}))$ -DDC for short) is a

$$(\mathcal{U} \cup (\mathcal{Y} \cup \{\perp\}), (\{out\} \times \mathcal{V}) \cup (\{in\} \times \mathcal{X})) \text{ -DDS}$$

whose input alphabet at the beginning is $\mathcal{U}$.¹⁰ After an output (in, x) the input alphabet is $\mathcal{Y}$, and after an output of the form (out, v) the input alphabet is $\mathcal{U}$. There is a finite upper bound on the number of consecutive outputs of the form (in, x) .

Definition 3.9. The application of a $((\mathcal{U}, \mathcal{V}), (\mathcal{X}, \mathcal{Y}))$ -DDC α to a $(\mathcal{X}, \mathcal{Y})$ -DDS s yields the $(\mathcal{U}, \mathcal{V})$ -DDS, denoted as αs , described (informally) above. Upon an input $u \in \mathcal{U}$ to αs , input u is given to α . Whenever α outputs (in, x) , then x is given as input to $s^\perp$, and if $y \in \mathcal{Y} \cup \{\perp\}$ is the output of $s^\perp$, then y is given as input to α . Whenever α outputs (out, v) , then v is defined to be the output of αs for the most recent input $u \in \mathcal{U}$ to αs .

We do not give a completely formal definition of the application of a converter to a system. Formally, one would have to show that the described object αs is indeed a $(\mathcal{U}, \mathcal{V})$ -DDS. Intuitively, this is obvious.

3.4.3 Filters

A special type of converter, which we call a *filter*, is one that restricts the domain $\text{dom}(s)$ of a DDS s to a subset of $\text{dom}(s)$ (again closed under taking prefixes). If we denote the filter by φ , then φs is a DDS with domain

$$\text{dom}(\varphi s) \subseteq \text{dom}(s)$$

and such that

$$(\varphi s)(x_1, \dots, x_k) = s(x_1, \dots, x_k)$$

for all $(x_1, \dots, x_k) \in \text{dom}(\varphi s)$.

The simplest type of filter is one that restricts access to s to a certain fixed number q of queries. Thus $[q]s$ is the system s restricted to q queries and is undefined as of the $(q+1)$ -st query.

Definition 3.10. The filter that restricts access to q queries is denoted as $[q]$.

3.4.4 Cascading Systems

In this and the next subsection we discuss two natural ways for combining discrete systems.

¹⁰This means that the function is undefined for inputs outside of $\mathcal{U}$.

Definition 3.11. Consider an $(\mathcal{X}, \mathcal{Y})$ -DDS s and a $(\mathcal{Y}, \mathcal{Z})$ -DDS t . The *cascade* of s and t , denoted $s \triangleright t$, is the $(\mathcal{X}, \mathcal{Z})$ -DDS defined as follows. The input $x \in \mathcal{X}$ (to $s \triangleright t$) is given as input to s , the output $y \in \mathcal{Y}$ of s (if defined) is given as input to t , and the output $z \in \mathcal{Z}$ of t (if defined) is returned as the output of $s \triangleright t$:

$$(s \triangleright t)(x_1, \dots, x_k) = t(y_1, \dots, y_k),$$

where, for $1 \leq j \leq k$,

$$y_j = s(x_1, \dots, x_j).$$

If either value is undefined, the output of $s \triangleright t$ is undefined.

One can alternatively define a converter casc which implements $s \triangleright t$ when given parallel access to s and t , i.e., such that

$$\text{casc}[s, t] = s \triangleright t.$$

3.4.5 Combining the Outputs of Systems

Definition 3.12. Consider two $(\mathcal{X}, \mathcal{Y})$ -DDS s and t . For an operation¹¹ $\star$ on $\mathcal{Y}$ the system $s \star t$ is the $(\mathcal{X}, \mathcal{Y})$ -DDS where the input is given to both s and t and their outputs are combined using $\star$ to obtain the output of $s \star t$:

$$(s \star t)(x_1, \dots, x_k) = s(x_1, \dots, x_k) \star t(x_1, \dots, x_k).$$

If either value is undefined, the output of $s \star t$ is undefined.

One can alternatively define a converter comb^* which implements $s \star t$ when given parallel access to s and t , i.e., such that

$$\text{comb}^*[s, t] = s \star t.$$

3.4.6 Connecting a Converter to an Interface

Recall Definition 3.5 of a DDS with interface set $\mathcal{I}$. We define the operation of applying a converter α at interface i of such a $(\mathcal{P} \times \mathcal{X}, \mathcal{Y})$ -DDS s .

Definition 3.13. The application of a $((\mathcal{X}, \mathcal{Y}), (\mathcal{X}, \mathcal{Y}))$ -DDC α at interface i of a $(\mathcal{P} \times \mathcal{X}, \mathcal{Y})$ -DDS s , denoted as $\alpha^i s$, is the $(\mathcal{P} \times \mathcal{X}, \mathcal{Y})$ -DDS defined as follows. Inputs at interfaces different from i are handled as before. An input (i, x) (at interface i) is handled as follows: x is given as input to α , and all outputs of α of the form (in, x) are given to $s^\perp$ as (i, x) . Once α outputs (out, y) , y is the output of the DDS $\alpha^i s$ for the input (i, x) .

We state the following lemma without a proof.

Lemma 3.1. *Appending converters at distinct interfaces commutes: For $i \neq j$ we have*

$$\alpha^i \beta^j s = \beta^j \alpha^i s.$$

¹¹typically the operation $\oplus$ on $\{0, 1\}^n$

3.5 Probabilistic Systems

We now define systems with a probabilistic behavior. To make things easy, we stay in the realm of discrete probability theory (avoiding σ -algebras and measure theory) and consider only finite sample spaces. In many cases one can consider a random experiment with an infinite sample space as corresponding to an infinite sequence of random experiments with finite sample spaces.

Definition 3.14. A *probabilistic discrete $(\mathcal{X}, \mathcal{Y})$ -system* (an $(\mathcal{X}, \mathcal{Y})$ -PDS for short) is a random variable over a set of $(\mathcal{X}, \mathcal{Y})$ -DDS.

A PDS, which we typically denote by a capital bold-face symbol such as S , is characterized by the probability distribution (over DDS).

A probabilistic system can, like deterministic systems, be described in many different ways, for example by appropriately modifying the description of a deterministic system, say by introducing a randomness space $\mathcal{R}$ and making everything depend on the random value r chosen from $\mathcal{R}$. Many different descriptions can correspond to the same PDS. We do not focus on *descriptions* of systems, but on the *mathematical object* described by a particular description.

Definition 3.15. An $(\mathcal{X}, \mathcal{Y})$ -*random function* (for finite $\mathcal{X}$ and $\mathcal{Y}$) is a random variable taking on functions $\mathcal{X} \rightarrow \mathcal{Y}$ as values. It corresponds to the system which for each input replies with the function value. In particular, equal inputs are always answered by identical outputs.

An $\mathcal{X}$ -*random permutation* is the special case where $\mathcal{Y} = \mathcal{X}$ and only bijective functions $\mathcal{X} \rightarrow \mathcal{X}$ have non-zero probability.

Example 3.5. A uniform random function [permutation] is called URF [URP]. A $(\{0, 1\}^m, \{0, 1\}^n)$ -URF is often denoted as $\mathbf{R}_{m,n}$ and a $\{0, 1\}^m$ -URP is often denoted as $\mathbf{P}_m$. When m and n are understood, the subscripts are also dropped.

Definition 3.16. A *probabilistic discrete environment* for an $(\mathcal{X}, \mathcal{Y})$ -DDS (a $(\mathcal{Y}, \mathcal{X})$ -PDE for short) is a random variable over a set of $(\mathcal{Y}, \mathcal{X})$ -DDE.

The transcript $\text{tr}(S, E)$ of a probabilistic system S in a probabilistic environment E is well-defined via the function $\text{tr}(\cdot, \cdot)$ (see Definition 3.7). In the random experiment consisting of selecting, independently, a system s and an environment e according to their respective distributions, $\text{tr}(S, E)$ is the random variable taking on as values sequences $x_1, y_1, x_2, y_2, x_3, y_3, \dots$ and can essentially equivalently be understood as the sequence $\text{tr}(S, E) = X_1, Y_1, X_2, Y_2, X_3, Y_3, \dots$ of random variables.

Definition 3.17. A *probabilistic discrete converter* (a PDC for short) (for given alphabets) is a random variable over a set of DDC (for the same alphabets).

The composition of a probabilistic converter with a probabilistic system is a probabilistic system and is defined naturally via the function $(\alpha, s) \mapsto \alpha s$, where the arguments and hence also the function value are random variables.

3.6 The Behavior of Discrete Systems

A (probabilistic) system S defines the probability distribution of the transcript $tr(S, E)$ for every environment E . To characterize the complete observable behavior of a system, it suffices to define the transcript distribution for every possible environment E . Surprisingly, two different PDS S and T (i.e., two different random variables over $(\mathcal{X}, \mathcal{Y})$ -DDS) can behave identically in any environment and can hence be considered equivalent (see Definition 3.19 below).¹² We hence want to define the mathematical object of the *behavior* of a probabilistic system.

3.6.1 A First Step: Channels in Communication Theory

To address the problem of defining the mathematical type of the input-output behavior of a discrete system, we first consider a simple type of system C , which only takes a single input from alphabet $\mathcal{X}$ and produces an output from alphabet $\mathcal{Y}$. In information theory and communication theory such a system is often called a *discrete memoryless channel* and is defined by a conditional probability distribution¹³

$$p_{Y|X}^C,$$

where the superscript C makes explicit which system we consider. Such a channel is characterized exactly by specifying, for each $x \in \mathcal{X}$ and $y \in \mathcal{Y}$, the probability that output y is generated when input x is given, which we can denote as $p_{Y|X}^C(y, x)$. We point out that defining $p_{Y|X}^C$ does *not* define a random experiment. This is the reason why we use a small p to avoid suggesting that a random experiment (and hence probabilities) are defined (at this point).

In contrast, if we follow our approach taken so far, we can define a deterministic channel c as a function $c : \mathcal{X} \rightarrow \mathcal{Y}$, a DDS with a single input x and corresponding output y . A probabilistic channel C is thus a random variable over functions $\mathcal{X} \rightarrow \mathcal{Y}$.

Such a probabilistic channel defines the conditional distribution $p_{Y|X}^C(y, x)$, as follows. Consider now the random experiment of choosing C from the corresponding distribution. For any fixed $x \in \mathcal{X}$, the output $C(x)$ is a $\mathcal{Y}$ -valued random variable, which we call Y . We can now define the function $p_{Y|X}^C : \mathcal{X} \rightarrow \mathcal{Y}$ defined by

$$p_{Y|X}^C(y, x) := \Pr^C(C(x) = y)$$

and call it the *behavior* of C . Note that $p_{Y|X}^C : \mathcal{X} \rightarrow \mathcal{Y}$ is a conditional probability distribution (or, alternatively, a probability distribution for each $x \in \mathcal{X}$. If we consider partially defined functions c , then the probability distributions are partial, meaning that the sum of the probabilities can be smaller than 1.

¹²This is not true for deterministic systems. For two distinct DDS, there is always an environment such that the resulting transcripts are different.

¹³The function $p_{Y|X}^C : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$ corresponds to a so-called stochastic matrix.

It is important to point out that two random variables C_1 and C_2 with different distributions over functions $\mathcal{X} \rightarrow \mathcal{Y}$ can have the same behavior, i.e., correspond to the *same* conditional distribution, i.e., we can have

$$p_{Y|X}^{C_1} = p_{Y|X}^{C_2}, \quad \neq$$

where equality means that the two functions are equal, i.e., $p_{Y|X}^{C_1}(y, x) = p_{Y|X}^{C_2}(y, x)$ for all $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. In this case we can say that C_1 and C_2 are *equivalent*, denoted

$$C_1 \equiv C_2.$$

Therefore, specifying a conditional distribution $p_{Y|X}^C$ does not specify a PDS, and in particular it does not specify a random experiment. But we can interpret a conditional distribution $p_{Y|X}^C$ as the *equivalence class* of all single-input PDS C' corresponding to the conditional distribution $p_{Y|X}^C$, i.e., as the set

$$\{C' \mid C' \equiv C\}.$$

Let us now consider the cascade $C \triangleright D$ of an $(\mathcal{X}, \mathcal{Y})$ -channel C and a $(\mathcal{Y}, \mathcal{Z})$ -channel D , where C and D are PDS with corresponding conditional probability distributions $p_{Y|X}^C$ and $p_{Z|Y}^D$, respectively. Note that the operation $\triangleright$ is defined for deterministic channels (i.e., functions), i.e., at the level of the values the random variables C and D can take on (not at the level of the random variables). This means that $C \triangleright D$ is also a PDS, a random variable taking on as values functions $\mathcal{X} \rightarrow \mathcal{Z}$.

Consider now the random experiment of choosing C and D independently (in which probabilities are denoted by the superscript CD , i.e., as $\Pr^{CD}$) and define the function $p_{Z|X}^{C \triangleright D}$ (a conditional distribution) as follows:

$$p_{Z|X}^{C \triangleright D}(z, x) := \Pr^{CD}((C \triangleright D)(x) = z).$$

We have

$$\begin{aligned} \Pr^{CD}(C(x) = y \wedge D(y) = z) &= \Pr^{CD}(C(x) = y) \cdot \Pr^{CD}(D(y) = z) \\ &= \Pr^C(C(x) = y) \cdot \Pr^D(D(y) = z) \\ &= p_{Y|X}^C(y, x) \cdot p_{Z|Y}^D(z, y). \end{aligned}$$

This justifies the multiplication of conditional probabilities appearing in information theory at the related literature. Moreover, we have

$$\begin{aligned} p_{Z|X}^{C \triangleright D}(z, x) &= \sum_{y \in \mathcal{Y}} \left(\Pr^{CD}(C(x) = y \wedge D(y) = z) \right) \\ &= \sum_{y \in \mathcal{Y}} \left(p_{Y|X}^C(y, x) \cdot p_{Z|Y}^D(z, y) \right). \end{aligned}$$

3.6.2 Defining the Behavior of Discrete Systems

A general $(\mathcal{X}, \mathcal{Y})$ -system is a more complex object than a channel, as discussed above, because it is invoked multiple times and keeps state between invocations. For a (probabilistic) $(\mathcal{X}, \mathcal{Y})$ -system $\mathbf{S}$, the first output Y_1 for input X_1 is generated according to some conditional probability distribution $p_{Y_1|X_1}^{\mathbf{S}}$. Then, when X_2 is input (by the environment), Y_2 depends on X_1 , X_2 , and the previous output Y_1 , i.e., it is generated according to some conditional probability distribution $p_{Y_2|X_1X_2Y_1}^{\mathbf{S}}$. Similarly, when X_i is input, Y_i depends on $X^i = (X_1, \dots, X_i)$ and the previous output $Y^{i-1} = (Y_1, \dots, Y_{i-1})$, i.e., it is generated according to some conditional probability distribution $p_{Y_i|X^iY^{i-1}}^{\mathbf{S}}$.¹⁴

These conditional distributions are well-defined and can be computed from the probability distribution (of $\mathbf{S}$) over deterministic systems. To determine $p_{Y_1|X_1}^{\mathbf{S}}$ one (separately) considers each value x_1 and determines the probability distribution of Y_1 induced by $\mathbf{S}(x_1)$.¹⁵

$$p_{Y_1|X_1}^{\mathbf{S}}(y_1, x_1) = \Pr^{\mathbf{S}}(\mathbf{S}(x_1) = y_1).$$

The probability on the right side is in the random experiment of choosing $\mathbf{S}$ at random, which is made explicit by the superscript $\mathbf{S}$. Similarly, to determine $p_{Y_2|X_1X_2Y_1}^{\mathbf{S}}$ one considers each pair (x_1, x_2) and each $y_1 \in \mathcal{Y}$ and determines the conditional probability distribution of $Y_2 = \mathbf{S}(x_1, x_2)$, given $Y_1 = \mathbf{S}(x_1) = y_1$, i.e., one determines

$$p_{Y_2|X_1X_2Y_1}^{\mathbf{S}}(y_2, x_1, x_2, y_1) = \Pr^{\mathbf{S}}(\mathbf{S}(x_1, x_2) = y_2 \mid \mathbf{S}(x_1) = y_1)$$

for all x_1, x_2, y_1 , and y_2 . Here $p_{Y_2|X_1X_2Y_1}^{\mathbf{S}}(y_2, x_1, x_2, y_1)$ is undefined if $p_{Y_1|X_1}^{\mathbf{S}}(y_1, x_1) = 0$.

Definition 3.18. The *behavior* of a probabilistic $(\mathcal{X}, \mathcal{Y})$ -system $\mathbf{S}$, denoted $b(\mathbf{S})$, is the sequence $p_{Y_i|X^iY^{i-1}}^{\mathbf{S}}$ of conditional probability distributions, for $i \geq 1$, where $p_{Y_i|X^iY^{i-1}}^{\mathbf{S}}$ is defined by

$$p_{Y_i|X^iY^{i-1}}^{\mathbf{S}}(y_i, x^i, y^{i-1}) := \Pr^{\mathbf{S}}(\mathbf{S}(x^i) = y_i \mid \mathbf{S}(x_1) = y_1, \dots, \mathbf{S}(x^{i-1}) = y_{i-1}),$$

which is undefined if the conditioning event has probability 0.¹⁶

¹⁴Recall that such a conditional probability distribution is a partial function $\mathcal{Y} \times \mathcal{X}^i \times \mathcal{Y}^{i-1} \rightarrow \mathbb{R}^+$ such that for all choices of the arguments x^i and y^{i-1} , the sum of the function values over the choices y_i equals 1. Note also that $p_{Y_i|X^iY^{i-1}}^{\mathbf{S}}$ is not defined for arguments x^i and y^{i-1} such that $p_{Y_j|X^jY^{j-1}}^{\mathbf{S}}(y_j, x^j, y^{j-1}) = 0$ for some $j < i$.

¹⁵Note that $\mathbf{S}$ is a function-valued random variable, hence $\mathbf{S}(x_1)$ is a $\mathcal{Y}$ -valued random variable.

¹⁶This means that y^{i-1} has probability 0 of occurring for input sequence x^{i-1} . For ease of notation we may allow in the following that a description of $p_{Y_i|X^iY^{i-1}}^{\mathbf{S}}$ is actually defined for some of the values x^i and y^{i-1} that occur with probability 0; the understanding is that they would have to be changed (to being undefined) in order to be compatible with the above definition.

If $\mathbf{S}$ is finite (with probability 1), then a finite sequence of conditional probability distributions suffices to describe the behavior.

Example 3.6. For an $(\mathcal{X}, \mathcal{Y})$ -URF $\mathbf{R}$ (see Example 3.5) we have

$$p_{Y_i|X^iY^{i-1}}^{\mathbf{R}}(y_i, x^i, y^{i-1}) = \begin{cases} 1 & \text{if } x_i = x_j \text{ for some } j < i \text{ and } y_i = y_j \\ 0 & \text{if } x_i = x_j \text{ for some } j < i \text{ and } y_i \neq y_j \\ 1/|\mathcal{Y}| & \text{if } x_i \neq x_j \text{ for all } j < i. \end{cases} \quad (3.1)$$

$p_{Y_i|X^iY^{i-1}}^{\mathbf{R}}(y_i, x^i, y^{i-1})$ is undefined if $x_j = x_k$ and $y_j \neq y_k$ for some $j < k < i$.

3.6.3 Equivalence of Probabilistic Systems

Different probabilistic systems can have the same behavior, which motivates the following equivalence relation.

Definition 3.19. Two probabilistic $(\mathcal{X}, \mathcal{Y})$ -systems $\mathbf{S}$ and $\mathbf{T}$ are *equivalent*, denoted $\mathbf{S} \equiv \mathbf{T}$, if they have the same behavior, i.e., if $b(\mathbf{S}) = b(\mathbf{T})$.

A behavior can be understood as the equivalence class of all probabilistic systems with that behavior. When considering probabilistic systems, it is usually sufficient to consider the corresponding behavior. Symbols like $\mathbf{S}$ and $\mathbf{E}$ are therefore also used when only the behavior is defined, i.e., without necessarily specifying an explicit PDS with that behavior.

We can call a sequence of conditional probability distributions as appearing in Definition 3.18 a *behavior*, even if no PDS (over a finite set of DDS) with that behavior can be defined. This can be useful. We give two examples.

Example 3.7. Equation (3.1) applies equally well if $\mathcal{X}$ is infinite, but this behavior does not correspond to a PDS as defined here since the sample space would be uncountable. However, we can consider the behavior without an underlying probabilistic system. The behavior of a URF with $\mathcal{X} = \{0, 1\}^*$ is called a *variable input-length URF* (VIL-URF) and is sometimes denoted as $\mathbf{V}_n$ if $\mathcal{Y} = \{0, 1\}^n$.¹⁷

Example 3.8. A $\mathcal{Y}$ -*beacon*, often denoted as $\mathbf{B}$, is a system with unary input alphabet $\{\diamond\}$ (a trigger) which outputs a new independent and uniformly distributed (over $\mathcal{Y}$) output Y_i for every new input X_i . In other words, we have

$$p_{Y_i|X^iY^{i-1}}^{\mathbf{B}}(y_i, x^i, y^{i-1}) = 1/|\mathcal{Y}|$$

for all choices of the arguments. A beacon for $\mathcal{Y} = \{0, 1\}^n$ is denoted as $\mathbf{B}_n$ (for any $\mathcal{X}$). One could also consider a sequence of finite beacons, for an increasing number of inputs.

¹⁷One could also understand $\mathbf{V}_n$ as a sequence of $(\{0, 1\}^m, \{0, 1\}^n)$ -URFs (i.e., probabilistic systems) for $m = 1, 2, 3, \dots$. For analyzing a particular setting, in which inputs have arbitrarily large but finite length, one uses a large enough m .

3.6.4 Cumulative Description of the Behavior

As mentioned earlier, a probabilistic system $\mathbf{S}$ can be analyzed for a fixed input sequence $x^i = (x_1, \dots, x_i)$. The output sequence $Y^i = (Y_1, \dots, Y_i)$ is a random variable, where

$$\begin{aligned} Y_1 &= \mathbf{S}(x_1), \\ Y_2 &= \mathbf{S}(x_1, x_2), \\ Y_3 &= \mathbf{S}(x_1, x_2, x_3), \dots \end{aligned}$$

i.e., $\mathbf{S}$ defines the conditional probabilities $\Pr^{\mathbf{S}}(\mathbf{S}(x^i) = y_i)$. We can hence define $p_{Y^i|X^i}^{\mathbf{S}}(y^i, x^i)$ as follows:

Definition 3.20. The cumulative description of the behavior of a probabilistic $(\mathcal{X}, \mathcal{Y})$ -system $\mathbf{S}$ is the sequence $p_{Y^i|X^i}^{\mathbf{S}}(y^i, x^i)$ of conditional probability distributions, for $i \geq 1$, where

$$p_{Y^i|X^i}^{\mathbf{S}}(y^i, x^i) = \Pr^{\mathbf{S}}((\mathbf{S}(x_1), \mathbf{S}(x_1, x_2), \dots, \mathbf{S}(x^i)) = y^i).$$

It is not difficult to see that, as one would expect,

$$p_{Y^i|X^i}^{\mathbf{S}}(y^i, x^i) = \prod_{j=1}^i p_{Y^j|X^j Y^{j-1}}^{\mathbf{S}}(y_j, x^j, y^{j-1}), \quad (3.2)$$

where $p_{Y^j|X^j Y^{j-1}}^{\mathbf{S}}(y_j, x^j, y^{j-1})$ is defined in Definition 3.18. Note that $p_{Y^i|X^i}^{\mathbf{S}}$ implies the conditional distributions $p_{Y^j|X^j}^{\mathbf{S}}$ for all $j < i$ (how?).

The conditional distribution $p_{Y^i|X^i}^{\mathbf{S}}$ satisfies a consistency condition which captures that for $j < i$, Y_j does not depend on $X_{j+1}, X_{j+2}, \dots$. The distributions $p_{Y^i|X^i Y^{i-1}}^{\mathbf{S}}$ can be computed from the distributions $p_{Y^i|X^i}^{\mathbf{S}}$, using (3.2):

$$p_{Y^i|X^i Y^{i-1}}^{\mathbf{S}}(y_i, x^i, y^{i-1}) = \frac{p_{Y^i|X^i}^{\mathbf{S}}(y^i, x^i)}{p_{Y^{i-1}|X^{i-1}}^{\mathbf{S}}(y^{i-1}, x^{i-1})}$$

(whenever $p_{Y^{i-1}|X^{i-1}}^{\mathbf{S}}(y^{i-1}, x^{i-1}) \neq 0$). Hence the behavior of a system $\mathbf{S}$ can alternatively be described by the sequence $p_{Y^i|X^i}^{\mathbf{S}}$ of conditional distributions, for $i \geq 1$. This cumulative description is redundant but often easier to work with. The behavior of a system $\mathbf{S}$ answering at most a certain number q of queries is hence completely specified by $p_{Y^q|X^q}^{\mathbf{S}}$.

3.6.5 Computing the Transcript Distribution

The behavior of an environment $\mathbf{E}$ is defined analogously to Definition 3.18.

Definition 3.21. The behavior of a $(\mathcal{Y}, \mathcal{X})$ -environment $\mathbf{E}$, denoted $b(\mathbf{E})$ is the sequence $p_{X^i|X^{i-1}Y^{i-1}}^{\mathbf{E}}$ (for $i \geq 1$) of conditional probability distributions.

The cumulative description of the behavior is defined analogously.

As mentioned in Section 3.5, an $(\mathcal{X}, \mathcal{Y})$ -PDS $\mathbf{S}$ and a $(\mathcal{Y}, \mathcal{X})$ -PDE $\mathbf{E}$ together define a random experiment (where $\mathbf{S}$ and $\mathbf{E}$ are, as always, independent), in which the transcript

$$tr(\mathbf{S}, \mathbf{E}) = X_1, Y_1, X_2, Y_2, X_3, Y_3, \dots$$

is defined. We denote the random variable corresponding to the initial segment of the transcript (up to the k th step) by $X^k Y^k$. Its probability distribution can be described in terms of the behavior of $\mathbf{S}$ and the behavior of $\mathbf{E}$. We omit the proof of the following lemma.

Lemma 3.2. For a PDS $\mathbf{S}$ and a PDE $\mathbf{E}$ we have¹⁸

$$\begin{aligned} p_{X^k Y^k}^{\mathbf{ES}}(x^k, y^k) &= \prod_{i=1}^k \left(p_{X^i|X^{i-1}Y^{i-1}}^{\mathbf{E}}(x_i, x^{i-1}, y^{i-1}) \cdot p_{Y^i|X^i Y^{i-1}}^{\mathbf{S}}(y_i, x^i, y^{i-1}) \right) \\ &= p_{X^k|Y^{k-1}}^{\mathbf{E}}(x^k, y^{k-1}) \cdot p_{Y^k|X^k}^{\mathbf{S}}(y^k, x^k). \end{aligned}$$

We point out that $p_{X^k Y^k}^{\mathbf{ES}}$ depends only on the behavior of $\mathbf{S}$ and $\mathbf{E}$. We also note that in any such random experiment, the conditional probability distribution of Y_i , given all the random variables $X_1, \dots, X_i, Y_1, \dots, Y_{i-1}$, is equal to $p_{Y_i|X^i Y^{i-1}}^{\mathbf{S}}$:

$$p_{Y_i|X^i Y^{i-1}}^{\mathbf{ES}} = p_{Y_i|X^i Y^{i-1}}^{\mathbf{S}}, \quad (3.3)$$

for all arguments for which both functions are defined. However, in contrast to equation (3.3), for a general environment $\mathbf{E}$ we have

$$p_{Y^i|X^i}^{\mathbf{ES}} \neq p_{Y^i|X^i}^{\mathbf{S}},$$

i.e., $p_{Y^i|X^i}^{\mathbf{S}}$ does generally *not* correspond to the conditional probability distribution of Y^i , given X^i , in a random experiment in which the system $\mathbf{S}$ is embedded.

3.7 Discrete Computational Problems

3.7.1 Discrete Games

A (deterministic) discrete game $\mathbf{G}$ can be described by a discrete $(\mathcal{X}, \mathcal{Y})$ -system which interacts with its environment by taking inputs $x_1, x_2, \dots$ and answering

¹⁸To be entirely explicit, we have put the random variables defining the random experiment (i.e., $\mathbf{ES}$) as a superscript also for (conditional) probability distributions of random variables.

with outputs $y_1, y_2, \dots$. In addition, after every input it also generates a bit a_i indicating whether the game has been won. This bit is monotone in the sense that it is initially set to 0 and that, once it has turned to 1 (i.e., the game is won), it does not turn back to 0 (even if the game were continued). This motivates the following definition, which captures the notion of game-winning.

Definition 3.22. For a $(\mathcal{X}, \mathcal{Y} \times \{0, 1\})$ -DDS g the binary component a_i of the output (y_i, a_i) is called a *monotone binary output (MBO)* if $a_i = 1$ implies $a_j = 1$ for $j \geq i$. Such a DDS g with MBO is also called a *deterministic discrete $(\mathcal{X}, \mathcal{Y})$ -game (DDG)*.

Obviously, a probabilistic discrete game G is now a probabilistic discrete system (PDS, see Definition 3.14) with MBO.

A (deterministic) winner w for an $(\mathcal{X}, \mathcal{Y})$ -game g generates x_1 as an input to g , receives the output y_1 of g as its (first) input, then generates x_2 , receives y_2 , etc.

Definition 3.23. A (deterministic) winner w for an $(\mathcal{X}, \mathcal{Y})$ -game g is a discrete $(\mathcal{Y}, \mathcal{X})$ -environment (DDE, see Definition 3.16).

Note that the MBO a_i is *not* returned to w . One can think of a_i as being output at a special interface of g which we can think of as depicted at the right side of g . We can nevertheless understand a_i to be part of the transcript. The transcript (see Definition 3.7) is hence

$$tr(g, w) = x_1, (y_1, a_1), x_2, (y_2, a_2), x_3, (y_3, a_3), \dots$$

Winner w wins game g if $a_i = 1$ for some $i \geq 1$.

Whether the winner “sees” a_i or not may appear to be irrelevant since all we care about is whether the game is won. However, for so-called multi-games, the natural generalization where the game has several MBO’s, it does matter whether these bits are (or are not) hidden from the winner. We point out that even though the MBO’s are hidden, a DDG can of course be defined in a way that the regular outputs y_i contain information about a_i or even explicitly contains a copy of a_i .

3.7.2 Discrete Distinguishers

Definition 3.24. A (deterministic) *discrete distinguisher* for $(\mathcal{X}, \mathcal{Y})$ -system (DDD) is an environment d for $(\mathcal{X}, \mathcal{Y})$ -systems which, when it stops, also outputs a bit. More formally, it is an environment with two stop symbols $\neg_0$ and $\neg_1$. The output of d for system s is equal to the index of the stopping symbol. If d does not stop (i.e., never outputs $\neg_0$ or $\neg_1$), then the output is defined to be 0.

Chapter 4

Computational Problems and Reductions

4.1 Preliminaries

Computational problems and reductions between problems have been studied and formalized in complexity theory. The types of problems arising in cryptography are more general, in several ways, than those usually considered in complexity theory.

4.1.1 Decision Problems

In classical complexity theory (where classes such as P and NP are defined), an important type of computational problem are *decision problems*. A decision problem is characterized by an instance set $\mathcal{X}$ and a predicate $P: \mathcal{X} \rightarrow \{0, 1\}$. The problem consists of deciding, for a given $x \in \mathcal{X}$, whether $P(x) = 1$.

A specific type of decision problem is characterized by a predicate $Q: \mathcal{X} \times \mathcal{W} \rightarrow \{0, 1\}$, where $\mathcal{X}$ is an instance set and $\mathcal{W}$ is a witness set. The problem consists of deciding, for a given $x \in \mathcal{X}$, whether there exists a witness $w \in \mathcal{W}$ such that $Q(x, w) = 1$, i.e., $P(x) := \exists w Q(x, w)$.

Example 4.1. Decide whether a given graph with n vertices contains a Hamiltonian cycle. The instance set is (some subset of) the set of Boolean $n \times n$ matrices (the adjacency matrix of a graph), the witness set is the set of (somehow encoded) paths of length n , and the predicate Q consisting of verifying that a path is a cycle in the graph. This problem is known to be NP-complete.¹

¹see M.R. Garey and D.S. Johnson, *Computers and Intractability – A Guide to the Theory of NP-Completeness*, New York: W.H. Freeman and Co., 1979.

Example 4.2. Decide whether a given number is a prime number. This problem has been shown to be in P .²

4.1.2 Search Problems

The search problem corresponding to a decision problem (of the above type) is the problem of *finding* a witness w (which can be called a solution) for a given x .

Definition 4.1. A search problem is a 4-tuple $(\mathcal{X}, \mathcal{W}, Q, P_X)$ consisting of an *instance set* $\mathcal{X}$, a *solution set* (or *witness set*) $\mathcal{W}$, a predicate $Q : \mathcal{X} \times \mathcal{W} \rightarrow \{0, 1\}$, as well as a probability distribution P_X over the instance set.³

The search problem $(\mathcal{X}, \mathcal{W}, Q, P_X)$ consists of finding, for a given instance $x \in \mathcal{X}$ drawn according to P_X , a $w \in \mathcal{W}$ such that $Q(x, w) = 1$.⁴ Typically one considers cases where every instance x has at least one solution w , but other cases can also be considered by allowing a special output “no solution”.

Example 4.3. Finding a Hamiltonian cycle in a graph with n vertices (see Example 4.1). A random graph with n nodes containing a Hamiltonian cycle can, for example, be generated by starting from an n -cycle and then adding random edges according to some distribution.

Exercise 4.1. Suppose that for a search problem there exists a probabilistic algorithm with average success probability 1%. Can one amplify the success probability by repeating the algorithm several times?

Example 4.4. The discrete logarithm (DL) problem (see Definition 2.12) for a group $G = \langle g \rangle$ is a search problem where the instance set is G , the solution set is $\mathbb{Z}_q$, and the predicate Q is defined as

$$Q(x, w) = 1 \iff g^w = x,$$

and the instance distribution P_X is the uniform distribution.

Example 4.5. *Function inversion.* The problem of inverting a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ will be denoted as $\mathcal{I}^f$ ($\mathcal{I}$ stands for inversion.). The instance set is $\mathcal{X} = \{0, 1\}^m$, the solution set is $\mathcal{W} = \{0, 1\}^n$, and the predicate Q is defined as

$$Q(x, w) = 1 \iff f(w) = x.$$

The distribution P_X is usually assumed to be the distribution implied by the uniform distribution on the input of f , i.e., $X = f(U)$, where U is a uniform n -bit string. Note that the DL problem is also a function inversion problem. A function f for which the function inversion problem is computationally hard is called a *one-way function*.

²see M. Agrawal, N. Kayal, and N. Saxena, PRIMES is in P , *Annals of Mathematics*, 160 (2): 781–793, 2004.

³Often, but not always, $\mathcal{X} \subseteq \{0, 1\}^*$, $\mathcal{W} \subseteq \{0, 1\}^*$, and P_X is the uniform distribution.

⁴One can distinguish two types of search problems, depending on whether or not Q is efficiently computable. If Q is efficiently computable, then problems with a small solution set $\mathcal{W}$ are easy to solve by an exhaustive search.

4.1.3 Beyond Worst-case Analyses

In traditional complexity theory, one usually considers only algorithms that work for *all* instances, i.e., in the worst case. Such problems do not specify an instance distribution, i.e., they can be considered to be triples $(\mathcal{X}, \mathcal{W}, Q)$. In cryptography, the worst-case hardness of a problem is useless. Even if (the computational problem of) breaking a cryptosystem is hard in the worst case (for example NP-complete), it is still possible that the problem is easy with substantial probability or even with very high probability. What is required in cryptography are statements about *almost-everywhere* hardness. This forces us to reconsider the notion of success of an algorithm for solving a certain computational problem and the notion of a reduction.

4.1.4 Problems Relevant in Cryptography

We briefly repeat what was already discussed in Section 2.3: The three types of computational problems relevant in cryptography.

In cryptography, an important class of computational problems are so-called *games*. In a game, the instance is not only a value, as in a search problem, but an instance is an interactive process or discrete system with which an algorithm (trying to win the game) can interact in several steps. The game is characterized by a special *winning condition*. The algorithm succeeds if it provokes this winning condition to become true. Search problems are a special type of game which are non-interactive.

Example 4.6. The security definition of a MAC (see Definition 2.11) can be phrased as a game which the adversary is supposed to win. The game is interactive.

Another important class of computational problems in cryptography are so-called *distinction problems*. The task of an algorithm is to determine which of two options (e.g. values) it is given access to.

Example 4.7. The so-called Decisional Diffie-Hellman (DDH) problem (see Definition 2.14) is a distinction problem.

Similarly to the generalization from search problems to interactive games, distinction problems can be generalized to a setting where for each option not only a value, but access to an interactive system is provided.

Example 4.8. The real-world system and the ideal-world system for symmetric encryption (see Section 2.4.4) are simple examples of interactive systems, where the distinguishing algorithm can input a message at interface A and observe the outputs at interfaces B and E .

Closely related to distinction problem are *bit-guessing problem* where the task is, when given access to a system, to guess a bit hidden within the system. The two types of problems can be shown to be in a certain sense equivalent.

4.2 Example 1: Security of the RSA LSB

Recall the reduction concept mentioned in Section 1.5.2. We introduce a first example of a non-trivial cryptographically interesting reduction. We describe it in a conventional language (using terms like efficient algorithm); our terminology and concepts will be introduced later.

The security of the RSA trapdoor one-way permutation relies on the computational hardness of computing e -th roots modulo n . However, even if this problem is hard, it is conceivable that finding certain parts of the e -th root (i.e., of the message in the naïve RSA system), for example the least significant bit (LSB) is nevertheless easy. In this section we prove the surprising result that finding the least significant bit of an RSA-plaintext from a given ciphertext is computationally equivalent to finding the entire plaintext.

Theorem 4.1. *If there exists an efficient algorithm A which for a given k -bit RSA-modulus n , exponent e , and ciphertext c , computes the LSB of the corresponding plaintext m (where $c \equiv m^e$), then there exists an efficient algorithm which for a given c computes the entire corresponding m (by calling algorithm A k times as a subroutine).*

Proof. On input n, e and $c = m^e$ modulo n , algorithm A computes the LSB of m , i.e., $A(c) = \text{LSB}(m)$. Such an algorithm A can be exploited as follows.

By the multiplicative property of the RSA function we (i.e., the reduction algorithm) can easily compute

$$(2m)^e = 2^e m^e = 2^e c,$$

and similarly

$$(4m)^e = 4^e c,$$

etc., always to be understood modulo n . If $m < n/2$, then $2m < n$ and hence $2m$ is not reduced modulo n . If $m > n/2$, then $2m > n$ and hence $2m$ is reduced modulo n (i.e., n is subtracted). Because n is odd we have

$$A(2^e c) = 0 \iff m < n/2.$$

By a similar argument one can show that

$$A(4^e c) = 0 \iff \begin{cases} 0 < m < n/4 \\ \text{or} \\ n/2 < m < 3n/4 \end{cases}$$

Each value $A((2^i)^e c)$ allows us to exclude one half of the remaining possible region for m . For instance, if $A(2^e c) = 1$ we know that $m > n/2$. If in addition $A(4^e c) = 0$ we know that $n/2 < m < 3n/4$. Similarly, $A(8^e c) = 1$ tells us that $5n/8 < m < 3n/4$, etc. After k applications of algorithm A one ends up with a single value m . In other words, we can determine the binary expansion of m/n , bit-by-bit: $A(2^e c)$ is the LSB of $\lfloor 2 \frac{m}{n} \rfloor$, $A(4^e c)$ is the LSB of $\lfloor 4 \frac{m}{n} \rfloor$, etc. Hence, we have proved the following lemma, which concludes the proof of the theorem. $\square$

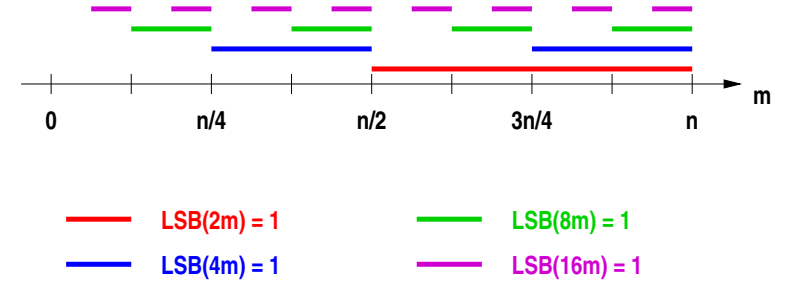


Figure 4.1: Illustration of the proof of Theorem 4.1. The regions within the interval $[0, n - 1]$ are shown for which $A(2^e c) = 1$, $A(4^e c) = 1$, $A(8^e c) = 1$ and $A(16^e c) = 1$. The expression $\text{LSB}(2m)$ is to be understood in the sense that $2m$ is reduced modulo n before the LSB is computed. (Similarly for the other expressions.)

Lemma 4.2. $A((2^i)^e c) = \text{LSB}(\lfloor 2^i \frac{m}{n} \rfloor)$.

Let us discuss the significance (or insignificance) of Theorem 4.1. It seems to state that a single-bit message b can be securely encrypted by choosing a random m with LSB b . (Note that this is a randomized PKE scheme.) However, Theorem 4.1 is not strong enough to prove the IND-CPA security of this scheme. One would need that guessing the LSB correctly with probability significantly greater than 0.5 allows to invert the RSA function.⁵ Indeed, such a stronger statement can be proved. Moreover, an even stronger result can be proved:⁶ The $\log k$ least significant bits are simultaneously unapproximable if inverting the RSA function is hard. More precisely, an algorithm which distinguishes the $\log k$ least significant bits of a random plaintext from uniformly random bits with non-negligibly advantage can be used to compute the entire plaintext. This shows that one could use the naïve RSA encryption (with randomization) to encrypt a few bits.

We note that a reduction statement of this type can be read in two different directions, which are each valid:

- The least significant bits are secure if we (only) assume the hardness of inverting the RSA function.
- The RSA function can be inverted if one can slightly distinguish the least significant bits from random. This task appears much easier than inverting RSA, and this can be seen as casting doubt on the security of RSA.

We also note that in a reasonable cryptographic design, the RSA-function is

⁵This statement can be rephrased as follows: The LSB of the RSA function is a *hard-core predicate*.
⁶W. Alexi, B. Chor, O. Goldreich, and C. P. Schnorr, RSA and Rabin functions: Certain parts are as hard as the whole, *SIAM Journal on Computing*, 17(2):194–209, 1988.

usually only used as a TOWP and not as a (naïve) public-key encryption scheme. In this case the relevance of understanding the security of the LSB is unclear.

4.3 Example 2: Guessing the LSB of a Discrete Logarithm

4.3.1 Introduction

In this section we discuss another example of a cryptographic reduction. This example is more interesting than that of the previous section, for several reasons. First, it is related to the result of a famous paper.⁷ Second, the reduction goes all the way from the hardness of computing a value (a discrete logarithm) to showing that a certain bit cannot even be guessed with a slight advantage over a random guess. This result can be used to obtain a pseudo-random generator based on the sole assumption that the DL problem in the given group is worst-case hard. Third, the reduction is obtained in a sequence of steps, which can each be understood as an individual reduction, where the overall statement is obtained as the composition of these individual reductions.

The overall treatment is quite involved, and this provides motivation for our abstract approach to defining and understanding reductions, which is to follow in the rest of the chapter.

Already in this section we use terminology introduced later in the chapter: We use the term *solver* (instead of algorithm) for the object that solves a computational problem. We also use the term *performance* to measure the amount of success of a solver:⁸

- For a game, the performance of a solver is the probability of winning the game (see Definition 2.5).
- For a bit-guessing problem like the LSB-problem, the performance of a solver is the advantage in guessing the bit correctly, i.e., $2p - 1$ if p is the probability of guessing the bit correctly (see Definition 2.9).

A worst-case problem means that one requires a solver to achieve a certain performance for all instances of the problem. In other words, the performance of a solver for a worst-case problem is defined as the minimum of the performance over all instances of the problem.

An average-case problem means that one requires a solver to achieve a certain performance on average over the instances of the problem chosen according to a specified distribution.

⁷see Manuel Blum and Silvio Micali, How to Generate Cryptographically Strong Sequences of Pseudorandom Bits, *SIAM Journal on Computing*. vol. 13, no. 4, 1984, pp. 850–864.

⁸The term performance does not refer to efficiency or speed.

4.3.2 The Problem Statement

We consider a group $\mathcal{H} = \langle h \rangle$ of order $|\mathcal{H}| = m$, generated by the generator h , and we consider two computational problems for $\mathcal{H}$:

- the DL-problem in $\mathcal{H}$, i.e., given $y = h^x$, compute x ;
- the LSB-problem in $\mathcal{H}$, i.e., given $y = h^x$, compute the least significant bit (LSB) of x .

Since the LSB-problem is easy for even group order (see exercise below) we assume from now on that $|\mathcal{H}| = m$ is odd.

Exercise 4.2. Show that if the group order m is even, then guessing the LSB is easy even in the worst case, i.e., there exists an efficient solver for the worst-case LSB-problem with performance 1.

We use the following notation to denote different computational problems for the group $\mathcal{H}$. We use DL and LSB to refer to discrete logarithm and LSB problems, respectively. A set $\mathcal{I}$ as superscript for $\mathcal{I} \subseteq \mathcal{H}$ denotes the worst-case problem for the instance set $\mathcal{I}$. Often $\mathcal{I} = \mathcal{H}$. A $\mathcal{H}$ -valued random variable Y as superscript denotes the average-case problem when the instance is (chosen according to the distribution of) Y . A uniform random variable over $\mathcal{H}$ is denoted as H . The computational problems that will arise in our context are:

- $\text{DL}^{\mathcal{H}}$ denotes the worst-case DL-problem (for the full instance set $\mathcal{H}$).
- $\text{DL}^{\mathcal{I}}$ denotes the worst-case DL-problem for an instance set $\mathcal{I} \subseteq \mathcal{H}$.
- DL^H denotes the average-case DL-problem (uniform over $\mathcal{H}$).
- $\text{LSB}^{\mathcal{H}}$ denotes the worst-case LSB-problem (for the full instance set $\mathcal{H}$).
- $\text{LSB}^{\mathcal{I}}$ denotes the worst-case LSB-problem for an instance set $\mathcal{I} \subseteq \mathcal{H}$.
- LSB^H denotes the average-case LSB-problem (uniform over $\mathcal{H}$).

Our goal is to prove that if the DL-problem is hard (the assumption) then the apparently much simpler problem of guessing (only) the least significant bit (LSB) of the discrete logarithm, is also hard (the conclusion). The goal is to prove such a statement

- for the *weakest* possible form of the assumption, and
- for the *strongest* possible form of the conclusion.

The (weak) assumption is that solving the worst-case DL-problem $\text{DL}^{\mathcal{H}}$ with performance close to 1 is hard. This means that there is no efficient algorithm that can solve every instance of the DL-problem with probability at least $1 - \delta$, for some small $\delta > 0$. Note that this assumption is weaker than the assumption that solving the worst-case DL-problem with performance ϵ for some $\epsilon > 0$ is hard,

and it is also weaker than the assumption that the average-case DL-problem DL^H with performance $1 - \delta$ is hard. In other words, our assumption does not exclude the existence of an efficient worst-case DL-algorithm that works only mildly well or one that works very well on average.

The (strong) conclusion is that the average-case LSB-guessing problem LSB^H is hard, even if only a small advantage (i.e., a guessing probability slightly above 0.5) is required. Note that this is a (much) stronger statement than that the worst-case LSB-problem LSB^H is hard.

The described statement corresponds to the following theorem, which is a reduction statement. It is only informally stated; what “simple operations” is not made formal.

Theorem 4.3. *For any $\delta, \epsilon > 0$, and for any solver S for LSB^H with performance at least ϵ , there exists a solver for DL^H with performance at least $1 - \delta$ which invokes S a polynomial (in $\log(1/\delta)$ and $1/\epsilon$ and $\log m$) number of times and otherwise performs only a few simple operations.⁹*

From this theorem one can conclude the following corollary, assuming that negligible is made precise in an adequate manner. This is a typical formulation of Theorem 4.3 appearing in the literature.

Corollary 4.4. *If for some $\delta < 1$ there exists no polynomial-time algorithm for solving DL^H with performance at least $1 - \delta$, then there exists no polynomial-time algorithm for solving LSB^H with non-negligible performance.*

Theorem 4.3 is proved by the composition of a sequence of reductions. We develop a natural line of thinking in which these individual reductions are developed, making explicit the various interesting and clever ideas that one needs in order to succeed.

4.3.3 Idea 1: Using an LSB-oracle to Compute the DL

Recall that raising any element $y \in \mathcal{H}$ to the group order m yields the neutral element (denoted as 1), i.e., we have

$$y^m = 1.$$

We first describe two useful operations on group elements. Given $y = h^x$ with $x > 0$ one can reduce x by 1 (modulo m), i.e., one can compute $y' = h^{x-1}$ as follows:

$$y' = y \cdot h^{-1}.$$

⁹The theorem is not a totally precise mathematical statement since we have not defined “a few simple operations”, but the proof will make explicit which operations are needed and one will see that they are simple. One could make a more formal statement involving the notion of polynomial-time.

Moreover, the (unique) square root z of $y = h^x$ can be computed as

$$z = y^{(m+1)/2}$$

because $z^2 = y^{m+1} = y$. If x is even, then $z = h^{x/2}$, i.e., the binary representation of x is shifted to the right by one bit.

If we have a solver S for LSB^H that is always correct (i.e., with performance 1), then a solver T for DL^H can be obtained as follows. Given an instance $y = h^x$, T determines the LSB of x by invoking S . Then, T computes y' for which the LSB of x is masked: If the LSB of x is 1 (i.e., x is odd), T computes $y' = y \cdot h^{-1}$, otherwise it sets $y' = y$. If we define x' by $y' = h^{x'}$, then x' is even. Thus T can shift (the binary representation of) x' to the right by one bit by computing

$$z = y'^{(m+1)/2} = h^{x'/2} = h^{\lfloor x/2 \rfloor}.$$

Then T sets $y = z$ and repeats the above steps until $z = h$, thus computing all the bits of x , and hence x . This requires at most $\lceil \log_2 x \rceil \leq \lceil \log_2 m \rceil$ invocations of S .

We next observe that the above argument also works if S guesses the LSB correctly only with probability at least $1 - \delta'/2$ for any instance, i.e., if it has performance $1 - \delta'$ for LSB^H . In this case, the constructed solver T for DL^H has success probability (and hence performance) at least $1 - \delta' \lceil \log_2 x \rceil / 2$. This follows from the union bound of probability theory.¹⁰ The algorithm is incorrect only if at least one of the invocations of S yields a wrong result. Since we will want the performance to be at least $1 - \delta$, one has to choose δ' accordingly.

4.3.4 Idea 2: From Worst-Case to Average Case

We actually have to work with a much weaker LSB-solver, namely one for which only the *average-case* performance is guaranteed and, moreover, where the performance is *very small*, namely only guaranteed to be at least ϵ .

A technique that sometimes works for solving the first problem is to randomize the instance so that it becomes an average-case instance. A technique that sometimes works for solving the second problem is to amplify the performance by repetition (see idea 4 below).

A given instance $y = g^x$ can be randomized by choosing a random $r \in \mathbf{Z}_m$ and computing the randomized instance

$$y' = y \cdot g^r = g^{x+r}$$

(where exponents are reduced modulo m). Unfortunately, this randomization does not work because if

$$x + r \geq m,$$

¹⁰The union bounds states the simple fact that the probability of the union of events is upper bounded by the sum of the probabilities.

i.e., if $x + r$ is reduced modulo m (which we call an overflow). In this case, the LSB changes because m is odd. Since x is unknown, one does not know whether the LSB was changed, and hence the LSB-solver's answer for the instance y' is useless.

We therefore need another idea and come back to idea 2 in Section 4.3.7.

4.3.5 Idea 3: Restriction to the Initial Segment

The above randomization technique would work if x were known to be very small. So the following idea can be used. For a fixed integer parameter n , we divide the interval $[0, m - 1]$ for x into n segments of length $\lceil m/n \rceil$ (where the last segment can be shorter) and assume we have a solver that works if x is in the first segment. Let

$$\mathcal{I} = \{h^0, h^1, \dots, h^{\lceil m/n \rceil - 1}\} \subseteq \mathcal{H}$$

denote the "initial segment" of the instance set of the problems we consider. The worst-case DL- and LSB-problems for instances in $\mathcal{I}$ are denoted as $\text{DL}^{\mathcal{I}}$ and $\text{LSB}^{\mathcal{I}}$, respectively. Moreover, let

$$t = \lceil \log_2(\lfloor m/n \rfloor + 1) \rceil$$

be the number of bits needed to represent the discrete logarithm of an element of $\mathcal{I}$.

For this idea to work we need to show three things.

- First, a solver S for $\text{DL}^{\mathcal{I}}$ can be used to construct a solver for $\text{DL}^{\mathcal{H}}$ (this is idea 3). If we have a worst-case solver S for the initial segment, i.e., for $\text{DL}^{\mathcal{I}}$, then we can obtain a worst-case solver T for $\text{DL}^{\mathcal{H}}$, as follows. For $\ell = 0, \dots, n - 1$, T computes

$$y' = y \cdot g^{-\ell \lceil m/n \rceil} = g^{x - \ell \lceil m/n \rceil}$$

and invokes S for instance y' . One of these shifted instances is guaranteed to be in $\mathcal{I}$. If a solution x' is reported by S , then it is tested for correctness (which is possible efficiently), and the shift is adjusted to obtain the correct DL-value:

$$x = x' + \ell \lceil m/n \rceil.$$

T needs to invoke S at most n times.

- Second, we need to observe that for instances of $\text{DL}^{\mathcal{I}}$, the above-described technique for using an $\text{LSB}^{\mathcal{H}}$ -solver to construct a $\text{DL}^{\mathcal{H}}$ -solver (idea 1 above) works also if we only have an $\text{LSB}^{\mathcal{I}}$ -solver. This is the case because all instances for which the LSB-solver S is invoked have exponents in the interval $\mathcal{I}$.
- Third, we need to investigate how the randomization and amplification technique works for instances known to be in $\mathcal{I}$ (see below).

4.3.6 Idea 4: Performance Amplification for Worst-Case Solvers

Given a solver S for $\text{LSB}^{\mathcal{I}}$ with performance ϵ' we can construct a solver T for $\text{LSB}^{\mathcal{I}}$ with performance $1 - \delta'$ by **statistical amplification: For an adequate parameter q , T invokes S q times and then takes a majority decision.** By the law of large numbers, it suffices to choose q roughly on the order of $1/(\epsilon')^2$ and $\log(1/\delta')$.

Exercise 4.3. Work this out.

4.3.7 Making Idea 2 Work: Randomizing the Instance

A solver S for $\text{LSB}^{\mathcal{H}}$ (i.e., for the uniform distribution) with performance ϵ can be used to construct a solver T for $\text{LSB}^{\mathcal{I}}$ with performance ϵ' , for ϵ' to be discussed below, as follows.

Given $y = g^x$ with $y \in \mathcal{I}$, T chooses an offset r uniformly at random from $[0, k - 1]$ for k discussed below, adds the offset r to x by computing

$$y' = y \cdot h^r = h^{x+r},$$

calls S (once) for the instance y' , receives the answer b of S , and returns $b \oplus r_0$, where r_0 is the LSB of r .¹¹

To avoid an overflow (i.e., $x + r \geq m$), we must have $0 \leq r \leq k - 1$ for

$$k = m - \lfloor m/n \rfloor.$$

Note that $x + k - 1 < m$. An instance $x \in \mathcal{I}$ is hence randomized to a uniform instance in the interval

$$J(x) = \{h^x, h^{x+1}, \dots, h^{x+k-1}\},$$

which depends on x . The size of this interval satisfies

$$|J(x)| = k = m - \lfloor m/n \rfloor \geq m(1 - 1/n).$$

4.3.8 Idea 5: Changing the Instance Distribution

The problem with this approach is that we consider the computational problem LSB^{Y_x} for Y_x uniform in $J(x)$ instead of the problem $\text{LSB}^{\mathcal{H}}$ (for H uniform in $\mathcal{H}$), for which we assumed that the performance of S is at least ϵ . This problem can be addressed by using the following fact (which the reader can prove as an exercise).

¹¹The answer of S must be complemented if r is odd (by computing $b \oplus r_0$) because if b is the correct LSB of $x + r$, then $b \oplus r_0$ is the correct LSB of x (provided there is no overflow).

Exercise 4.4. Prove that for a bit-guessing problem and a solver S for it, if one changes the instance distribution by at most d in terms of statistical distance, then the performance of S changes by at most $2d$.¹² Also state this formally.

Exercise 4.5. Prove that in our example, the statistical distance between the instance distribution of LSB^{Y_x} (for any x) and LSB^H is at most $1/n$, i.e., for all $x \in \mathcal{I}$,

$$\delta(Y_x, H) \leq 1/n.$$

Using the results of these two exercises, we conclude that the performance of solver T for LSB^{Y_x} is at least

$$\epsilon' = \epsilon - 2/n.$$

4.3.9 Putting Things Together

For given δ and ϵ in Theorem 4.3, one can choose the parameters n and q involved in the above reductions so as to obtain an overall reduction with as small as possible complexity blow-up.

The parameter n must be chosen so that $\epsilon > 2/n$ since otherwise ϵ' is not positive. One can for example choose $n = 4/\epsilon$. The parameter q must be chosen sufficiently large so as to provide the necessary amplification.

4.3.10 Summary

Let us summarize the complete argument, which consists of five steps, each making use of one of the ideas and providing a reduction of a computational problem Q to another computational problem P (where $P = Q$ is possible). Each such reduction is characterized by the following two factors:

- The mapping of a solver S for P to a solver T for Q .
- The mapping translating the performance of S to the performance of T .

The five relevant computational problems are DL^H , DL^I , LSB^I , LSB^{Y_x} , and LSB^H . The five reduction steps, in the order as they are composed (not in the order in which they were discussed), are:

- DL^H is reduced to DL^I (idea 3): The given instance is shifted (at most) n times such that one of the shifted instances lies in the initial segment I . The solver for DL^I must be called at most n times, and the performance remains the same.

¹²The same statement holds for games, without the factor 2.

- DL^I is reduced to LSB^I (idea 1): The solver for LSB^I is called $t = \lceil \log_2(\lfloor m/n \rfloor + 1) \rceil$ times. If the performance for LSB^I is $1 - \delta'$, then the performance for DL^I is $1 - 2t\delta'$. Since this should be at least $1 - \delta$, we have to choose $\delta' = \delta/(2t)$.
- LSB^I is reduced to itself (idea 4): The solver for LSB^I is called q times, such that the performance is amplified from ϵ' to $1 - \delta'$.
- LSB^I is reduced to LSB^{Y_x} (idea 2): The solver for LSB^{Y_x} is called once, and the performance remains identical (namely ϵ').
- LSB^{Y_x} is reduced to LSB^H (idea 5): The solver is the same but the performance is reduced by $2/n$ from ϵ to $\epsilon' = \epsilon - 2/n$.

4.3.11 Some Remarks

We point out that, formally, the reduction of LSB^I to LSB^{Y_x} via randomization is actually a reduction for a specific x , i.e., for a problem with a single instance. The claimed reduction holds for every x , and the reduction of LSB^{Y_x} to LSB^H (idea 5) also works for every $x \in \mathcal{I}$. Therefore their combination works for every $x \in \mathcal{I}$ and hence for the problem LSB^I .¹³

4.4 Abstract Computational Problems and Reductions

4.4.1 Introduction

The goal of complexity theory is to investigate the computational complexity of computational problems. More precisely, one wants to characterize the best achievable performance in solving the problem as a function of the complexity of the solver. In other words, for a given complexity value c one maximizes the achievable performance over all solvers with complexity at most c .

To achieve this goal, one would need a concrete computational model in which complexity is defined. The choice of such a model would, at least to some extent, be arbitrary, and such a treatment would detract from the essence of what reductions are about. We therefore provide a more abstract treatment of reductions. However, one could always take a more concrete viewpoint by thinking of a solver as the set of all algorithms of a given complexity, i.e., one can think of a solver as being a complexity value if one insists on taking a complexity-based viewpoint.

¹³This illustrates (once more) that reductions make sense even for problems which are not hard, for example a problem with a single instance. Reductions are not only used to prove hardness implications.

4.4.2 Preliminaries

For the composition of functions f and g we write gf instead of $g \circ f$. Hence we write, for example, $hgf(a)$ instead of $h(g(f(a)))$. Sometimes we write simply $f a$ instead of $f(a)$.

For functions $f_i : \mathcal{A}_i \rightarrow \mathcal{B}_i$ (for $1 \leq i \leq k$), $(f_1, \dots, f_k)$ denotes the direct product of $f_1, \dots, f_k$, i.e., the function $\mathcal{A}_1 \times \dots \times \mathcal{A}_k \rightarrow \mathcal{B}_1 \times \dots \times \mathcal{B}_k$ defined by

$$(f_1, \dots, f_k)(a_1, \dots, a_k) = (f_1(a_1), \dots, f_k(a_k)).$$

For functions $f_i : \mathcal{A} \rightarrow \mathcal{B}_i$ (for $1 \leq i \leq k$), $[f_1, \dots, f_k]$ denotes the function $\mathcal{A} \rightarrow \mathcal{B}_1 \times \dots \times \mathcal{B}_k$ defined by

$$[f_1, \dots, f_k](a) = (f_1(a), \dots, f_k(a)).$$

The function assigning to a list of real numbers their sum is denoted as sum . For example, $\text{sum}(3, 4, 5) = 12$. Similarly, for boolean-valued lists we denote by $\wedge$ (or $\vee$) the logical AND (or logical OR) of the values. For functions $f : \mathcal{A} \rightarrow \mathcal{B}$ and $g : \mathcal{A} \rightarrow \mathcal{B}$, where a partial order $\leq$ is defined on $\mathcal{B}$, we write $f \leq g$ to mean $\forall a (f(a) \leq g(a))$.

4.4.3 Problems, Solvers, and Performance

In a theory of computation, at the most abstract level one considers two types of objects.

- *Problems* are objects that should be computed or solved.
- *Solvers* are objects that compute or solve a problem.

Definition 4.2. A problem p is (an object) equipped with a set Σ_p of solvers, a partially ordered set $(\Omega_p; \leq)$ of performance values¹⁴, and a performance function

$$\bar{p} : \Sigma_p \rightarrow \Omega_p$$

assigning a performance value $\bar{p}(s)$ to every solver $s \in \Sigma_p$. A solver s for which $\bar{p}(s) \geq a$ for $a \in \Omega_p$ is called an a -solver for p .

Performance values are often real numbers (i.e., $\Omega_p \subseteq \mathbb{R}$), for example a success probability or a distinguishing advantage, but one can also consider more general performance sets, for example pairs of real numbers. We will consider different types of problems, for example games with performance set $[0, 1]$ and distinction problems with performance set $[-1, 1]$.

¹⁴Here $a \leq b$ means that the performance value b is (in some sense) at least as good as a .

4.4.4 Upper Bounds on Performance

An important goal in cryptography and complexity theory is to prove that a certain problem p is hard (e.g., the problem of breaking a cryptosystem). Ideally, one wants to prove an upper bound on the performance of any solver for p , i.e., a statement of the form $\forall s \bar{p}(s) \leq \epsilon$ for some small ϵ . This can also be written simply as

$$\bar{p} \leq \epsilon,$$

where ϵ is understood as the constant function $\Sigma_p \rightarrow \Omega_p$ mapping every solver to the value ϵ . (Note that usually ϵ is real-valued.) Such a statement is often called *information-theoretic* or *unconditional* since it holds for any solver. In particular, it does not depend on the implementation complexity.

However, since for a usual computational problem p there always exists a (possibly inefficient) solver that solves the problem well (i.e., with high performance), one proves a weaker statement where the upper bound depends on the performance function $\bar{q}$ of another problem q . Such a statement is called a reduction. More generally, $\bar{p}$ can also be upper bounded in terms of the performance functions $\bar{q}_i$ ($i = 1, \dots, k$) of several problems.

4.4.5 The Reduction Concept

An important concept in computer science is to use a solver s for a problem p with performance at least a (i.e., $\bar{p}(s) \geq a$) to construct a solver s' for a problem q with performance at least a' (i.e., $\bar{q}(s') \geq a'$), where $s' = \rho(s)$ for a function

$$\rho : \Sigma_p \rightarrow \Sigma_q$$

(called the *reduction function*) and where $a' = \tau(a)$ for a $\leq$ -respecting function

$$\tau : \Omega_p \rightarrow \Omega_q$$

(called the *performance translation function*).¹⁵ This can be summarized by the function inequality

$$\tau \bar{p} \leq \bar{q} \rho \tag{4.1}$$

which states that for any a -solver s for p , $\rho(s)$ is a $(\tau(a))$ -solver for q :

$$\forall s \in \Sigma_p : \tau \bar{p}(s) \leq \bar{q} \rho(s).$$

In the following, performance translation functions will tacitly be assumed to be $\leq$ -respecting.

A special case of (4.1) is $\bar{p} \leq \bar{q} \rho$, where τ is the identity function, i.e., ρ transforms a solver for q into an equally good solver for p . Another special case of (4.1) is $\tau \bar{p} \leq \bar{q}$, where ρ is the identity function, i.e., every solver for p is also a solver for q with performance related via the function τ .

¹⁵ τ is $\leq$ -respecting if

$$a \leq b \Rightarrow \tau(a) \leq \tau(b),$$

i.e., a better solver for s does not result in a worse solver for s' .

Definition 4.3. For functions ρ and τ satisfying (4.1), ρ is called a τ -reduction of problem q to problem p .¹⁶

4.4.6 Interpretations of Reductions

The usefulness and strength of a reduction is determined by two factors:

- by how much ρ blows up the complexity of a solver s (i.e., reduces the efficiency), and
- by how much τ reduces the performance. (In some cases, τ can also increase the performance.)

Inequality (4.1) can be interpreted in two different ways: as implying a lower bound on $\bar{q}$ (in terms of $\bar{p}$), or as implying an upper bound on $\bar{p}$ (in terms of $\bar{q}$). For the latter interpretation it is convenient to write the inequality in a different form, by considering a $\leq$ -respecting function $\lambda : \Omega_q \rightarrow \Omega_p$ satisfying

$$\text{id} \leq \lambda \tau \quad (4.2)$$

(i.e., $a \leq \lambda \tau a$ for all $a \in \Omega_q$). If Ω_p and Ω_q are intervals of $\mathbb{R}$ and if τ is strictly $\leq$ -respecting, then one can choose $\lambda = \tau^{-1}$ (i.e., $\text{id} \leq \tau$). We have

$$\begin{aligned} \tau \bar{p} \leq \bar{q} \rho &\implies \lambda \tau \bar{p} \leq \lambda \bar{q} \rho \\ &\implies \bar{p} \leq \lambda \bar{q} \rho, \end{aligned} \quad (4.3)$$

where in the first step we have used the $\leq$ -respecting property of λ . If equality holds in (4.2), then (4.3) and (4.1) are (logically) equivalent. Inequality (4.3) is the form of a reduction statement used to state an upper bound on the performance of any solver for $\bar{p}$, in terms of the function $\bar{q}$.

4.4.7 Complexity-Theoretic Interpretation

This subsection is not needed for understanding the rest of the course.

Recall the discussion of Section 4.4.1 about complexity. Let us assume that all problems we consider have the same solver set Σ and that we have fixed a computational model with a complexity notion assigning to every solver $s \in \Sigma$ a complexity value $\gamma(s)$ in some domain Γ of complexity values, for example $\Gamma = \mathbb{N}$. Here $\gamma(s)$ is the implementation complexity when s is optimally implemented in the computational model. We can then associate with a complexity value c the class Σ_c of all solvers with at most that complexity:

$$\Sigma_c = \{s \mid \gamma(s) \leq c\}.$$

¹⁶The term “reduction” is motivated by the fact that, by virtue of the reduction, the problem of finding a solver for q is “reduced” to the problem of finding a solver for p .

We also consider a reduction $\rho : \Sigma \rightarrow \Sigma$.

Now, one particular interpretation of abstract reductions is that, formally, the (derived) solver set Σ' is the set Γ of complexity values, and the (derived) performance function $\bar{p}'$ is a function $\Gamma \rightarrow \Omega$ mapping complexities to performance and assigning to a complexity value c the best achievable performance for any solver with complexity at most c :

$$\bar{p}'(c) = \sup \{\bar{p}(s) \mid s \in \Sigma_c\}.$$

Moreover, the (derived) reduction function ρ' is a function $\Gamma \rightarrow \Gamma$ mapping complexities to complexities. A complexity value c is mapped to the maximal possible complexity of a solver resulting from applying the reduction ρ to any solver with complexity at most c :

$$\rho'(c) = \sup \{\gamma(\rho(s)) \mid s \in \Sigma_c\}.$$

We do not make this more formal but point out that a computational model would have to come with an exact description of the function ρ' . This would mean, among other things, that the maximal complexity of an algorithm A with complexity c invoking an algorithm B with complexity c' would need to be defined as a function of c and c' . This is very tedious, and to some extent arbitrary, already for simple computational models. The use of asymptotics allows to eliminate the arbitrariness to some extent, but it comes at the (high) price of a loss of concreteness and significance.

4.4.8 Composition of Reductions

Reductions can be composed:

Lemma 4.5. $\tau \bar{p} \leq \bar{q} \rho \wedge \tau' \bar{q} \leq \bar{r} \rho' \implies \tau' \tau \bar{p} \leq \bar{r} \rho' \rho.$

Proof. Composing both sides of $\tau \bar{p} \leq \bar{q} \rho$ with (the $\leq$ -respecting function) τ' on the left side results in $\tau' \tau \bar{p} \leq \tau' \bar{q} \rho$ since τ' is $\leq$ -respecting. Composing both sides of $\tau' \bar{q} \leq \bar{r} \rho'$ with ρ on the right side results in $\tau' \bar{q} \rho \leq \bar{r} \rho' \rho$.¹⁷ Combining the two inequalities yields $\tau' \tau \bar{p} \leq \bar{r} \rho' \rho$. $\square$

Reduction statements of the form (4.3) also compose:

Lemma 4.6. $\bar{p} \leq \lambda \bar{q} \rho \wedge \bar{q} \leq \lambda' \bar{r} \rho' \implies \bar{p} \leq \lambda \lambda' \bar{r} \rho' \rho.$

4.4.9 Generalized Reductions

A reduction statement $\tau \bar{p} \leq \bar{q} \rho$ can be generalized by letting the construction make use of a list $s = (s_1, \dots, s_n)$ of solvers, one for each problem in a list $p = (p_1, \dots, p_n)$ of problems. Letting, accordingly, τ be a $\leq$ -respecting¹⁸ function

¹⁷Note that $\tau' \bar{q} \leq \bar{r} \rho'$ means that $\tau' \bar{q}(s) \leq \bar{r} \rho'(s)$ for all $s \in \Sigma_p$, and hence it also holds for all s in the image of ρ , which is a subset of Σ_q . Therefore $\tau' \bar{q} \rho \leq \bar{r} \rho' \rho$.

¹⁸This means that $(\forall i (a_i \leq b_i)) \implies \tau(a_1, \dots, a_n) \leq \tau(b_1, \dots, b_n)$.

$\tau : \Omega_{p_1} \times \cdots \times \Omega_{p_n} \rightarrow \Omega_q$ and letting ρ be a function $\rho : \Sigma_{p_1} \times \cdots \times \Sigma_{p_n} \rightarrow \Sigma_q$, inequality (4.1) can be generalized to

$$\tau(\bar{p}_1(s_1), \dots, \bar{p}_n(s_n)) \leq \bar{q}\rho(s_1, \dots, s_n)$$

for all $(s_1, \dots, s_n)$ or, equivalently, the function inequality

$$\tau\bar{p} \leq \bar{q}\rho. \quad (4.4)$$

In other words, ρ yields a $(\tau(a_1, \dots, a_n))$ -solver $\rho(s_1, \dots, s_n)$ for q from any list of a_i -solvers s_i for p_i .

A reduction statement of the form $\bar{p} \leq \lambda \bar{q}\rho$ can also be generalized by replacing q by a list $\mathbf{q} = (q_1, \dots, q_k)$ of problems with the same solver set Σ , ρ by a list $\boldsymbol{\rho} = [\rho_1, \dots, \rho_k]$ of functions with $\rho_i : \Sigma \rightarrow \Sigma_{q_i}$, and λ by a $\leq$ -respecting function $\lambda : \Omega_{q_1} \times \cdots \times \Omega_{q_k} \rightarrow \Omega_p$. Let $\bar{\mathbf{q}} = (\bar{q}_1, \dots, \bar{q}_k)$. We can now consider the statement that

$$\bar{p}(s) \leq \lambda(\bar{q}_1\rho_1(s), \dots, \bar{q}_k\rho_k(s))$$

for all $s \in \Sigma$, which can equivalently be written as

$$\bar{p} \leq \lambda \bar{\mathbf{q}}\boldsymbol{\rho}, \quad (4.5)$$

where

$$\lambda \bar{\mathbf{q}}\boldsymbol{\rho} = \lambda[\bar{q}_1\rho_1, \dots, \bar{q}_k\rho_k] = \lambda(\bar{q}_1, \dots, \bar{q}_k)[\rho_1, \dots, \rho_k].$$

This inequality states that the performance of a solver s for p is upper bounded by a function λ of the performances of the solvers $\rho_1(s), \dots, \rho_k(s)$ on the problems $q_1, \dots, q_k$, respectively. A typical choice of function λ is the sum (of real numbers).

Composition statements for generalized reductions follow similarly to Lemma 4.6.

4.4.10 Worst-Case Problems

For a set $\mathcal{P}$ of problems one can define the corresponding worst-case problem, which is formally again a problem and denotes as $\mathcal{P}$.

Definition 4.4. For a set $\mathcal{P}$ of problems with the same solver set Σ and the same performance set Ω , where $\Omega \subseteq \mathbb{R}$, the *worst-case problem*, denoted also as $\mathcal{P}$, is the problem whose performance function $\bar{\mathcal{P}} : \Sigma \rightarrow \mathbb{R}$ is defined as follows: the performance of a solver s is the worst performance for any problem in $\mathcal{P}$:

$$\bar{\mathcal{P}}(s) = \inf_{p \in \mathcal{P}} \bar{p}(s).$$

Example 4.9. A computational problem, like the discrete logarithm problem, has typically many instances, each of which can be understood as a problem (with a single instance). Then the worst-case version of the problem (e.g. the worst-case DL-problem) is obtained by Definition 4.4.

4.5 Basic Types of Computational Problems

In this section we discuss the three basic types of computational problems important in cryptography: games, distinction problems, and bit-guessing problems, as instantiations of the abstract problem/solver concepts of the previous section. We recall Section 2.3 where these problem types were already discussed.

4.5.1 Games and Multi-games

To instantiate the concept of a game, we consider a setting with a set $\mathcal{G}$ of (deterministic) games and a set $\mathcal{W}$ of (deterministic) winners, and a function

$$\omega : \mathcal{W} \times \mathcal{G} \rightarrow \{0, 1\},$$

where 0 and 1 denote the Boolean truth values, and $\omega(w, g) = 1$ is interpreted as w winning game g .¹⁹

Definition 4.5. A game G is a $\mathcal{G}$ -valued random variable over $\mathcal{G}$, a winner W is a $\mathcal{W}$ -valued random variable,²⁰ the performance set is $\Omega = [0, 1]$, and the performance of W for game G is the winning probability, i.e.,

$$\bar{G}(W) := \Pr^{WG}(\omega(W, G) = 1).$$

We have already seen several examples of games, for example the function inversion problem or the DL problem.

Multi-games. A more general setting is when a game consists of several (say k) subgames, i.e., when there are k functions $\omega_1, \dots, \omega_k$ with

$$\omega_i : \mathcal{W} \times \mathcal{G} \rightarrow \{0, 1\},$$

where $\omega_i(w, g) = 1$ is interpreted as w winning the i -th subgame of g . Such a game g (or the probabilistic counter-part G) is called a *multi-game*.²¹

Example 4.10. We recall the hash-then-sign example discussed in Chapter 2, which can be described as a multi-game with three winning conditions:

1. the hash collision event (g_1),
2. the forgery event for the short-message signature scheme (g_2), and
3. the forgery event for the overall (long-message) signature scheme (g_3).

We showed that if g_3 is won, then either g_1 or g_2 is won, i.e.,

$$\omega_3(w, g) \rightarrow (\omega_1(w, g) \vee \omega_2(w, g)).$$

¹⁹Note that $\mathcal{G}$ and $\mathcal{W}$ are understood as arbitrary (abstract) sets, not necessarily discrete systems.

²⁰That is, the solver set Σ is the set of $\mathcal{W}$ -valued random variables.

²¹We often use the same symbols g (or G) for a game or a multi-game when no confusion can arise.

For the probabilistic case we therefore have

$$\Pr(\omega_3(W, G) = 1) \leq \Pr(\omega_1(W, G) = 1) + \Pr(\omega_2(W, G) = 1),$$

which can be written more concisely as the reduction inequality

$$\overline{G_3} \leq \overline{G_1} + \overline{G_2}.$$

Definition 4.6. For a multi-game g with k subgames, we denote by $g^\vee$ (and by $g^\wedge$) the game corresponding to the logical OR (the logical AND) of the k subgames, i.e., the game of winning at least one (respectively all) of the games:

$$\omega(w, g^\vee) = \bigvee_{i=1}^k \omega_i(w, g).$$

4.5.2 Distinction Problems

To instantiate the concept of a distinction problem (using the abstract concepts of Section 4.4), we consider a setting with a set $\mathcal{O}$ of objects, a set $\mathcal{D}$ of (deterministic) distinguishers, and a function

$$\kappa: \mathcal{D} \times \mathcal{O} \rightarrow \{0, 1\},$$

where 0 and 1 denote the Boolean truth values.

Definition 4.7. A *distinction problem* is a pair (S_0, S_1) of $\mathcal{O}$ -valued random variables (or, more precisely, distributions), and will be denoted as $\langle S_0 | S_1 \rangle$. A distinguisher D is a $\mathcal{D}$ -valued random variable, and the performance of D for distinction problem $\langle S_0 | S_1 \rangle$ is

$$\overline{\langle S_0 | S_1 \rangle}(D) := \Delta^D(S_0, S_1) = \Pr^{DS_1}(\kappa(D, S_1) = 1) - \Pr^{DS_0}(\kappa(D, S_0) = 1).$$

We recall Definition 2.7 that for a class $\mathcal{D}' \subseteq \mathcal{D}$ of distinguishers, the advantage is

$$\Delta^{\mathcal{D}'}(S, T) := \sup_{D \in \mathcal{D}'} \Delta^D(S, T).$$

If one can complement the bit $\kappa(d, s)$ “output” by a distinguisher $d \in \mathcal{D}'$, i.e., if for every $d \in \mathcal{D}'$ there is a $d' \in \mathcal{D}'$ such that $\kappa(d', s) = \kappa(d, s) \oplus 1$,²² then we have $\Delta^{d'}(S_0, S_1) = -\Delta^d(S_0, S_1)$ and therefore

$$\Delta^{\mathcal{D}'}(S_0, S_1) = \sup_{D \in \mathcal{D}'} |\Delta^D(S_0, S_1)|.$$

Indeed, we have:

²²This is the case for any reasonable set $\mathcal{D}'$.

Lemma 4.7. If $\mathcal{D}'$ is closed under complementing the output bit (see above), then $\Delta^{\mathcal{D}'}$ is a pseudo-metric.²³

Proof. Left as an exercise. $\square$

Lemma 2.2 can be restated as

$$\overline{\langle S_0 | S_k \rangle} = \text{sum}(\overline{\langle S_0 | S_1 \rangle}, \dots, \overline{\langle S_{k-1} | S_k \rangle}). \quad (4.6)$$

4.5.3 Bit-Guessing Problems

To instantiate the concept of a bit-guessing problem (using the abstract concepts of Section 4.4), we again, as for distinction problems, consider a setting with a set $\mathcal{O}$ of objects, a set $\mathcal{D}$ of (deterministic) distinguishers, and a function $\kappa: \mathcal{D} \times \mathcal{O} \rightarrow \{0, 1\}$.

Definition 4.8. A *bit-guessing problem* is a pair a $\mathcal{O} \times \{0, 1\}$ -valued random variable (S, B) , and is denoted as $[S; B]$. A distinguisher (or bit-guesser) D is a $\mathcal{D}$ -valued random variable, and the performance of D for bit-guessing problem $[S; B]$ is

$$\overline{[S; B]}(D) := \Lambda^D((S, B)) = 2 \cdot \left(\Pr^{DS}(\kappa(D, S) = B) - \frac{1}{2} \right).$$

The performance $\overline{[S; B]}(D)$ is between -1 and 1 , where performance 1 means that D 's guess is correct with probability 1 , and performance -1 means that D 's guess is correct probability 0 .

We recall Lemmas 2.3 and 2.4 which state that

$$\overline{[S_U; U]} = \overline{\langle S_0 | S_1 \rangle}, \quad (4.7)$$

where U is an unbiased random bit, independent of anything else, and

$$\overline{(S, B)} = 2 \cdot \overline{\langle (S, B) | (S, U) \rangle} p. \quad (4.8)$$

4.6 Discrete Computational Problems

Up to now, the treatment of computational problems and reductions was at an abstract level and holds for any instantiation of these concepts. In order to instantiate games as in Definition 4.5 we need to define the sets $\mathcal{G}$ and $\mathcal{W}$ as well as the function $\omega: \mathcal{W} \times \mathcal{G} \rightarrow \{0, 1\}$. In order to instantiate distinction and bit-guessing problems as in Definitions 4.7 and 4.8 we need to define the sets $\mathcal{O}$ and $\mathcal{D}$ as well as the function $\kappa: \mathcal{D} \times \mathcal{O} \rightarrow \{0, 1\}$. This was done in Section 3.7.

²³Recall that a pseudo-metric on a set $\mathcal{X}$ is a function $f: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ satisfying $f(x, x) = 0$, $f(x, y) = f(y, x)$, and $f(x, z) \leq f(x, y) + f(y, z)$ for all $x, y, z \in \mathcal{X}$ (triangle inequality).

From now on we will use bold-face letters to refer to discrete systems, where small letters (e.g. g) and capital letters (e.g. G) refer to deterministic and probabilistic systems, respectively.

4.7 Basic Reduction Types

We consider two simple types of reduction functions ρ for discrete game winners and distinguishers: reductions by a converter and reductions by multiple invocation.

4.7.1 Multiple Independent Instantiation and Cloning

We introduce two types of operations on random variables: multiple independent copies and multiple clones. These operations can be applied to probabilistic systems, understood as random variables over a set of (deterministic) systems.

Definition 4.9. For an $\mathcal{X}$ -random variable X , we denote by X^q the $\mathcal{X}^q$ -random variable consisting of q independent copies of X . More precisely, $X^q = (X_1, \dots, X_q)$ where $X_1, \dots, X_q$ are independent and each X_i has the same (marginal) probability distribution as X . Moreover, we denote by $\langle X \rangle$ the $\mathcal{X}^\infty$ -random variable consisting of a countable number independent copies of X .²⁴

Definition 4.10. For an $\mathcal{X}$ -random variable X , we denote by $X^{[q]}$ the $\mathcal{X}^q$ -random variable consisting of q clones of X . More precisely, $X^{[q]} = (X_1, \dots, X_q)$ where $X_1 = \dots = X_q$ and each X_i has the same (marginal) probability distribution as X .

Example 4.11. If X is a binary random variable with uniform distribution, then X^q is the random variable with uniform distribution over $\{0, 1\}^q$, and $X^{[q]}$ is the $\{0, 1\}^q$ -random variable which takes on the two values $(0, 0, \dots, 0)$ and $(1, 1, \dots, 1)$ with probability $\frac{1}{2}$.

We will use these two concepts for the case where the random variables are discrete probabilistic systems (systems, environments, or converters). If S is a probabilistic system, then we denote by S^q the parallel composition of q independent copies of S , and by $S^{[q]}$ the parallel composition of q clones of S which all behave identically. If C is a (probabilistic) converter, then C^q is the q -fold instantiation of C , and we have $C^q S^q = (CS)^q$. If E is a (probabilistic) environment, then E^q is the q -fold instantiation of E , where the order of outputs given

²⁴Formally, when considering random variables over $\mathcal{X}^\infty$, one leaves the realm of discrete probability theory since $\mathcal{X}^\infty$ is not countable. The formal treatment then becomes more cumbersome. However, we can think of an infinite sequence as standing for the (infinite) sequence of finite-length prefixes, which means that we consider an infinite sequence of discrete random experiments. This should not cause confusion in the rest of the course.

by the different instantiations is considered to be arbitrary but fixed. If E^q is connected to S^q , the execution can be thought of as proceeding in parallel in all q copies, where the relative order is not relevant.

4.7.2 Reductions by a Converter

Recall Definition 3.8. The following holds for the deterministic or the probabilistic case. A discrete $((\mathcal{U}, \mathcal{V}), (\mathcal{X}, \mathcal{Y}))$ -converter c transforms a $(\mathcal{X}, \mathcal{Y})$ -DDS s into a $(\mathcal{U}, \mathcal{V})$ -DDS cs . This also holds for discrete games g as we view the monotone binary output as not being seen or affected by the system connected to g . In other words, if g is a game, then cg is also a game.

A converter c can also be understood as a transformation of a winner w into another winner, denoted wc , where we have

$$\omega(wc, g) = \omega(w, cg). \quad (4.9)$$

One can view a (possibly probabilistic) converter C as a reduction function ρ mapping probabilistic winners to probabilistic winners. We denote this function as ρ^C :

$$\rho^C : W \mapsto \rho^C(W) = WC.$$

Using (4.9), we thus have the following trivial reduction equality:

$$\overline{CG} = \overline{G} \rho^C, \quad (4.10)$$

where on the left side the converter C and system G are composed to form a game, and on the right side the functions $\overline{G}$ and ρ^C are composed as functions.

4.7.3 Reductions by Multiple Instantiation

The second type of reduction is by multiple invocation of a solver, for example a winner W . The reduction function ρ mapping W to W^q is denoted as σ^q , i.e.,

$$\sigma^q : W \mapsto \sigma^q(W) = W^q.$$

4.8 Some Reduction Statements for Games

4.8.1 Performance Amplification for Games by Repetition

A natural idea for amplifying the performance of a game winner W for a game G is to repeat it independently for the same instance of the problem, hoping that one of the attempts will be successful. We saw in Exercise 4.1 that performance amplification by repetition is not guaranteed to work, i.e., the performance is not necessarily greater than for a single attempt. Nevertheless, repetition is one

of the most important reduction techniques. However, as such, it only works for independent instances (see below).

Let $\psi_q(x)$ denote the probability that among q independent binary random variables, each of which is 1 with probability x , at least one of them is 1. Moreover, let $\chi_q : [0, 1] \rightarrow [0, 1]$ denote the inverse function, i.e., $\chi_q = \psi_q^{-1}$:

Definition 4.11. We define

$$\psi_q(x) := 1 - (1 - x)^q$$

and the inverse function

$$\chi_q(x) := 1 - (1 - x)^{1/q}.$$

We note that $\chi_q \circ \psi_q = \text{id}$, the identity function. We also note (using $1 + x \leq e^x$ for all x) that $\psi_q(x) \geq 1 - e^{-xq}$ and hence that

$$\chi_q(x) \leq \frac{-\ln(1 - x)}{q}. \quad (4.11)$$

Note that $\mathbf{G}^q$ is a multi-game and recall that $\mathbf{G}^{q^\vee}$ denotes the game consisting of winning one of the q independent copies of the game $\mathbf{G}$. Equation (4.12) of the following lemma states that for any winner $\mathbf{W}$ for $\mathbf{G}$, the performance of $\mathbf{W}^q$ for the problem $\mathbf{G}^{q^\vee}$ is strongly amplified, namely by the function ψ_q .

Lemma 4.8. For any game $\mathbf{G}$ and any q we have

$$\psi_q \overline{\mathbf{G}} = \overline{\mathbf{G}^{q^\vee} \sigma^q}. \quad (4.12)$$

Proof. Equation (4.12) (of functions) can also be written as

$$\forall \mathbf{W} \quad \psi_q \overline{\mathbf{G}}(\mathbf{W}) = \overline{\mathbf{G}^{q^\vee}}(\mathbf{W}^q). \quad (4.13)$$

If $\mathbf{W}^q$ is connected to $\mathbf{G}^q$, this corresponds to q independent copies of $\mathbf{W}$ connected to $\mathbf{G}$, i.e., $\mathbf{W}^q \mathbf{G}^q = (\mathbf{W} \mathbf{G})^q$. The q MBO's of the q copies of $\mathbf{G}$ are hence independent bits, each of which is equal to 1 with probability $\overline{\mathbf{G}}(\mathbf{W})$. Hence their logical OR is 1 with probability $\psi_q(\overline{\mathbf{G}}(\mathbf{W}))$. This corresponds to (4.13). $\square$

The above reduction only works for independent copies of $\mathbf{G}$, i.e., for $\mathbf{G}^{q^\vee}$. What one actually wants is to amplify the performance for a *given* instance $\mathbf{g}$ of a game. In order for this to work, a game must have two properties: it must be clonable and it must be self-reducible, two concepts discussed in the next two subsections.

4.8.2 Cloning Game Systems

In many reductions one needs multiple copies of the *same* instance of a game. For a search problem it is obvious what it means to copy an instance if it is assumed to be represented as a bit-string. In contrast, for general games, which can be systems with state, it may not even be possible to repeat the game in the same initial configuration. This is possible only if one can *clone* a given instance of a game.

Definition 4.12. A (game) system $\mathbf{G}$ is called *q-clonable by converter $\mathbf{K}$* if²⁵

$$\mathbf{K} \mathbf{G} \equiv \mathbf{G}^{[q]^\vee}.$$

Recall the notion of equivalence of probabilistic systems (Definition 3.19). If two systems are equivalent, they behave identically in any environment, i.e., they generate the same transcript distribution. In particular, for any winner $\mathbf{W}$, the winning probability of two equivalent games $\mathbf{G}$ and $\mathbf{H}$ is the same: $\overline{\mathbf{G}}(\mathbf{W}) = \overline{\mathbf{H}}(\mathbf{W})$, or in short: $\overline{\mathbf{G}} = \overline{\mathbf{H}}$. This implies

$$\overline{\mathbf{K} \mathbf{G}} = \overline{\mathbf{G}^{[q]^\vee}}.$$

A cloning-converter $\mathbf{K}$ can typically be defined as emulating q copies of $\mathbf{G}$ at the left interface, interacting with a winner (designed for the game $\mathbf{G}^{[q]^\vee}$), checking any obtained solution (for one of the subgames) for correctness, and then forwarding a correct solution, if there is any, at the right interface to $\mathbf{G}$. If $\mathbf{G}$ is won, this means that one of the clones of $\mathbf{G}$ was won, i.e., that $\mathbf{G}^{[q]^\vee}$ is won. Hence we have

$$\overline{\mathbf{G}^{[q]^\vee}} = \overline{\mathbf{G}} \rho^{\mathbf{K}}. \quad (4.14)$$

Exercise 4.6. Explain why the MAC-game does not seem to be clonable by an efficient system. What about the MAC game if the adversary has to forge the MAC for a fixed message, say for the all-zero string of a certain length?

4.8.3 Random Self-Reduction of Games

Definition 4.13. A probabilistic game $\mathbf{G}$ defined as a probability distribution over a set $\mathcal{G}$ of DDS (the instance set) is called *random self-reducible* by converter $\mathbf{R}$ if

$$\forall \mathbf{g} \in \mathcal{G} : \mathbf{R} \mathbf{g} \equiv \mathbf{G}.$$

The following lemma states that for a random game $\mathbf{G}$ that is random self-reducible by $\mathbf{R}$, an average-case winner can be transformed by the reduction function $\rho^{\mathbf{R}}$ into a worst-case winner with the same success probability.

²⁵Note that $\mathbf{K}$ depends on q , but to simplify the notation we do not make this explicit.

Lemma 4.9. *If $\mathbf{G}$ is random self-reducible by $\mathbf{R}$, then*

$$\overline{\mathbf{G}} = \overline{\mathcal{G}} \rho^{\mathbf{R}}.$$

Proof. For every instance $\mathbf{g} \in \mathcal{G}$ and every winner $\mathbf{W}$ we have $\overline{\mathbf{g}}(\mathbf{WR}) = \overline{\mathbf{G}}(\mathbf{W})$, hence

$$\overline{\mathbf{G}}(\mathbf{W}) = \inf_{\mathbf{g} \in \mathcal{G}} \overline{\mathbf{g}}(\mathbf{WR}) = \overline{\mathcal{G}}(\mathbf{WR}) = \overline{\mathcal{G}} \rho^{\mathbf{R}}(\mathbf{W}).$$

□

4.8.4 Combining Clonability and Random Self-Reduction

We now show that random self-reducibility and clonability of a game $\mathbf{G}$ can be used to amplify the performance of a winner.

Theorem 4.10. *If game $\mathbf{G}$ is random self-reducible by $\mathbf{R}$ and q -clonable by $\mathbf{K}$, then*

$$\psi_q \overline{\mathbf{G}} = \overline{\mathbf{G}} \rho^{\mathbf{K}} \sigma^q \rho^{\mathbf{R}}.$$

Proof. If $\mathbf{G}$ is random self-reducible by $\mathbf{R}$, then

$$\mathbf{R}^q \mathbf{G}^{[q]} \equiv \mathbf{G}^q$$

and hence also

$$\overline{\mathbf{G}^{[q]}}^{\vee} \rho^{\mathbf{R}^q} = \overline{\mathbf{R}^q \mathbf{G}^{[q]}}^{\vee} = \overline{\mathbf{G}^q}^{\vee}. \quad (4.15)$$

Using clonability of $\mathbf{G}$ via $\mathbf{K}$ and (4.14), as well as equations (4.15) and (4.12), we obtain

$$\begin{aligned} \psi_q \overline{\mathbf{G}} &= \overline{\mathbf{G}^q}^{\vee} \sigma^q \\ &= \overline{\mathbf{G}^{[q]}}^{\vee} \rho^{\mathbf{R}^q} \sigma^q \\ &= \overline{\mathbf{G}} \rho^{\mathbf{K}} \rho^{\mathbf{R}^q} \sigma^q \\ &= \overline{\mathbf{G}} \rho^{\mathbf{K}} \sigma^q \rho^{\mathbf{R}}. \end{aligned}$$

The last step follows from $\rho^{\mathbf{R}^q} \sigma^q = \sigma^q \rho^{\mathbf{R}}$ since

$$\rho^{\mathbf{R}^q} \sigma^q(\mathbf{W}) = \rho^{\mathbf{R}^q} \mathbf{W}^q = \mathbf{W}^q \mathbf{R}^q = (\mathbf{WR})^q = \sigma^q(\mathbf{WR}) = \sigma^q \rho^{\mathbf{R}}(\mathbf{W}).$$

□

The reduction function in Theorem 4.10 is $\rho^{\mathbf{K}} \sigma^q \rho^{\mathbf{R}}$, and hence the theorem states that for any winner $\mathbf{W}$ for $\mathbf{G}$, the winning probability of $\rho^{\mathbf{K}} \sigma^q \rho^{\mathbf{R}}(\mathbf{W}) = \mathbf{W}^q \mathbf{R}^q \mathbf{K}$ for $\mathbf{G}$ is strongly amplified, namely by the function ψ_q .

We briefly discuss the consequences of this result in terms of asymptotic statements. If the reduction function $\rho^{\mathbf{K}} \sigma^q \rho^{\mathbf{R}}$ is considered as preserving the

efficiency of a winner $\mathbf{W}$, then if no efficient winner can win $\mathbf{G}$ with probability p , this implies that no efficient winner can win $\mathbf{G}$ with probability $\chi_q(p)$. In view of (4.11), this probability $\chi_q(p)$ can be made arbitrarily small, even for p very close to 1, by an appropriate choice of q . There is a trade-off: increasing the parameter q makes the upper bound of the theorem stronger (i.e., smaller), but on the other hand makes $\mathbf{W}^q \mathbf{R}^q \mathbf{K}$ less efficient.

From this statement one can obtain the following asymptotic statement as a corollary: If it is computationally hard to solve $\mathbf{G}$ with probability more than β (for any β , e.g. $\beta = 0.999$) then it is even hard to achieve any non-negligible success probability. In other words, a weak intractability assumption provably implies a strong one.

Example 4.12. The DL problem in group $\mathcal{H} = \langle h \rangle$ is random self-reducible and efficiently clonable. The converter R takes an instance $y = h^x$ as input (at the right interface), chooses $r \in \{0, \dots, |\mathcal{H}| - 1\}$ at random, outputs $y' = y \cdot h^r$ at the left interface, and for the (attempted) solution w at the left interface outputs $w - r$ at the right interface.

This randomizer system $\mathbf{R}$ is correct since the value h^r and hence also $y' = y \cdot h^r$ is a uniformly random group element and because w is a solution for y' if and only if $w - r$ is a solution for y .

Exercise 4.7. Show that the CDH problem is random self-reducible.

4.9 Hardness Amplification for Games

4.9.1 Combining Several Games

One can combine several independent games $\mathbf{G}_1, \dots, \mathbf{G}_k$ into a single game by requesting that *all* games be won. Recall Definition 4.6 and that this game is denoted as

$$[\mathbf{G}_1, \dots, \mathbf{G}_k]^{\wedge}.$$

Example 4.13. Recall the function inversion game for a function f . Given several functions (which could be the same), one can define the combined game of inverting all the functions for independent uniformly random inputs. This is equivalent to inverting the direct product of all functions on a uniformly random input.

4.9.2 The Goal of Hardness Amplification

We want to investigate the following apparently simple statement:

If one cannot win game $\mathbf{G}$ with probability better than β and one cannot win game $\mathbf{H}$ with probability better than γ , then, if $\mathbf{G}$ and $\mathbf{H}$ are independent, one cannot win both games with probability better than $\beta\gamma$.

This statement is trivially true for games corresponding to search problems if we consider computationally unbounded winners.²⁶ If β is the probability that $\mathbf{G}$ can be won and γ is the probability that $\mathbf{H}$ can be won, then $\beta\gamma$ is the probability that both games can be won.

However, if we consider *computationally bounded* winners, things are substantially more involved. One can say that $\mathbf{G}$ is hard above β (denoted $\text{hard}(\mathbf{G}, \beta)$, and possibly defined precisely later), if no efficient winner has a winning probability of non-negligibly more than β . Then the statement we want to prove is: **If $\mathbf{G}$ is hard above β and $\mathbf{H}$ is hard above γ , then $[\mathbf{G}, \mathbf{H}]^\wedge$ is hard above $\beta\gamma$:**

$$\text{hard}(\mathbf{G}, \beta) \wedge \text{hard}(\mathbf{H}, \gamma) \implies \text{hard}([\mathbf{G}, \mathbf{H}]^\wedge, \beta\gamma).$$

The proof of this apparently simple statement is not as easy as it may seem, and it holds only for clonable games. One has to be careful about what this statement means, i.e., what needs to be proved. It means the following:

If there exists an efficient $\beta\gamma$ -winner for $[\mathbf{G}, \mathbf{H}]^\wedge$, then there exists either an efficient β -winner for $\mathbf{G}$ or an efficient γ -winner for $\mathbf{H}$.

A constructive proof of this statement shows how one can construct, given any efficient $\beta\gamma$ -winner $\mathbf{W}$ for $[\mathbf{G}, \mathbf{H}]^\wedge$, either an efficient β -winner for $\mathbf{G}$ or an efficient γ -winner for $\mathbf{H}$.

Before we state the theorem and the proof, we need a lemma.

4.9.3 A Lemma on Multi-Argument $[0, 1]$ -Valued Functions

In this section we state a general lemma on functions $\mathcal{X} \rightarrow [0, 1]$. Such a function, say $\mu : \mathcal{X} \rightarrow [0, 1]$, can be interpreted as follows. We consider a random experiment in which an event $\mathcal{E}$ is defined and where the values $x \in \mathcal{X}$ correspond to (mutually exclusive) cases that can occur. Then $\mu(x)$ can be interpreted as the probability of the event $\mathcal{E}$ in case x . In our application, x will be an instance of a game and $\mathcal{E}$ will be the event that a given winner W wins this game instance.

We now consider the case where $\mathcal{X} = \mathcal{S} \times \mathcal{T}$, i.e., a function $\mu : \mathcal{S} \times \mathcal{T} \rightarrow [0, 1]$, as well as probability distributions P_S on $\mathcal{S}$ and P_T on $\mathcal{T}$, defining (independent) random variables S and T , respectively. Moreover, we define $\mu_1(s)$ as the average of μ when $S = s$, i.e.,

$$\mu_1(s) = E_T[\mu(s, T)] = \sum_{t \in \mathcal{T}} P_T(t) \mu(s, t)$$

and, similarly, $\mu_2(t)$ as the average of μ when $T = t$:

$$\mu_2(t) = E_S[\mu(S, t)] = \sum_{s \in \mathcal{S}} P_S(s) \mu(s, t).$$

²⁶For general games it is also true, but the statement is not trivial (see U. Maurer, K. Pietrzak, and R. Renner, *Indistinguishability amplification*, Proc. CRYPTO 2007).

Note that μ_1 is a function $\mathcal{S} \rightarrow [0, 1]$ and μ_2 is a function $\mathcal{T} \rightarrow [0, 1]$. We have

$$E_{ST}[\mu(S, T)] = E_S[\mu_1(S)] = E_T[\mu_2(T)].$$

For some $\epsilon \geq 0$ we are interested in the probability (over S) that $\mu_1(S) \geq \epsilon$, i.e., in

$$\Pr^S(\mu_1(S) \geq \epsilon),$$

and, similarly, for some ϵ' , in the probability (over T) that $\mu_2(T) \geq \epsilon'$

$$\Pr^T(\mu_2(T) \geq \epsilon').$$

Lemma 4.11. For every $0 \leq \epsilon, \epsilon' < 1$ and every function $\mu : \mathcal{S} \times \mathcal{T} \rightarrow [0, 1]$,

$$E_{ST}[\mu(S, T)] \leq \Pr^S(\mu_1(S) \geq \epsilon) \cdot \Pr^T(\mu_2(T) \geq \epsilon') + \epsilon + \epsilon'.$$

Proof. Let

$$\mathcal{S}' = \{s : \mu_1(s) \geq \epsilon\} \quad \text{and} \quad \mathcal{T}' = \{t : \mu_2(t) \geq \epsilon'\},$$

as well as $\mathcal{S}'' = \mathcal{S} \setminus \mathcal{S}'$ and $\mathcal{T}'' = \mathcal{T} \setminus \mathcal{T}'$. Using

$$\mathcal{S} \times \mathcal{T} = (\mathcal{S}' \times \mathcal{T}') \cup (\mathcal{S}'' \times \mathcal{T}) \cup (\mathcal{S} \times \mathcal{T}'') \quad (4.16)$$

we can bound

$$E_{ST}[\mu(S, T)] = \sum_{(s,t) \in \mathcal{S} \times \mathcal{T}} P_S(s) P_T(t) \mu(s, t) \quad (4.17)$$

as the sum of three terms corresponding to the summations over the three sets on the right side of (4.16).²⁷ Since $\mu(s, t) \leq 1$, the first of these terms can be bounded as

$$\sum_{(s,t) \in \mathcal{S}' \times \mathcal{T}'} P_S(s) P_T(t) \mu(s, t) \leq \underbrace{\Pr^S(S \in \mathcal{S}')}_{=\Pr^S(\mu_1(S) \geq \epsilon)} \cdot \underbrace{\Pr^T(T \in \mathcal{T}')}_{=\Pr^T(\mu_2(T) \geq \epsilon')}. \quad (4.18)$$

Moreover, since $\mu_1(s) < \epsilon$ for $s \in \mathcal{S}''$, we have

$$\begin{aligned} \sum_{(s,t) \in \mathcal{S}'' \times \mathcal{T}} P_S(s) P_T(t) \mu(s, t) &= \sum_{s \in \mathcal{S}''} P_S(s) \underbrace{\sum_{t \in \mathcal{T}} P_T(t) \mu(s, t)}_{=\mu_1(s)} \\ &\leq \Pr^S(S \in \mathcal{S}'') \cdot \epsilon \\ &\leq \epsilon \end{aligned} \quad (4.19)$$

and, analogously,

$$\sum_{(s,t) \in \mathcal{S} \times \mathcal{T}''} P_S(s) P_T(t) \mu(s, t) \leq \Pr^T(T \in \mathcal{T}'') \cdot \epsilon' \leq \epsilon'. \quad (4.20)$$

Combining equations (4.16) to (4.20) completes the proof. $\square$

²⁷This is best illustrated in a rectangular figure (see lecture).



Figure 4.2: The converters $\underline{H}$ (left) and $\underline{G}$ (right). The converter $\underline{H}$ emulates H , connects to a system (say S) at the right interface, and provides an interface to both systems, H and S , at the left interface. The output at the right interface of H is ignored. $\underline{G}$ is defined analogously.

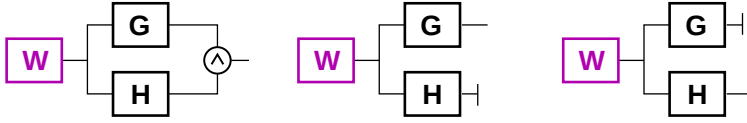


Figure 4.3: The winner W connected to game $[G, H]^\wedge$ (left), the winner $W\underline{H}$ connected to game G (middle), and the winner $W\underline{G}$ connected to game H (right).

4.9.4 Hardness Amplification for Two Instances

We consider the problem of winning two independent games G and H , i.e., we consider the game $[G, H]^\wedge$ and a winner W for this game. We assume that G and H are clonable by K and by L , respectively.

Definition 4.14. Let $\underline{H}$ and $\underline{G}$ be the converters shown in Figure 4.2.

The idea behind this definition is as follows (see also Figure 4.3). We want to consider the game consisting of G and H , but where H 's MBO is ignored (rather than ANDed with G 's MBO). In other words, while a winner W “plays” with both games G and H , the game we consider, which can be written as $\underline{H}G$, is won already if G alone is won. (Note that this is *not* the game G .) Clearly, a winner W winning game $[G, H]^\wedge$ also wins game $\underline{H}G$, i.e., winner $W\underline{H}$ wins game G . More precisely, we have the logical implication

$$\omega(w, [g, h]^\wedge) \rightarrow \omega(w, \underline{h}g)$$

for all $g \in \mathcal{G}$, $h \in \mathcal{H}$, and $w \in \mathcal{W}$, which implies

$$\overline{[g, h]^\wedge}(w) \leq \overline{\underline{h}g}(w) = \overline{g}(w\underline{h}). \quad (4.21)$$

Since this inequality holds for all arguments, it also holds on average, i.e., we have for example, for every $g \in \mathcal{G}$,

$$\overline{[g, H]^\wedge}(W) \leq \overline{g}(W\underline{H}). \quad (4.22)$$

To make use of Lemma 4.11, we let $\mathcal{S} = \mathcal{G}$ and $\mathcal{T} = \mathcal{H}$ be the set of instances of game G and of game H , respectively. Moreover we define, for a given (in the following fixed) α -solver W for $[G, H]^\wedge$, the function $\mu : \mathcal{G} \times \mathcal{H} \rightarrow [0, 1]$ as follows:

$$\mu(g, h) := \overline{[g, h]^\wedge}(W),$$

i.e., $\mu(g, h)$ is the success probability of W for the instance $[g, h]^\wedge$. By definition of probabilistic games we have

$$E_{GH}[\mu(G, H)] = \overline{[G, H]^\wedge}(W). \quad (4.23)$$

Then $\mu_1(g) = E_H[\mu(g, H)]$ denotes W 's success probability, given that $G = g$. Thus we have

$$\begin{aligned} \mu_1(g) &= E_H[\mu(g, H)] \\ &= E_H[\overline{[g, H]^\wedge}(W)] \\ &\leq E_H[\overline{g}(W\underline{H})] \\ &= \overline{g}(W\underline{H}), \end{aligned} \quad (4.24)$$

where the inequality follows from inequality (4.22) and the last step corresponds to the fact that (by Definition 4.5) the performance of a probabilistic winner (here $W\underline{H}$) is the expected value of the winning probability over the winner's randomness. Similarly, $\mu_2(h) = E_G[\mu(G, h)]$ denotes W 's success probability, given that $H = h$, and we have

$$\mu_2(h) \leq \overline{h}(W\underline{G}). \quad (4.25)$$

Let

$$A := \Pr^G(\mu_1(G) \geq \epsilon) \quad (4.26)$$

and

$$B := \Pr^H(\mu_2(H) \geq \epsilon'). \quad (4.27)$$

Lemma 4.11 and equation (4.23) imply that

$$\overline{[G, H]^\wedge}(W) \leq AB + \epsilon + \epsilon'. \quad (4.28)$$

Equation (4.24), namely $\overline{g}(W\underline{H}) \geq \mu_1(g)$, states that, for every game instance $g \in \mathcal{G}$, the winner $W\underline{H}$ has success probability at least $\mu_1(g)$. Thus, according to (4.26), with probability at least A (over instances g of G), the success probability of $W\underline{H}$ is at least ϵ , i.e., $\overline{g}(W\underline{H}) \geq \epsilon$. Therefore, the q -fold winner $(W\underline{H})^q = W^q \underline{H}^q$ has success probability at least $A \cdot \psi_q(\epsilon)$ in winning game $G^{[q]}$ (see Definition 4.11).²⁸ Thus, for q large enough such that

$$\psi_q(\epsilon) \geq 1/\sqrt{1+\delta} \quad (4.29)$$

²⁸In this argument we ignore that contribution to the success probability of the instances g for which $\overline{g}(W\underline{H}) < \epsilon$.

holds, we obtain

$$\overline{\mathbf{G}}(\mathbf{W}^q \mathbf{H}^q \mathbf{K}) = \overline{\mathbf{G}^{[q]^\vee}}(\mathbf{W}^q \mathbf{H}^q) \geq A \cdot \psi_q(\epsilon) \geq A/\sqrt{1+\delta}, \quad (4.30)$$

where in the first step we have used the fact that $\mathbf{G}$ is clonable by $\mathbf{K}$, i.e., $\overline{\mathbf{G}^{[q]^\vee}} = \overline{\mathbf{K}\mathbf{G}} = \overline{\mathbf{G}}\rho^K$. Similarly, using $\overline{\mathbf{H}^{[q']^\vee}} = \overline{\mathbf{L}\mathbf{H}} = \overline{\mathbf{H}}\rho^L$ and assuming that q' is large enough such that

$$\psi_{q'}(\epsilon') \geq 1/\sqrt{1+\delta}, \quad (4.31)$$

we obtain

$$\overline{\mathbf{H}}(\mathbf{W}^{q'} \mathbf{G}^{q'} \mathbf{L}) \geq B \cdot \psi_{q'}(\epsilon') \geq B/\sqrt{1+\delta}. \quad (4.32)$$

Using equations (4.30) and (4.32) we obtain

$$AB \leq (1+\delta) \cdot \overline{\mathbf{G}}(\mathbf{W}^q \mathbf{H}^q \mathbf{K}) \cdot \overline{\mathbf{H}}(\mathbf{W}^{q'} \mathbf{G}^{q'} \mathbf{L}).$$

We can now let $\epsilon = \epsilon' = \delta'/2$ and choose q and q' large enough so that (4.29) and (4.31) are satisfied, which is the case (as one sees after a short calculation) for

$$q = q' \approx \frac{2 \ln(2/\delta)}{\delta'}. \quad (4.33)$$

Using equation 4.28 we have thus proved the following theorem. Recall the notion of a generalized reduction of Section 4.4.9.

Theorem 4.12. *If $\mathbf{G}$ and $\mathbf{H}$ are clonable by $\mathbf{K}$ and $\mathbf{L}$, respectively, then for any $\delta, \delta' > 0$ we have the generalized reduction*

$$(\overline{\mathbf{G}}, \overline{\mathbf{H}})^\wedge \leq \lambda(\overline{\mathbf{G}}, \overline{\mathbf{H}}) [\rho_1, \rho_2]$$

for

$$\lambda(x, y) := (1+\delta)xy + \delta'$$

for $\rho_1(\mathbf{W}) = \mathbf{W}^q \mathbf{H}^q \mathbf{K}$ and $\rho_2(\mathbf{W}) = \mathbf{W}^{q'} \mathbf{G}^{q'} \mathbf{L}$, where q and q' are as described.

The theorem has two parameters, δ and δ' , which one wants to choose as small as possible. Examining (4.33), we note that δ' is the more critical parameter since q and q' are proportional to its inverse, while the dependence of q and q' on δ is only logarithmic.

4.9.5 Hardness Amplification for Many Instances

Hardness amplification can be generalized from $k = 2$ to a general number k of games, i.e., to the game $[\mathbf{G}_1, \dots, \mathbf{G}_k]^\wedge$. Lemma 4.11 can be generalized as follows, where for simplicity we let all ϵ 's be equal.

Lemma 4.13. *For any function $\mu : S_1 \times \dots \times S_k \rightarrow [0, 1]$ we have*

$$\mathbb{E}_{S_1 \dots S_k} [\mu(S_1, \dots, S_k)] \leq \prod_{i=1}^k \Pr^{S_i} (\mathbb{E}_{S_1 \dots S_{i-1} S_{i+1} \dots S_k} [\mu(S_1, \dots, S_k)] \geq \epsilon) + k\epsilon.$$

Proof. Left as an exercise. $\square$

Theorem 4.12 can now be generalized to the game $[\mathbf{G}_1, \dots, \mathbf{G}_k]^\wedge$. To state the theorem is left as an exercise.

We are now interested in the special case where all the games $\mathbf{G}_i$ are identical, i.e., we consider the game $\mathbf{G}^{k^\wedge}$ consisting of solving k independent instances of $\mathbf{G}$ simultaneously. We leave as an exercise to prove the following theorem, where ρ must be an appropriately defined reduction which invokes an appropriate number q of independent copies of a winner for $\mathbf{G}^{k^\wedge}$.

Theorem 4.14. *For any $k \in \mathbb{N}$, any $\delta, \delta' > 0$ and any clonable (by $\mathbf{K}$) game $\mathbf{G}$ we have*

$$\overline{\mathbf{G}^{k^\wedge}} \leq \lambda \overline{\mathbf{G}} \rho$$

for $\lambda(x) = (1+\delta)x^k + \delta'$.

The theorem can be shown to imply

$$\text{hard}(\mathbf{G}, \beta) \implies \text{hard}(\mathbf{G}^{k^\wedge}, \beta^k).$$

This is a hardness amplification statement because k independent copies of $\mathbf{G}$ are much harder to solve than a single copy of $\mathbf{G}$.

4.10 Relating Games and Distinction Problems

Recall Section 4.6 (and hence also Section 3.7), where discrete games, distinction problems, and bit-guessing problems were introduced. In this section we investigate relations between discrete games and distinction problems. This is important because it is generally easier to show the hardness of a game than the hardness of a distinction problem, and the transition between the two is required if one wants to prove a statement about indistinguishability, as for example in constructive cryptography, based on assumptions about the hardness of one or several games.

As explained in Section 3.7, a game $\mathbf{G}$, a winner $\mathbf{W}$, an object $\mathbf{S}$, and a distinguisher $\mathbf{D}$ are probabilistic discrete systems. The statements we make in this section depend only on the behavior of these systems (see Definition 3.19), i.e., they are not affected by which element of the equivalence class of systems with the same behavior is considered. We can therefore understand a statement about (system) behavior as a statement holding for any specific probabilistic system with that behavior.

In this section we make the following convenient simplification (which is not necessary). We only consider environments (a winner $\mathbf{W}$ or a distinguisher $\mathbf{D}$) that stop after a finite number of queries, and we let q be an upper bound on the number of queries. Without loss of generality, and just for the purpose of the analysis, we can think of $\mathbf{W}$ or $\mathbf{D}$ as, after stopping, continuing up to q queries by making dummy queries. In the analysis we therefore consider transcripts (X^q, Y^q) of q inputs and outputs rather than transcripts of variable length.

4.10.1 Game Equivalence

Since for a game $\mathbf{G}$ we are only interested in how $\mathbf{G}$ can be won (but not what happens afterwards), the behavior of $\mathbf{G}$ is characterized as follows:

Definition 4.15. The *pre-winning behavior* of a game $\mathbf{G}$ is the sequence

$$p_{Y^i, A_i=0|X^i Y^{i-1}, A_{i-1}=0}^{\mathbf{G}} \quad \text{for } i \geq 1 \quad (4.34)$$

of conditional probability distributions.

The following definition captures a restricted type of equivalence of games, capturing only that they behave equivalently *as long as the game is not won*.

Definition 4.16. Two (probabilistic) games $\mathbf{G}$ and $\mathbf{H}$ are *equivalent as games*, denoted

$$\mathbf{G} \stackrel{g}{=} \mathbf{H},$$

if their pre-winning behavior is identical.

Recall that the cumulative description of the behavior of $\mathbf{G}$ is given by

$$p_{Y^i, A_i|X^i}^{\mathbf{G}} \quad \text{for } i \geq 1.$$

One can show that an equivalent condition for $\mathbf{G} \stackrel{g}{=} \mathbf{H}$ is

$$p_{Y^i, A_i=0|X^i}^{\mathbf{G}}(y^i \cdot x^i) = p_{Y^i, A_i=0|X^i}^{\mathbf{H}}(y^i \cdot x^i)$$

for $i \geq 1$ and for all x^i and y^i .

Consider the random experiment where a winner $\mathbf{W}$ is connected to a game system $\mathbf{G}$, and consider a fixed number q of queries. The probability distribution of the transcript (X^q, Y^q) and the event “game not won” is given, as already explained in Section 3.6.5, by

$$\begin{aligned} p_{X^q Y^q, A_q=0}^{\mathbf{WG}}(x^q, y^q) &= \\ &= \prod_{i=1}^q \left(p_{X^i|X^{i-1} Y^{i-1}}^{\mathbf{W}}(x_i, x^{i-1}, y^{i-1}) \cdot p_{Y^i, A_i=0|X^i Y^{i-1}, A_{i-1}=0}^{\mathbf{G}}(y_i, x^i, y^{i-1}) \right) \\ &= p_{X^q|Y^{q-1}}^{\mathbf{W}}(x^q, y^{q-1}) \cdot p_{Y^q, A_q=0|X^q}^{\mathbf{G}}(y^q, x^q), \end{aligned} \quad (4.35)$$

where we have made use of

$$p_{X^q|Y^{q-1}}^{\mathbf{W}}(x^q, y^{q-1}) = \prod_{i=1}^q p_{X^i|X^{i-1} Y^{i-1}}^{\mathbf{W}}(x_i, x^{i-1}, y^{i-1})$$

and

$$p_{Y^q, A_q=0|X^q}^{\mathbf{G}}(y^q, x^q) = \prod_{i=1}^q p_{Y^i, A_i=0|X^i Y^{i-1}, A_{i-1}=0}^{\mathbf{G}}(y_i, x^i, y^{i-1}). \quad (4.36)$$

The winning probability $\overline{\mathbf{G}}(\mathbf{W})$ of winner $\mathbf{W}$ making at most q queries for game $\mathbf{G}$ is

$$\begin{aligned} \overline{\mathbf{G}}(\mathbf{W}) &= \Pr^{\mathbf{WG}}(A_q = 1) = 1 - \Pr^{\mathbf{WG}}(A_q = 0) \\ &= 1 - \sum_{x^q \in \mathcal{X}^q} \sum_{y^q \in \mathcal{Y}^q} p_{X^q Y^q, A_q=0}^{\mathbf{WG}}(x^q, y^q). \end{aligned} \quad (4.37)$$

If $\mathbf{G}$ and $\mathbf{H}$ are equivalent as games, then the process of winning either of them is identical up to the point where it is won. Therefore we have:

Lemma 4.15. If $\mathbf{G} \stackrel{g}{=} \mathbf{H}$ then, for any winner $\mathbf{W}$ for $(\mathcal{X}, \mathcal{Y})$ -games, we have

$$\overline{\mathbf{G}} = \overline{\mathbf{H}}.$$

Proof. According to (4.37), the term $\overline{\mathbf{G}}(\mathbf{W})$ is computed as 1 minus the sum of $|\mathcal{X}|^q \cdot |\mathcal{Y}|^q$ terms, each of which, according to (4.35), is identical in the two random experiments since the probability of any event defined on the transcript (X^q, Y^q) of $\mathbf{W}$ with $\mathbf{G}$ (or with $\mathbf{H}$) for which $A_q = 0$, is the same for $\mathbf{G}$ and $\mathbf{H}$. This follows from (4.36). In particular, we have $\Pr^{\mathbf{WG}}(A_q = 0) = \Pr^{\mathbf{WH}}(A_q = 0)$ and thus $\Pr^{\mathbf{WG}}(A_q = 1) = \Pr^{\mathbf{WH}}(A_q = 1)$, i.e., $\overline{\mathbf{G}}(\mathbf{W}) = \overline{\mathbf{H}}(\mathbf{W})$. $\square$

Definition 4.17. We denote the maximal winning probability for game $\mathbf{G}$ as

$$\Gamma(\mathbf{G}) := \sup_{\mathbf{W}} \overline{\mathbf{G}}(\mathbf{W}).$$

4.10.2 Distinguishing $(\mathcal{X}, \mathcal{Y})$ -Systems

Consider the random experiment where a distinguisher $\mathbf{D}$ is connected to a system $\mathbf{S}$, and again consider a fixed number q of queries. Let us denote $\mathbf{D}$ ’s output bit as Z . It is generated according to the conditional probability distribution

$$p_{Z|X^q Y^q}^{\mathbf{D}}.$$

In analogy to (4.35), the probability distribution $p_{X^q Y^q Z}^{\mathbf{DS}}$ of the transcript (X^q, Y^q, Z) can be expressed as

$$p_{X^q Y^q Z}^{\mathbf{DS}}(x^q, y^q, z) = p_{X^q|Y^{q-1}}^{\mathbf{D}}(x^q, y^{q-1}) \cdot p_{Y^q|X^q}^{\mathbf{S}}(y^q, x^q) \cdot p_{Z|X^q Y^q}^{\mathbf{D}}(z, x^q, y^q).$$

Recall that

$$\Delta^{\mathbf{D}}(\mathbf{S}, \mathbf{T}) = \overline{\langle \mathbf{S} | \mathbf{T} \rangle}(\mathbf{D}) = \Pr^{\mathbf{DT}}(Z = 1) - \Pr^{\mathbf{DS}}(Z = 1)$$

and that the maximal distinguishing advantage for systems $\mathbf{S}$ and $\mathbf{T}$ as

$$\Delta(\mathbf{S}, \mathbf{T}) := \sup_{\mathbf{D}} \Delta^{\mathbf{D}}(\mathbf{S}, \mathbf{T}).$$

4.10.3 Relating Game Winning and Distinguishing

For a game it is useful to define the system when the MBO is ignored:

Definition 4.18. For an $(\mathcal{X}, \mathcal{Y} \times \{0, 1\})$ -system $\mathbf{S}$ with MBO we define $\mathbf{S}^-$ as the $(\mathcal{X}, \mathcal{Y})$ -system resulting from $\mathbf{S}$ by ignoring the MBO, i.e.,

$$p_{Y_i | X^i Y^{i-1}}^{\mathbf{S}^-} = p_{Y_i | X^i Y^{i-1}}^{\mathbf{S}}.$$

The following lemma states that if $\mathbf{S} \stackrel{g}{=} \mathbf{T}$, then the distinguishing advantage of any distinguisher $\mathbf{D}$ in distinguishing $\mathbf{S}^-$ and $\mathbf{T}^-$ is upper bounded by $\mathbf{D}$'s winning probability for game $\mathbf{S}$, i.e., $\overline{\langle \mathbf{S}^- | \mathbf{T}^- \rangle}(\mathbf{D}) \leq \overline{\mathbf{S}}(\mathbf{D})$ for all $\mathbf{D}$.

Lemma 4.16. If $\mathbf{S} \stackrel{g}{=} \mathbf{T}$, then

$$\overline{\langle \mathbf{S}^- | \mathbf{T}^- \rangle} \leq \overline{\mathbf{S}}.$$

Proof. Let q be the maximal number of queries that $\mathbf{D}$ makes. We have

$$\begin{aligned} \overline{\langle \mathbf{S}^- | \mathbf{T}^- \rangle}(\mathbf{D}) &= \Pr^{\mathbf{DT}}(Z = 1) - \Pr^{\mathbf{DS}}(Z = 1) \\ &= \Pr^{\mathbf{DT}}(Z = 1 \wedge A_q = 0) + \Pr^{\mathbf{DT}}(Z = 1 \wedge A_q = 1) \\ &\quad - \Pr^{\mathbf{DS}}(Z = 1 \wedge A_q = 0) - \Pr^{\mathbf{DS}}(Z = 1 \wedge A_q = 1) \\ &= \Pr^{\mathbf{DT}}(Z = 1 \wedge A_q = 1) - \Pr^{\mathbf{DS}}(Z = 1 \wedge A_q = 1) \\ &\leq \Pr^{\mathbf{DT}}(Z = 1 \wedge A_q = 1) \\ &\leq \Pr^{\mathbf{DT}}(A_q = 1) \\ &= \overline{\mathbf{T}}(\mathbf{D}) \\ &= \overline{\mathbf{S}}(\mathbf{D}). \end{aligned}$$

In the first step we have used the fact that the event $Z = 1$ is the union of the two (mutually exclusive) events $(Z = 1 \wedge A_q = 0)$ and $(Z = 1 \wedge A_q = 1)$. In the second step we have used the fact that

$$\Pr^{\mathbf{DS}}(Z = 1 \wedge A_q = 0) = \Pr^{\mathbf{DT}}(Z = 1 \wedge A_q = 0).$$

This follows since, as mentioned above, the probability of any event defined on the transcript (X^q, Y^q) , which is a subset of the event $A_q = 0$, is identical in the random experiments defined by $\mathbf{DS}$ and $\mathbf{DT}$. More specifically:

$$\Pr^{\mathbf{DS}}(Z = 1, A_q = 0) = \sum_{x^q \in \mathcal{X}^q} \sum_{y^q \in \mathcal{Y}^q} \left(p_{X^q Y^q, A_q=0}^{\mathbf{DS}}(x^q, y^q) p_{Z | X^q Y^q}^{\mathbf{D}}(1, x^q, y^q) \right),$$

and replacing $\mathbf{S}$ by $\mathbf{T}$ leaves all terms in this expression unchanged. In the fourth step we have used the fact that the event $A_q = 1$ is a superset of the event $(Z = 1 \wedge A_q = 1)$. In the last step we have used Lemma 4.15. $\square$

4.11 Proving Indistinguishability

In this section we investigate techniques for proving the information-theoretic indistinguishability of two systems, i.e., for bounding the distinguishing advantage of any distinguisher, with no restriction on the computing power.

4.11.1 Conditional Equivalence

The following notion will lead to a useful tool for proving the indistinguishability of systems.

Definition 4.19. For an $(\mathcal{X}, \mathcal{Y}) \times \{0, 1\}$ -system $\mathbf{S}$ with MBO (denoted A_i) and an $(\mathcal{X}, \mathcal{Y})$ -system $\mathbf{T}$ we say that $\mathbf{S}$ is *conditionally equivalent* to $\mathbf{T}$, denoted

$$\mathbf{S} \equiv \mathbf{T},$$

if, for $i \geq 1$,²⁹

$$p_{Y^i | X^i, A_i=0}^{\mathbf{S}} = p_{Y^i | X^i}^{\mathbf{T}}.$$

Since $p_{Y^i, A_i=0 | X^i}^{\mathbf{S}} = p_{A_i=0 | X^i}^{\mathbf{S}} \cdot p_{Y^i | X^i, A_i=0}^{\mathbf{S}}$, the above condition is equivalent to

$$p_{Y^i, A_i=0 | X^i}^{\mathbf{S}} = p_{A_i=0 | X^i}^{\mathbf{S}} \cdot p_{Y^i | X^i}^{\mathbf{T}}. \quad (4.38)$$

This shows that $\mathbf{T}$ is actually the system $\mathbf{S}^-$. That a system $\mathbf{S}$ is conditionally equivalent to some system (namely $\mathbf{S}^-$) can in view of (4.38) be interpreted as $\mathbf{S}$ being *factorizable* into a system component generating the Y_i -outputs and an independent system component generating the A_i -outputs, as long as the MBO is 0.

²⁹Two conditional probability distributions are considered to be equal if they are equal for all arguments for which they are both defined. (Here one considers only x^i for which A_i has non-zero probability.)

Example 4.14. Consider the MBO defined (for any system) by $A_i = 0$ if and only if the first i inputs are distinct. Let $\hat{\mathbf{R}}_{m,n}$ denote the game obtained by enhancing the URF $\mathbf{R}_{m,n}$ with this MBO. Then for all m and n we have

$$\hat{\mathbf{R}}_{m,n} \equiv \mathbf{B}_n.$$

Stated informally, a URF $\mathbf{R}_{m,n}$ behaves like a beacon $\mathbf{B}_n$ as long as the inputs are distinct.

Example 4.15. Consider the MBO defined (for any system) by $A_i = 0$ if and only if for any two distinct inputs the corresponding two outputs are distinct. (In particular, if all inputs are distinct, then all outputs are distinct.). Then we have

$$\hat{\mathbf{R}}_{n,n} \equiv \mathbf{P}_n$$

as the reader can easily verify as an exercise. Stated informally, a URF $\mathbf{R}_{n,n}$ behaves like a URP $\mathbf{P}_n$ as long as the outputs are distinct (whenever the inputs are distinct).

4.11.2 From Conditional Equivalence to Indistinguishability

In this section we show that conditional equivalence can be used to prove indistinguishability statements in a simple manner, by showing that one can restrict the analysis to *non-adaptive* distinguishers.

Definition 4.20. For a game $\mathbf{S}$ we define $\mathbf{bS}$ as the game system $\mathbf{S}$ for which the outputs Y_i are blocked, i.e., $\mathbf{b}$ is the simple converter that is transparent for the queries X_i but blocks the replies Y_i .

To win game $\mathbf{bS}$ means to win game $\mathbf{S}$ blindly, without seeing the outputs. Equivalently, this means to win the game *non-adaptively* since the inputs $x_1, \dots, x_q$ can be interpreted as being chosen in advance, before seeing any outputs. The best probability in winning a game $\mathbf{S}$ non-adaptively (i.e., $\Gamma(\mathbf{bS})$) is generally lower than the best probability in winning it adaptively (i.e., $\Gamma(\mathbf{S})$). Of interest for us is that proving a bound on the best winning probability is generally much easier for $\mathbf{bS}$.

Often one considers an $(\mathcal{X}, \mathcal{Y})$ -system $\mathbf{S}$ for which one can define an MBO $A_0, A_1, A_2, \dots$. This results in a game $\hat{\mathbf{S}}$, characterized by $p_{Y^i A_i | X^i}^{\hat{\mathbf{S}}}$, where, by definition,

$$\hat{\mathbf{S}}^- = \mathbf{S},$$

$$\text{i.e., } p_{Y^i | X^i}^{\hat{\mathbf{S}}^-} = p_{Y^i | X^i}^{\mathbf{S}}.$$

Definition 4.21. For a system $\mathbf{T}$, let $\tilde{\mathbf{T}}$ denote the converter which is identical to $\mathbf{T}$ at the left interface but, in addition, copies all queries from the left interface to the (system connected to the) right interface. It ignores replies received at the right interface.

The following theorem is useful for indistinguishability proofs. It states that if $\hat{\mathbf{S}} \equiv \mathbf{T}$, then for any distinguisher $\mathbf{D}$, the distinguishing advantage for $\mathbf{S}$ and $\mathbf{T}$ (i.e., $\langle \mathbf{S} | \mathbf{T} \rangle(\mathbf{D})$) is upper bounded by the probability of winner $\mathbf{D}\tilde{\mathbf{T}}$ in winning game $\mathbf{S}$ *blindly*. This also means that the optimal (adaptive) distinguishing advantage for $\mathbf{S}$ and $\mathbf{T}$ is upper-bounded by $\Gamma(\mathbf{bS})$, the optimal non-adaptive game-winning probability for game $\mathbf{S}$. This is useful because non-adaptive settings are generally much easier to analyze.

Theorem 4.17. If for an $(\mathcal{X}, \mathcal{Y})$ -system $\mathbf{S}$ one can define an MBO such that $\hat{\mathbf{S}} \equiv \mathbf{T}$, then

$$\langle \mathbf{S} | \mathbf{T} \rangle \leq \overline{\mathbf{bS}} \circ \rho^{\tilde{\mathbf{T}}}.$$

In particular,

$$\Delta(\mathbf{S}, \mathbf{T}) \leq \Gamma(\mathbf{bS}).$$

Proof. One can enhance $\mathbf{T}$ with an MBO (denoted A_i) to a game $\hat{\mathbf{T}}$, as follows (this can be nicely illustrated in a figure):

$$p_{Y^i A_i | X^i}^{\hat{\mathbf{T}}} = p_{Y^i | X^i}^{\mathbf{T}} \cdot p_{A_i | X^i}^{\hat{\mathbf{S}}}$$

(i.e., $p_{A_i | X^i Y^i}^{\hat{\mathbf{T}}} = p_{A_i | X^i}^{\hat{\mathbf{S}}}$). Then we have

$$\hat{\mathbf{S}} \stackrel{g}{\equiv} \hat{\mathbf{T}} \tag{4.39}$$

since, using $\hat{\mathbf{S}} \equiv \mathbf{T}$,

$$p_{Y^i, A_i=0 | X^i}^{\hat{\mathbf{S}}} = \underbrace{p_{A_i=0 | X^i}^{\hat{\mathbf{S}}}}_{=p_{A_i=0 | X^i}^{\mathbf{T}}} \cdot \underbrace{p_{Y^i | X^i, A_i=0}^{\hat{\mathbf{S}}}}_{=p_{Y^i | X^i}^{\mathbf{T}} = p_{Y^i | X^i}^{\tilde{\mathbf{T}}}} = p_{Y^i, A_i=0 | X^i}^{\hat{\mathbf{T}}}.$$

We observe that

$$\hat{\mathbf{T}} = \tilde{\mathbf{T}}\mathbf{b}\hat{\mathbf{S}} \tag{4.40}$$

since the system $\tilde{\mathbf{T}}\mathbf{b}\hat{\mathbf{S}}$ generates the outputs Y_i and the MBOs A_i independently by $\hat{\mathbf{T}}$ and $\hat{\mathbf{S}}$, respectively. Now we obtain

$$\langle \mathbf{S} | \mathbf{T} \rangle = \langle \hat{\mathbf{S}}^- | \hat{\mathbf{T}}^- \rangle \leq \tilde{\mathbf{S}} = \tilde{\mathbf{T}} = \tilde{\mathbf{T}}\mathbf{b}\hat{\mathbf{S}} = \overline{\mathbf{bS}} \circ \rho^{\tilde{\mathbf{T}}},$$

where the first step follows from $\hat{\mathbf{S}}^- = \mathbf{S}$ and $\hat{\mathbf{T}}^- = \mathbf{T}$, the second step follow from Lemma 4.16, the third step follows from (4.39) and Lemma 4.15, the fourth step follows from (4.40), and the fifth step follows by absorbing $\tilde{\mathbf{T}}$ into the winner (see (4.10)).

Finally, the claim $\Delta(\mathbf{S}, \mathbf{T}) \leq \Gamma(\mathbf{bS})$ trivially follows since $\mathbf{D}\tilde{\mathbf{T}}$ is a game winner for any choice of $\mathbf{D}$. $\square$

4.11.3 Example: The URP-URF “Switching Lemma”

We discuss a simple result which is in the literature usually called the PRP-PRF “switching lemma”, where PRF (PRP) stands for pseudo-random function (permutation). But it is independent of complexity-theoretic arguments and is simply the statement that a URP $\mathbf{P}_n$ is indistinguishable from a URF $\mathbf{R}_{n,n}$ unless the number of queries is close to $2^{n/2}$, which is exponential and generally infeasible. This means that whenever one needs a pseudo-random function (PRF) one can also use a pseudo-random permutation (PRP) without losing much security. Since a block cipher (like AES) is often assumed to be a PRP, it can also be used as a PRF, as is for instance the case in the CBC-MAC.

We need the following definition and lemma.

Definition 4.22. The probability that a set of q independent and uniformly chosen values from an alphabet of size t contains a value twice (a collision) is denoted as $p_{\text{coll}}(t, q)$.

Lemma 4.18. $p_{\text{coll}}(t, q) \leq \frac{1}{2}q^2/t$

Proof. The proof is left as an exercise. $\square$

Recall that $[q]\mathbf{S}$ denotes the system $\mathbf{S}$ limited to q queries.

Lemma 4.19. $\Delta([q]\mathbf{R}_{n,n}, [q]\mathbf{P}_n) \leq \frac{1}{2}q^22^{-n}$

The lemma can also be stated as

$$\langle [q]\mathbf{R}_{n,n} | [q]\mathbf{P}_n \rangle \leq \frac{1}{2}q^22^{-n}.$$

Proof. As shown in Example 4.15, we can define an MBO for the system $\mathbf{R}_{n,n}$, resulting in the game $\hat{\mathbf{R}}_{n,n}$, where the MBO is 1 if and only if for some distinct inputs the outputs are equal (a collision). We have $\hat{\mathbf{R}}_{n,n} \equiv \mathbf{P}_n$ (see Example 4.15) and hence, according to Theorem 4.17, we obtain

$$\Delta([q]\mathbf{R}_{n,n}, [q]\mathbf{P}_n) \leq \Gamma(\mathbf{b}[q]\hat{\mathbf{R}}_{n,n}).$$

It remains to analyze $\Gamma(\mathbf{b}[q]\hat{\mathbf{R}}_{n,n})$. An optimal strategy is to choose the input sequence x^q to $\mathbf{b}[q]\hat{\mathbf{R}}_{n,n}$ with (arbitrary) distinct values. For any such choice of x^q , the probability of causing a collision is $p_{\text{coll}}(2^n, q)$, hence Lemma 4.18 can be applied. $\square$

Chapter 5

Constructive Cryptography

5.1 The Construction Paradigm

In this chapter we discuss selected aspects of a theory, called *constructive cryptography*, that was initiated in 2011¹ and which is still under development.

5.1.1 Modular Construction of Complex Systems

A central paradigm in any constructive discipline, for example in software design, the construction of machines, the definition of communication protocols, and also in security engineering and cryptography, is to construct a powerful complex system from simpler component systems (e.g. called modules), which each may consist of yet simpler component systems, and so on. The technical meaning of the term “construction” differs from context to context.

For example, the purpose of a software component is to correctly implement a system from given library modules. Similarly, the purpose of applying error-correcting codes can be seen as constructing a reliable, error-free communication channel from an unreliable (error-prone or noisy) channel. In the context of cryptography, the purpose of encryption can be seen as constructing a secure communication channel from an authenticated channel and a secret key, as already briefly discussed in Section 2.4.

Such a construction can be represented as a tree.² There are two dual and equivalent interpretations of the iterative construction process leading to this tree: *top-down* and *bottom-up*. In a top-down interpretation one starts from the final object, the root, and in each step one replaces a leaf in the current tree by a subtree, by replacing the object at that leaf by a construction of it from lower-

¹The first paper was: U. Maurer and R. Renner, Abstract Cryptography, *The Second Symposium in Innovations in Computer Science, ICS 2011*, Tsinghua University Press, pp. 1–21, Jan. 2011.

²More generally, it could be a directed acyclic graph.

level objects. This paradigm is sometimes called *step-wise refinement*, especially in software design. In a bottom-up interpretation one starts from a list of (parallel) resources corresponding to the leafs in the tree and subsequently constructs the resources at higher-level nodes in the tree, until one arrives at the root. In both interpretations, each inner node in the tree corresponds to a construction step.

In order for this modular construction approach to be useful in a context where one wants to prove a property of the overall construction (corresponding to the entire tree), it should be sufficient to prove the property *locally*, for each node in the tree. The modular approach can be called *sound* if the property holds globally, provided it holds locally, i.e., if one can use an induction proof on the tree. This can also be called *composition*.

In the following we treat the concept of composable constructions at the most abstract level. The treatment in this section is not specific to cryptography and applies in principle to any constructive discipline. Our goal is to capture the general notion of a construction, i.e., what is common to the various notions of a construction.

5.1.2 Abstract Constructions

Two types of objects are involved in constructions: the objects one constructs, which we call *resources* (or *resource specifications*), and the methods by which one performs a construction, which we call *constructors*. If resource S can be constructed from resource R by means of constructor a , then we write

$$R \xrightarrow{a} S.$$

Typically, S and R are *specifications* of systems (see below) and a is a constructor that is guaranteed (or assumed) to transform any system satisfying specification R into one satisfying specification S .

More generally, a constructor can make use of several resources $R_1, \dots, R_m$, which can be understood as a single resource, denoted $[R_1, \dots, R_m]$, and we can write

$$[R_1, \dots, R_m] \xrightarrow{a} S.$$

We capture the above in the following definition:

Definition 5.1. A *construction* for a resource set Ω and a constructor set Γ is a subset of $\Omega \times \Gamma \times \Omega$ (i.e., a relation). A construction is often denoted by an arrow, for example $\longrightarrow$, as follows: If (R, a, S) is in the construction, then we write $R \xrightarrow{a} S$ and say that S can be *constructed from* R by a . We write $R \longrightarrow S$ if there exists an $a \in \Gamma$ such that $R \xrightarrow{a} S$.

A construction $\longrightarrow$ for a set Ω and a constructor set Γ can equivalently be interpreted as a collection $\{\xrightarrow{a}\}_{a \in \Gamma}$ of relations on Ω , indexed by elements of Γ .

5.1.3 Composition

Constructors can be composed: If S is constructed by a from R and T is constructed by b from S , then the two constructors a and b can be composed to obtain a new constructor, denoted as $a \circ b$. Soundness of the construction approach means that

$$R \xrightarrow{a} S \wedge S \xrightarrow{b} T \implies R \xrightarrow{a \circ b} T. \quad (5.1)$$

This is a property that a construction notion does not automatically satisfy; it must be *proved* for a given definition of a construction notion.

Definition 5.2. A construction $\longrightarrow$ for Ω and Γ is called *composable* if (5.1) holds.

More generally, we assume that a parallel composition (or tuple-building) operation is defined which allows to view several resources $R_1, \dots, R_m$ as a single resource denoted $[R_1, \dots, R_m]$. One can also require that

$$\begin{aligned} [R_1, \dots, R_m] \xrightarrow{a} S_i \wedge [S_1, \dots, S_n] \xrightarrow{b} T \\ \implies [S_1, \dots, S_{i-1}, [R_1, \dots, R_m], S_{i+1}, \dots, S_n] \xrightarrow{a \circ_i b} T, \end{aligned} \quad (5.2)$$

where $a \circ_i b$ is the constructor (where the operation $\circ_i$ would need to be explicitly defined) which “plugs in” constructor a at the i -th argument of constructor b . One can also define parallel composition:

$$\bigwedge_{i=1}^k (R_i \xrightarrow{a_i} S_i) \implies [R_1, \dots, R_k] \xrightarrow{[a_1, \dots, a_k]} [S_1, \dots, S_k], \quad (5.3)$$

where, in addition to sequential composition of constructors, we also assume a list-forming (parallel) composition operation on constructors. Composition of the form (5.2) and (5.3) will not be discussed in this course.

5.2 Resource Specifications, Constructions, and Relaxations

In this section we introduce a specific, very natural instantiation of the construction notion.

5.2.1 Resource Specifications

We consider a set Φ of objects, called resources, and let the set Ω of the previous section be $\Omega = \mathcal{P}(\Phi)$. In the context of this course, resources are probabilistic

³Recall that for a set A , $\mathcal{P}(A)$ denotes the power set of A , i.e., the set of subsets of A .

discrete systems (PDS), see Section 5.3, but in this section they can be seen as (more) abstract objects.

Definition 5.3. A *resource specification* is a subset of Φ . If $R \in \Phi$ satisfies $R \in \mathcal{R}$ we say that R satisfies specification $\mathcal{R}$. For two specifications $\mathcal{R}$ and $\mathcal{S}$, if $\mathcal{R} \subseteq \mathcal{S}$ we say that $\mathcal{R}$ is (at least) as specific as $\mathcal{S}$.

Let $\mathcal{R} \subseteq \mathcal{S}$. For a given resource R , it is more useful to know that $R \in \mathcal{R}$ than to know that $R \in \mathcal{S}$. However, it is often useful to replace a given specification $\mathcal{R}$ by a weaker specification $\mathcal{S}$ if $\mathcal{S}$ is easier to understand and analyse. We can call $\mathcal{S}$ an *abstraction* of $\mathcal{R}$.

5.2.2 Constructions

Consider a set Γ of functions $\Phi \rightarrow \Phi$, called *constructions*. Such functions naturally extend to specifications: For $\gamma \in \Gamma$ we have⁴

$$\gamma(\mathcal{R}) = \{\gamma(R) \mid R \in \mathcal{R}\}.$$

Definition 5.4. A specification $\mathcal{S}$ is *constructed* by γ from specification $\mathcal{R}$, denoted $\mathcal{R} \xrightarrow{\gamma} \mathcal{S}$, if $\gamma(\mathcal{R})$ satisfies specification $\mathcal{S}$, i.e.,

$$\mathcal{R} \xrightarrow{\gamma} \mathcal{S} :\iff \gamma(\mathcal{R}) \subseteq \mathcal{S}.$$

$\mathcal{R}$ and $\mathcal{S}$ are called the *assumed* and the *constructed* resource specification, respectively.

For the identity construction $\text{id} \in \Gamma$ we have

$$\mathcal{R} \subseteq \mathcal{S} \implies \mathcal{R} \xrightarrow{\text{id}} \mathcal{S}.$$

The following lemma states that constructions can be composed.

Lemma 5.1. The construction notion of Definition 5.4 is composable:

$$\mathcal{R} \xrightarrow{\gamma} \mathcal{S} \wedge \mathcal{S} \xrightarrow{\gamma'} \mathcal{T} \implies \mathcal{R} \xrightarrow{\gamma' \circ \gamma} \mathcal{T}.$$

Proof. Left as an exercise. $\square$

5.2.3 Relaxations

Often, a construction statement of the form $\mathcal{R} \xrightarrow{\gamma} \mathcal{S}$ is not true for the desired $\mathcal{S}$, but only for a relaxed version of $\mathcal{S}$.

⁴More generally, we adopt the standard practice in mathematics that the argument of a function is replaced by a set of arguments, then this denotes the resulting set of function values.

Definition 5.5. A *relaxation* ρ is a function $\mathcal{P}(\Phi) \rightarrow \mathcal{P}(\Phi)$ such that $\mathcal{R} \subseteq \rho(\mathcal{R})$.

Typically, a relaxation is defined by a function $\rho : \Phi \rightarrow \mathcal{P}(\Phi)$ which induces a function $\rho : \mathcal{P}(\Phi) \rightarrow \mathcal{P}(\Phi)$ as follows:

$$\rho(\mathcal{R}) = \bigcup_{R \in \mathcal{R}} \rho(R).$$

If a (pseudo-)metric d is defined on Φ , a natural type of relaxation is the so-called ϵ -relaxation: For $\epsilon > 0$ we have

$$\mathcal{R}^\epsilon = \{R' \mid d(R, R') \leq \epsilon\}$$

and hence

$$\mathcal{R}^\epsilon = \{R' \mid \exists R \in \mathcal{R} : d(R, R') \leq \epsilon\}.$$

We will also discuss other relaxation types.

Definition 5.6. A relaxation ρ is *compatible* with the construction set Γ if any construction statement holds also if the assumed and the constructed resource specification is relaxed by ρ , i.e., if

$$\mathcal{R} \xrightarrow{\gamma} \mathcal{S} \implies \rho(\mathcal{R}) \xrightarrow{\gamma} \rho(\mathcal{S}).$$

A function $\gamma \in \Gamma$ is called *non-expanding* for the pseudo-metric d if

$$d(\gamma(R), \gamma(S)) \leq d(R, S).$$

If all $\gamma \in \Gamma$ are non-expanding for d , then the ϵ -relaxation is compatible with Γ .

Definition 5.7. A relaxation ρ is *compatible* with the parallel composition (or tuple-forming) operation if⁵

$$[R_1, \dots, R_{i-1}, \rho(R_i), R_{i+1}, \dots, R_n] \subseteq \rho([R_1, \dots, R_n]).$$

This definition naturally extends to specifications $\mathcal{R}_1, \dots, \mathcal{R}_n$.

These two compatibility properties allow us to compose construction statements that contain relaxations on the right (the constructed) side by “pulling relaxations to the outside”. For example, we have

$$\mathcal{R} \xrightarrow{\gamma} \rho(\mathcal{S}) \wedge \mathcal{S} \xrightarrow{\gamma'} \mathcal{T} \implies \mathcal{R} \xrightarrow{\gamma' \circ \gamma} \rho(\mathcal{T}) \quad (5.4)$$

because $\rho(\mathcal{S}) \xrightarrow{\gamma} \rho(\mathcal{T})$ and hence Lemma 5.1 can be applied to obtain (5.4). We also have

$$\begin{aligned} [R_1, \dots, R_m] \xrightarrow{a} \rho(S_i) \wedge [S_1, \dots, S_n] \xrightarrow{b} T \\ \implies [S_1, \dots, S_{i-1}, [R_1, \dots, R_n], S_{i+1}, \dots, S_n] \xrightarrow{a \circ_i b} \rho(T) \end{aligned}$$

if Definition 5.7 is satisfied.

⁵Recall (see previous footnote) that $R_1, \dots, R_{i-1}, \rho(R_i), R_{i+1}, \dots, R_n$ is a set of resources.

5.3 Constructions for Discrete Systems

In this section we consider the special case of the previous section where the resource set Φ is the set of probabilistic discrete systems (PDS, see Section 3.5), and constructions are defined as application one or more converters.

5.3.1 Interface Types

We consider three different types of interfaces of resources:

- **Party interfaces:** model the access that regular parties have to the resource.
- **Adversary interfaces:** model the access that malicious entities have to the resource. Typically we consider a single adversarial interface, often denoted by E (for Eve).
- **Free interfaces:** model the access that the environment has to the resource.

Example 5.1. An authenticated communication channel can be modeled as having two party-interfaces, A and B , an adversary interface E , as well as a free interface F . Messages can be input at A , forwarded by a trigger input at F , and retrieved at B (if previously forwarded). Messages input at A can be retrieved at E (without need for forwarding). The forwarding at F models that a channel need not be reliable. The idea is that in a channel (e.g. a secure channel) constructed from the authenticated channel, the message is retrievable by B exactly if this is the case in the assumed authenticated channel.

5.3.2 Constructions

A construction is defined by a tuple of converters, one for each party (interface). Such a tuple of converters is often called a *protocol*.

Definition 5.8. A protocol for a set $\mathcal{P} = \{P_1, \dots, P_n\}$ of parties consists of a converter π_i for each party $P_i \in \mathcal{P}$, i.e., it is a tuple $\pi = (\pi_1, \dots, \pi_n)$ of converters. The construction function defined by protocol $\pi = (\pi_1, \dots, \pi_n)$ is

$$\mathcal{R} \mapsto \pi_1^{P_1} \dots \pi_n^{P_n} \mathcal{R}.$$

5.3.3 Resources vs. Converters

Up to now, converters, such as an encryption converter or a decryption converter, were understood as performing some computation (e.g. encrypting a message). A question that arises is where we draw the boundary between a resource providing computing power and the computing power of a converter.

The general guiding principle in constructive cryptography is that everything that is considered relevant (for the analysis one wants to perform) is modeled as a resource. In contrast, converters perform only operations considered to be irrelevant. If, for instance, computing power, memory, or randomness needed for a cryptographic construction is considered to matter, then it has to be explicitly modeled as part of the resource. To illustrate this point, we explain a few possible such explicit choices.

1. The term *information-theoretic security* is usually used when computation (at least by the adversary) is irrelevant. In such a case the converter set includes all systems, regardless of the computational complexity of implementing it.
2. If computing power is considered relevant, then one can consider converters that perform no computation by themselves but only connect systems and possibly input constants (for example a program). Any computational or randomness resource is modeled as a resource.
3. If, for some notion of efficiency, efficient computing power is considered irrelevant, then one can consider converters as efficiently implementable systems. Typically in cryptography, efficient is defined as some form of polynomial-time notion, which of course, and unfortunately, requires now the objects to be defined asymptotically in some way. A main reason for using polynomial-time is that this notion, if properly defined, is closed under composition.

We will take the second viewpoint, i.e., that any computation is performed in a resource. We will not formalize the class Σ of such trivial converters.

For example, if we so far considered a resource system $\alpha\mathbf{R}$ consisting of a converter α attached to system $\mathbf{R}$, then we can understand α as a resource, for example denoted $\tilde{\alpha}$, which is provided in parallel with $\mathbf{R}$. And we can consider a trivial converter π that simply connects $\tilde{\alpha}$ and $\mathbf{R}$, i.e., we have

$$\alpha\mathbf{R} = \pi[\mathbf{R}, \tilde{\alpha}].$$

However, now that this viewpoint is explained, we can continue to write $\alpha\mathbf{R}$ rather than $\pi[\mathbf{R}, \tilde{\alpha}]$.

5.3.4 ϵ -Relaxation

As already mentioned earlier, the ϵ -relaxation of a PDS-specification is defined as follows:

$$\mathcal{R}^\epsilon = \{\mathbf{R}' \mid \exists \mathbf{R} \in \mathcal{R} : \Delta(\mathbf{R}, \mathbf{R}') \leq \epsilon\}.$$

5.3.5 Reduction-based Relaxations

While ϵ -relaxations capture information-theoretic closeness, we also need to capture closeness in a “computational sense”. Let f be a performance function mapping distinguishers to $\mathbb{R}$, involving computational problems $p_1, \dots, p_n$, i.e., f is of the form

$$f = \lambda(\overline{p_1}, \dots, \overline{p_n})[\rho_1, \dots, \rho_n]$$

as discussed in Section 4.4.9. We define:

$$\mathcal{R}^f := \{\mathbf{R}' \mid \exists \mathbf{R} \in \mathcal{R} : \langle \mathbf{R} \mid \mathbf{R}' \rangle \leq f\}. \quad (5.5)$$

For computational closeness, as explained in Chapter 4, the concrete functions ρ_i and λ matter. The ρ_i should increase the complexity of a distinguisher as little as possible, and λ should be as small as possible.⁶

Exercise 5.1. Explain iterated PRG-expansion as a construction of a relaxed version of a (system outputting a) long uniform random string from (system outputting a) short uniform random string.

We do not discuss compatability properties of this relaxation type. This issue is a bit subtle.

5.3.6 *-Relaxations

We recall that an adversary entity Eve is only a hypothetical entity, taken into consideration in order to state guarantees to the honest parties no matter what Eve does. Eve simply stands for a universal quantifier ($\forall$) over strategies, not for an entity that actually participates. In other words, we want to capture that there is no guarantee for what happens at an adversary interface. An arbitrary converter could be applied at such an interface. This is modeled by the following relaxation type.

Definition 5.9. The **-relaxation* of a specification $\mathcal{R}$ for interface E is:

$$\mathcal{R}^{*E} := \{\alpha^E \mathbf{R} \mid \alpha \in \Sigma, \mathbf{R} \in \mathcal{R}\},$$

where Σ denotes the set of trivial converters mentioned above. If there is only a single adversary interface, we can drop the superscript E and simply write $\mathcal{R}^*$.

We point out that *-relaxation is compatible according to Definitions 5.6 and 5.7.

⁶In asymptotic terms, if the ρ_i are efficiency-preserving and λ is negligibility-preserving, then, if all problems $P_1, \dots, P_n$ are hard, then the distinction problem $\langle \mathbf{S} \mid \mathbf{T} \rangle$ is hard, i.e., $\mathbf{S}$ and $\mathbf{T}$ are computationally indistinguishable.

5.3.7 A new Look at the Simulation Paradigm

The notion of a simulator is central in cryptography. We recall Section 2.4.3 for an example of a simulator. A simulator is a converter that is used to argue that what an adversary Eve can achieve in a real-world setting she could also achieve in an ideal-world setting, by use of the simulator. This means that if one considers the ideal world adequate (e.g. secure), then so is the real world since in it, not more can go wrong than in the ideal world. A simulator is used in a thought experiment, it would never be actually implemented. Nevertheless, it must be as efficient as possible because it appears in reduction transformations.

More concretely, consider two systems $\mathbf{U}$ (real) and $\mathbf{V}$ (ideal) with an E -interface (accessible by Eve), and suppose there exists a simulator σ such that

$$\mathbf{U} \equiv \sigma^E \mathbf{V}.$$

This can be interpreted as follows. For any converter α that Eve might want to apply in the real world (i.e., she would form $\alpha^E \mathbf{U}$), there exists a converter β that Eve could apply in the ideal world (i.e., she would form $\beta^E \mathbf{V}$), such that $\alpha^E \mathbf{U} \equiv \beta^E \mathbf{V}$, i.e.,

$$\forall \alpha \exists \beta \quad \alpha^E \mathbf{U} \equiv \beta^E \mathbf{V}.$$

The simulator is a proof technique to prove this statement. Namely, the simulator provides the translation from α to β : $\alpha \mapsto \beta = \alpha\sigma$:

$$\beta^E \mathbf{V} \equiv (\alpha\sigma)^E \mathbf{V} = \alpha^E (\sigma^E \mathbf{V}) \equiv \alpha^E \mathbf{U}.$$

We can now take a new look at the simulation paradigm. As discussed in Section 5.3.3, a simulator σ can be seen as a resource $\tilde{\sigma}$ available to the adversary. We can write

$$\sigma^E \mathbf{V} = \sigma'^E [\mathbf{V}, \tilde{\sigma}_E] \in [\mathbf{V}, \tilde{\sigma}_E]^*.$$

with a trivial converter $\sigma' \in \Sigma$ appropriately connecting $\mathbf{V}$ and $\tilde{\sigma}$. Hence we have

$$\mathbf{U} \equiv \sigma^E \mathbf{V} \implies \mathbf{U}^* \subseteq [\mathbf{V}, \tilde{\sigma}_E]^*,$$

where we need that the converter set contains the identity element, i.e., $\mathbf{U} \in \mathbf{U}^*$. A classical cryptographic statement involving a simulator can hence be understood as a statement about *-relaxations.

5.3.8 Game-Relaxation

Definition 5.10. For a system $\mathbf{T}$, enhanced with an MBO to a game $\hat{\mathbf{T}}$, the *game-relaxation* of $\hat{\mathbf{T}}$, denoted $\hat{\mathbf{T}}^+$, is the set of PDS that behave as $\mathbf{T}$ as long as the MBO is 0 and behave arbitrarily once the MBO is 1.⁷

⁷It is not difficult to make this definition mathematically rigorous.

We state two lemmas without proofs. They state that one can interpret game equivalence and conditional equivalence statements as game relaxations.

Lemma 5.2. If $\hat{\mathbf{S}}$ is game-equivalent to a game $\hat{\mathbf{T}}$, then $\mathbf{S}$ is contained in the game-relaxation of $\hat{\mathbf{T}}$:

$$\hat{\mathbf{S}} \stackrel{g}{=} \hat{\mathbf{T}} \implies \mathbf{S} \subseteq \hat{\mathbf{T}}^+.$$

Lemma 5.3. Consider a game $\hat{\mathbf{S}}$ and a system $\mathbf{T}$. We have

$$\hat{\mathbf{S}} \models \mathbf{T} \implies \mathbf{S} \subseteq \hat{\mathbf{T}}^+.$$

where the MBO for $\mathbf{T}$ (to result in game $\hat{\mathbf{T}}$) is as described in the proof of Theorem 4.17, i.e., the inputs to $\mathbf{T}$ are also given to $\mathbf{b}\hat{\mathbf{S}}$ and the MBO (of $\hat{\mathbf{T}}$) is the MBO of $\hat{\mathbf{S}}$, i.e., the MBO is defined independently of $\mathbf{T}$.

The lemmas imply that one can view $\mathbf{S}$ as if it were $\mathbf{T}$, as long as the game is not won.

We point out that game-relaxation is compatible according to Definitions 5.6 and 5.7.

5.3.9 Substitution Relaxation

Substitution relaxations will not be discussed in this course.

5.4 Example: Authentication Amplification

We give an example of a construction statement based on game relaxation.

Suppose one can send a short message authentically (e.g. over the phone) but needs to send a very long message m authentically over the Internet. How is this possible? A very natural solution is to hash the message m , to send the hash value $h(m)$ over the authenticated channel, and to send m over the Internet.

Why is this secure, and which mathematical statement should we use to capture the security statement?

Informally, the statement one would typically make is: If h is collision-resistant, then the authentication system is secure. However, we already know that a single function can not be collision-resistant, so this statement does not make sense. This is why hash functions are often defined as asymptotic families, but we argue that introducing the artefact of asymptotics is the wrong approach for dealing with the stated problem.

A better statement is: There is an efficient reduction from the collision-finding problem for h to breaking the authentication scheme.

However, we can make an even more precise statement without need for reductions: If the constructed resource is not an authenticated channel, then a

hash collision explicitly occurs in it. This will be made precise in the lecture using game relaxation. The final statement we get is

Theorem 5.4. We have

$$\text{hsh} [\text{AUT}_k, \text{INS}_n, H_A, H_B] \text{chk} \subseteq [\text{AUT}_n, \gamma_{\text{coll}} H_E]^{*+},$$

where AUT_m is an m -bit authentic channel, INS_n is an n -bit insecure channel, H_i is a resource at interface i computing the hash function h once, hsh is the converter applied by A and chk is the converter applied by B , and γ_{coll} is the converter defining the collision game at the interface of H_E .

In the proof of the theorem one needs to provide a converter σ' (i.e., a “wiring”) at the E -interface and enhance the system $\text{hsh} [\text{AUT}_k, \text{INS}_n, H_A, H_B] \text{chk}$ by an MBO such that

$$\text{hsh} [\text{AUT}_k, \text{INS}_n, H_A, H_B] \text{chk} \stackrel{g}{=} \sigma'^E [\text{AUT}_n, \gamma_{\text{coll}} H_E].$$

5.5 Parameterized Resources and Constructions

Often, a resource $\mathbf{R}$ is parameterized by a parameter r , for example the allowed number of queries, and the parameter ϵ in an ϵ -relaxation or the function f in a reduction-based relaxation depend on r .

The parameterized restriction of an *a priori* unbounded resource $\mathbf{R}$ is modeled by filters ϕ_r (see Section 3.4.3).

Definition 5.11. A parameterized discrete single-interface resource with parameter set $\mathcal{Z}$ is a family $\{\phi_r \mathbf{R}\}_{r \in \mathcal{Z}}$ of finite resources, where $\mathbf{R}$ is a (generally infinite) resource system and $\{\phi_r\}_{r \in \mathcal{Z}}$ is a parameterized family of filter converters.

Example 5.2. Recall that $[r]$ is the special case of a filter restricting access to r queries. Hence $[r] \mathbf{R}_{n,n}$ is a uniform random function (URF) parameterized by the number r of queries it allows.

We will make parameterized statements of the following form:

$$\phi_r \mathbf{R} \xrightarrow{\psi_r \alpha} \psi_r \mathbf{S}^{f_r} \iff \psi_r \alpha \phi_r \mathbf{R} \subseteq \psi_r \mathbf{S}^{f_r}. \quad (5.6)$$

The constructing converter α does not depend on r , i.e., it works for any r , but α will be such that for each r ,

$$\psi_r \alpha \phi_r = \psi_r \alpha,$$

i.e., when accessed through ψ_r it makes only ϕ_r -restricted access to the attached system. For a specific r , (5.6) states that if one uses an ϕ_r -restricted version of $\mathbf{R}$ (i.e., $\phi_r \mathbf{R}$), then α constructs a resource that is f_r -close to an ψ_r -restricted version of $\mathbf{S}$ (i.e., $\psi_r \mathbf{S}$), as long as access (of a distinguisher) is ψ_r -restricted.

Chapter 6

Randomness Expansion

In this chapter we look at a number of constructions of single-interface resources, primarily the problem of expanding randomness. We omit the discussion of pseudo-randomness expansion.

6.1 The Randomness Expansion Problem

6.1.1 Introduction

Randomness is the most important resource in cryptography, especially secret randomness shared by some entities and not known to the other entities. An important paradigm in cryptography is to expand a given amount of randomness into a larger amount of randomness, for example to expand a secret key into several secret keys or to construct a uniform random function (URF) from a short uniform random value.

In a context where several parties need to perform the expansion operation (as in the key expansion problem mentioned above), the expansion must be *deterministic*. Deterministic computation can of course not really expand the given entropy, it can only expand the appearance of entropy. This can be achieved in two different ways:

- **Information-theoretic:** The expanded randomness is indistinguishable from true randomness even for computationally unbounded distinguishers, but there is a restriction on the distinguisher's access (e.g. the number of queries). See Section 6.2.
- **Computational:** The expanded randomness is only *pseudo-random*, i.e., computationally indistinguishable from true randomness.

Randomness expansion can in our framework be understood as a construc-

tion of the form (5.6), where $\mathbf{R}$ and $\mathbf{S}$ are randomness resources, where $\mathbf{S}$ provides *more* randomness than $\mathbf{R}$, and where α is a deterministic converter. In the information-theoretic case, f_r is a function of r , typically increasing with r .

6.1.2 Example: k -wise Independent Random Variables

An example of the first type are so-called k -wise independent random variables. Given k uniformly random values in $\{0, 1\}^m$ (for some m) one can construct 2^m values that appear perfectly uniformly random as long as one sees (i.e., queries) at most k of them. This corresponds to a function table $\{0, 1\}^m \rightarrow \{0, 1\}^m$ chosen according to some distribution. Such a construction can be achieved by letting the k random values be the coefficients $a_0, \dots, a_{k-1}$ of a polynomial

$$a(x) = a_{k-1}x^{k-1} + \dots + a_1x + a_0$$

over $GF(2^m)$ of degree $k - 1$, and considering the 2^m values $a(x)$ for all $x \in GF(2^m)$, i.e., considering the function

$$x \mapsto a(x).$$

The proof that this works is left as an exercise. Stated as a construction, we have, for any k :¹

$$\mathbf{U}_{km} \xrightarrow{[k]\alpha} [k]\mathbf{R}_{m,m},$$

where α is the converter that fetches the km -bit string at the inside (right interface) and continuously takes inputs at the outside (left interface) and for each such input x returns $a(x)$ at the outside.

6.2 Domain Extension of Uniform Random Functions

6.2.1 Motivation

In the above example, we have expanded k uniform random m -bit strings into 2^m m -bit strings which are uniformly random as long as one only accesses t values, where $t \leq k$. Intuitively, t can not be greater than k if information-theoretic security is required. Therefore, if such an expansion should work against an adversary who can ask a huge number of queries, then one must already assume to be given a large amount of randomness, for example a URF $\mathbf{R}_{m,n}$ which constitutes an exponential amount of randomness.

However, since the construction should be efficient, each query to the constructed randomness resource should involve only few of the given random values. Therefore the above polynomial-based construction does not work because each query involves the entire given randomness. We need an alternative approach.

¹Note that the resource $\mathbf{U}_{km}$ is considered to be read in one shot and therefore no access restriction must be modeled. Alternatively, one could model it as a URF $\{0, \dots, k-1\} \rightarrow \{0, 1\}^m$.

6.2.2 Variable Input-Length URFs

Cryptographic primitives, like a pseudo-random function (e.g. a block cipher), often have fixed input length. In applications (e.g. a message authentication code or an encryption function), one often needs a primitive with larger input length. Often the input length is not bounded, i.e., the input alphabet of a system is $\{0, 1\}^*$. Such a system is usually referred to as *variable input-length (VIL)*.

Definition 6.1. The term *variable input-length URF* (a VIL-URF) refers to the behavior of a $(\{0, 1\}^*, \{0, 1\}^n)$ -system which for each new input in $\{0, 1\}^*$ outputs a fresh uniformly random value in $\{0, 1\}^n$, and which replies consistently if inputs are repeated. This system (behavior) is denoted as $\mathbf{V}_n$.²

An important practical goal in cryptography is to construct a VIL-URF $\mathbf{V}_n$ from a URF $\mathbf{R}_{n,n}$ by a deterministic converter α :

$$[r]\mathbf{R}_{n,n} \xrightarrow{\psi_r, \alpha} (\psi_r \mathbf{V}_n)^{f_r}$$

for appropriate ψ_r and f_r . In this section we discuss two approaches to this problem. The restriction ψ_r will correspond roughly to a restriction on the total length of all inputs.

In practice, this construction is composed with a (computationally secure) construction of $\mathbf{R}_{n,n}$.

6.2.3 CBC-MAC as a Randomness Expander

A MAC scheme should allow to authenticate messages of arbitrary length. A completely random function $\mathbf{V}_n$ would be an ideal MAC for which the cheating probability is always 2^{-n} , independently of how many queries have been asked. Many MAC-functions are defined as a hashing process that makes use of a round function $\mathbf{F}$ which is assumed to behave like a URF $\mathbf{R}_{n,n}$. A typical implementation of $\mathbf{F}$ is by use of a block cipher, for example AES, for a uniform secret key. If $\mathbf{AES}$ denotes the converter that takes at the right (inside) interface as input a 128-bit string (a key $\mathbf{U}_{128}$) and at the left (outside) interface answers encryption queries, then we have $\mathbf{F} = \mathbf{AES} \mathbf{U}_{128}$.

A frequently used MAC is the so-called CBC-MAC (see Figure 2.9), which has also received considerable attention as an example for doing cryptographic security proofs. Let $\mathbf{CBC}$ denote the converter implementing the CBC-MAC, i.e., accepting messages and returning the MAC at the outside and making calls to the system in our case a URF $\mathbf{R}_{n,n}$ or $\mathbf{F}$ at the inside (right interface). For each (new) message, $\mathbf{CBC}$ applies a block former to the message and then digests the obtained block sequence block by block, each time invoking the system at its right interface. The initial value of the state is a fixed and known parameter.

²Note that because the input alphabet is infinite, $\mathbf{V}_n$ can not be described as a probabilistic discrete system, i.e., as a probability distribution over deterministic systems.

The security depends on how the block former within $\mathbf{CBC}$ is defined. If the CBC-MAC is used for messages of variable length, then the encoding of the message must be *prefix-free*, for example by prepending the message with an extra block encoding the message length. In other words, it must be guaranteed that for no two distinct messages m_1 and m_2 , the resulting block sequence (at the output of the block former) for m_1 is a prefix of the block sequence for m_2 .

For stating the following theorem we need to define a suitable restriction converter θ_r for $\mathbf{V}_n$, as follows. For a fixed block-former, for each message θ_r determines the number of blocks the block-former outputs (this is roughly the total length, in blocks, of the messages) and keeps track of the total number of such blocks resulting for all messages seen so far. When this number exceeds r , then θ_r stops replying to queries. Simply stated, we have³

$$\theta_r \hat{\mathbf{CBC}} = \theta_r \hat{\mathbf{CBC}}[r], \quad (6.1)$$

i.e., the filter $[r]$ is irrelevant because the restriction implied by θ_r guarantees that at most r queries are made to $\mathbf{R}_{n,n}$.

Theorem 6.1. For θ_r defined as above, if the block-former of the $\mathbf{CBC}$ -converter is prefix-free, we have (for any r)

$$[r]\mathbf{R}_{n,n} \xrightarrow{\theta_r \mathbf{CBC}} (\theta_r \mathbf{V}_n)^{\epsilon_r}$$

for $\epsilon_r = \frac{1}{2}r^2 2^{-n}$.

Note that r , and hence the bound on the distinguishing advantage, depends on the lengths of the messages. One can show that for fixed r , an optimal distinguisher actually chooses an (arbitrary) single message consisting of r blocks.

Proof. One can define an MBO A_i on the system $\mathbf{CBC} \mathbf{R}_{n,n}$ (more precisely, on $\hat{\mathbf{CBC}}$), resulting in the system $\hat{\mathbf{CBC}} \mathbf{R}_{n,n}$, such that

$$(\hat{\mathbf{CBC}} \mathbf{R}_{n,n}) \models \mathbf{V}_n. \quad (6.2)$$

We define A_i as follows: $A_i = 1$ if and only if, up to the evaluation of the i -th message (the i -th input to $\mathbf{CBC}$), a non-trivial collision has occurred at the input to $\mathbf{R}_{n,n}$. By non-trivial we mean that collisions do not count if they hold because two messages have the same prefix (in which case of course the same sequence of inputs to $\mathbf{R}_{n,n}$ is generated, until the two messages start to differ.) Since the encoding is prefix-free, $A_i = 0$ implies in particular that all the last-block inputs to $\mathbf{R}_{n,n}$ (i.e., those that result in an output of $\mathbf{CBC} \mathbf{R}_{n,n}$) are distinct, provided of course that the messages themselves are distinct. Conditioned on this event (i.e., on $A_i = 0$), all outputs are uniformly random, except of course

³Converters are probabilistic discrete systems and hence equivalence of converters is defined.

that for identical inputs (messages) the outputs are also identical. More precisely, $p_{Y^i|X^i, A_i=0}^{\text{CBC } \mathbf{R}_{n,n}}(y^i, x^i)$ is the uniform distribution over $\{0, 1\}^n$ for every list x^i of distinct inputs. This is equal to the distribution $p_{Y^i|X^i}^{\mathbf{V}_n}(y^i, x^i)$, i.e.,

$$p_{Y^i|X^i, A_i=0}^{\text{CBC } \mathbf{R}_{n,n}}(y^i, x^i) = p_{Y^i|X^i}^{\mathbf{V}_n}(y^i, x^i)$$

for all x^i and y^i . The extension of the proof to input lists x^i with repeating messages is obvious.

Hence we have proved (6.2), which is of course still true when both systems are restricted by θ_r :

$$(\theta_r \text{CBC } \mathbf{R}_{n,n}) \equiv \theta_r \mathbf{V}_n. \quad (6.3)$$

From (6.1) we have $\theta_r \text{CBC } \mathbf{R}_{n,n} \equiv \theta_r \text{CBC}[r] \mathbf{R}_{n,n}$, hence $(\theta_r \text{CBC}[r] \mathbf{R}_{n,n}) \equiv \theta_r \mathbf{V}_n$. Therefore we can apply Theorem 4.17 to obtain

$$\langle \theta_r \text{CBC}[r] \mathbf{R}_{n,n} | \theta_r \mathbf{V}_n \rangle \leq \Gamma(\mathbf{b} \theta_r \text{CBC } \mathbf{R}_{n,n}).$$

(Recall from Definition 4.17 that $\Gamma(\hat{\mathbf{S}})$ denotes the maximal probability of any winner of winning game $\hat{\mathbf{S}}$.)

It remains to analyze $\Gamma(\mathbf{b} \theta_r \text{CBC } \mathbf{R}_{n,n})$. To win the game $\mathbf{b} \theta_r \text{CBC } \mathbf{R}_{n,n}$ means to win the game $\theta_r \text{CBC } \mathbf{R}_{n,n}$ non-adaptively, i.e., it means to choose a fixed list of input messages to CBC (of lengths allowed by filter θ_r).

For the processing of any such list of messages, as long as the game is not won, the next value for which a collision should occur to win the game in the next step is uniformly random (as one can easily see). The probability of a collision in such a sequence is the same as the probability of having a collision in a list if r uniform n -bit strings, i.e., the upper bound of Lemma 4.18 can be applied and we have $\Gamma(\mathbf{b} \theta_r \text{CBC } \mathbf{R}_{n,n}) \leq \frac{1}{2} r^2 2^{-n}$. $\square$

6.2.4 A Construction Based on the Collision-Game for a Compression Function

We describe a construction of $\mathbf{V}_n$ from $\mathbf{R}_{n,n}$ based on the game of provoking a collision at the output of a system $\mathbf{S}$ that compresses input messages to n bits. Here $\mathbf{S}$ can be the evaluation of a hash function (see Section 6.2.5) or a system (involving a secret key) for which one can prove an information-theoretic bound on the probability of provoking a collision (see Section 6.2.6).

Recall from Definition 3.11 that for an $(\mathcal{X}, \mathcal{Y})$ -system $\mathbf{S}$ and a $(\mathcal{Y}, \mathcal{Z})$ -system $\mathbf{T}$, $\mathbf{S} \triangleright \mathbf{T}$ denotes the cascade of the two systems, which is an $(\mathcal{X}, \mathcal{Z})$ -system. We can write this cascade as

$$\text{casc}[\mathbf{S}, \mathbf{T}],$$

as discussed in Section 3.4.4, where $\text{casc} \in \Sigma$ is a converter.

For an $(\{0, 1\}^*, \{0, 1\}^n)$ -system $\mathbf{S}$, the system $\text{casc}[\mathbf{S}, \mathbf{R}_{m,n}]$ is a $(\{0, 1\}^*, \{0, 1\}^n)$ -system. If $\mathbf{S}$ is a random function, i.e., a random variable over (a finite subset of) functions $\{0, 1\}^* \rightarrow \{0, 1\}^n$, then also $\text{casc}[\mathbf{S}, \mathbf{R}_{m,n}]$ is a random function.

Intuitively, as long as no collision (equal output values) for distinct input values) occurs at the output of $\mathbf{S}$, the output of $\text{casc}[\mathbf{S}, \mathbf{R}_{m,n}]$ is uniformly random for every fresh input. Hence $\text{casc}[\mathbf{S}, \mathbf{R}_{m,n}]$ behaves as $\mathbf{V}_n$ as long as no collision occurs. The game of provoking an output collision in $\mathbf{S}$ is denoted as $\hat{\mathbf{S}}$.⁴

Theorem 6.2. For any $(\{0, 1\}^*, \{0, 1\}^n)$ -random function $\mathbf{S}$, and any bound r on the number of queries,

$$[r] \mathbf{S}, [r] \mathbf{R}_{m,n} \xrightarrow{[r] \text{casc}} ([r] \mathbf{V}_n)^{f_r}$$

for

$$f_r = \overline{\mathbf{b}[r] \hat{\mathbf{S}}} \circ [r] \mathbf{V}_n \leq \Gamma(\mathbf{b}[r] \hat{\mathbf{S}}),$$

where $\hat{\mathbf{S}}$ is the output-collision game for $\mathbf{S}$.

The theorem statement may look a bit complicated. It simply states that the advantage within r queries of a distinguisher $\mathbf{D}$ in distinguishing $\text{casc}[r] \mathbf{S}, [r] \mathbf{R}_{m,n}$ and $[r] \mathbf{V}_n$ is bounded by the probability that distinguisher $\mathbf{D}'$ non-adaptively wins the collision game for $\mathbf{S}$ within r queries, where $\mathbf{D}'$ is like $\mathbf{D}$, except that all inputs to $\mathbf{D}'$ are (replaced by) fresh uniformly random (but consistent) values. Moreover, $\mathbf{D}'$'s winning probability is of course upper bounded by the winning probability for $\hat{\mathbf{S}}$ of the best distinguisher (or winner).

Proof. We first observe that

$$[r] \text{casc}[\mathbf{S}, \mathbf{R}_{m,n}] \equiv \text{casc}[r] \mathbf{S}, [r] \mathbf{R}_{m,n}$$

i.e., it makes no difference whether we think of the query restriction $[r]$ as also being applied at the interface of $\mathbf{R}_{m,n}$. We have

$$\text{casc}[\hat{\mathbf{S}}, \mathbf{R}_{m,n}] \equiv \mathbf{V}_n$$

because, conditioned on $A_i = 0$, all outputs are uniformly random, except that for identical inputs (which does not count for a collision) the outputs are also identical. This is of course still true when the systems are restricted by $[r]$, i.e., $[r] \text{casc}[\hat{\mathbf{S}}, \mathbf{R}_{m,n}] \equiv [r] \mathbf{V}_n$. and hence

$$[r] \text{casc}[\hat{\mathbf{S}}, \mathbf{R}_{m,n}] \equiv [r] \mathbf{V}_n.$$

Therefore we can apply Theorem 4.17 to obtain

$$\langle [r] \text{casc}[r] \mathbf{S}, [r] \mathbf{R}_{m,n} | [r] \mathbf{V}_n \rangle \leq \overline{\mathbf{b}[r] \hat{\mathbf{S}}} \circ [r] \mathbf{V}_n \leq \Gamma(\mathbf{b}[r] \hat{\mathbf{S}}).$$

$\square$

⁴i.e., $\hat{\mathbf{S}}$ is the system $\mathbf{S}$ enhanced to a game by the MBO A_i , where $A_i = 1$ if and only if a collision occurs at the output of $\mathbf{S}$ (for distinct inputs), up to the i -th input.

6.2.5 Using a Collision-Resistant Hash Function

If $\mathbf{S}$ implements a hash function h , then $\hat{\mathbf{S}}$ is the collision-finding game for h . Theorem 6.2 states that any distinguisher $\mathbf{D}$ for systems $[r]\text{casc}[\mathbf{S}, \mathbf{R}_{m,n}]$ and $[r]\mathbf{V}_n$ can be simply transformed into a (non-adaptive) collision finder $\mathbf{D}'$ with the same performance.

This is an example of a reduction statement for which there exists no corresponding hardness statement. One would be tempted to make the following statement: If h is collision-resistant (i.e., no efficient algorithm can find a collision for h with non-negligible probability), then $\text{casc}[\mathbf{S}, \mathbf{R}_{m,n}]$ and $\mathbf{V}_n$ are computationally indistinguishable (i.e., no efficient distinguisher can distinguish $\text{casc}[\mathbf{S}, \mathbf{R}_{m,n}]$ and $\mathbf{V}_n$ with non-negligible advantage).⁵ However, this argument is flawed because one can not even define collision-resistance for a single function h meaningfully. For every hash function h , there exists an efficient collision finder, namely one that simply outputs a collision for h (which exists because h is shrinking).

6.2.6 δ -Almost Universal Hash Functions

Our goal is to make use of Theorem 6.2 to obtain an information-theoretically secure construction of $\mathbf{V}_n$. For this purpose we need to instantiate $\mathbf{S}$ as a system involving a secret key that is provably collision-resistant, in an information-theoretic sense.

Definition 6.2. Let δ be a function $\mathbb{N} \rightarrow \mathbb{R}^+$. A function $H : \mathcal{K} \times \mathcal{Y} \rightarrow \mathcal{Z}$ for $\mathcal{Y} \subseteq \{0, 1\}^*$ is called a δ -almost universal hash function (δ -AUH for short)⁶ if, for any distinct $y, y' \in \mathcal{Y}$,

$$\Pr(H_K(y) = H_K(y')) \leq \delta(\max(|y|, |y'|))$$

for uniformly selected key $K \in \mathcal{K}$.⁷

Note that we write the first argument of the function H as a subscript to indicate its special role.

We now describe a concrete construction of a δ -AUH, for $\mathcal{K} = \mathcal{Z} = \{0, 1\}^m$ and $\mathcal{Y} = \{0, 1\}^*$. An input $y \in \{0, 1\}^*$ is cut into m -bit blocks, then a 1-bit is appended, and then between 0 and $k - 1$ 0-bits are appended to complete the k -bit block.⁸ The resulting block sequence is

$$y' = [y_0, y_1, \dots, y_{t-1}],$$

⁵Note that one does not need to restrict the number of queries by filters in this statement because a distinguisher can only make at most one query per computation step and the number of steps of a feasible distinguisher is bounded.

⁶This definition is more general than the definition in the literature where δ does not depend on the length of the input.

⁷Here $|y|$ denotes the length of y .

⁸Other paddings are also possible.

where t is the length of y' in blocks, i.e., $t = \lceil (|y| + 1)/m \rceil$.

We consider a fixed representation of the elements of the finite field $GF(2^m)$ (with addition and multiplication operations) as m -bit strings. An input $y \in \{0, 1\}^*$ can be interpreted as a polynomial $p_y(x)$ of degree $t - 1$ over $GF(2^m)$, as follows: For y , let $y' = [y_0, y_1, \dots, y_{t-1}]$, as above, and define $p_y(x)$ as

$$p_y(x) := y_{t-1}x^{t-1} + y_{t-2}x^{t-2} + \dots + y_1x + y_0.$$

Consider the following function $H : \{0, 1\}^m \times \{0, 1\}^* \rightarrow \{0, 1\}^m$:

$$H_K(y) := p_y(K).$$

Lemma 6.3. H as described is a δ -AUH for $\delta(\ell) = 2^{-m}\ell/m$.

Proof. Any two polynomials of degree at most d over a field can agree for at most d values because the difference of the two polynomials has also degree at most d and hence has at most d roots. (Note that a root of the difference polynomial corresponds to a point where the two polynomials agree.) Thus the probability, when choosing K uniformly at random, that a point is selected at which the two polynomials agree, is at most the degree divided by the field size, 2^m . For $\ell := |y|$ the degree is $\lceil \ell/m \rceil - 1$ which is at most ℓ/m . $\square$

6.2.7 URF Domain Extension Using a δ -AUH

We now describe a construction of $\mathbf{V}_n$ from $\mathbf{R}_{m,n}$ that is substantially more efficient than the CBC-MAC construction and could be used as a replacement.

Let H be a δ -AUH with k -bit key and m -bit output, and let $\mathbf{h}$ be the converter that fetches a k -bit key value K at the inside interface and at the outside interface provides access to H_K . Hence $\mathbf{hU}_k$ corresponds to a system containing H equipped with a uniform key. Let $\hat{\mathbf{h}}$ be the collision game for $\mathbf{h}$. For any fixed set of r inputs, each of length at most ℓ , to $\mathbf{hU}_k$, it follows from Definition 6.2 and the union bound (as in Lemma 4.18) that the probability of a collision at the output is at most $\frac{1}{2}r^2\delta(\ell)$. Stated differently, we have

$$\Gamma(\mathbf{b} \tau_{r,\ell} \hat{\mathbf{h}} \mathbf{U}_k) \leq \frac{1}{2}r^2\delta(\ell),$$

where $\tau_{r,\ell}$ denotes the filter that restricts access to at most r queries, each of length at most ℓ .

If we define the converter α by $\alpha[\mathbf{S}, \mathbf{T}] = \text{casc}[\mathbf{hS}, \mathbf{T}]$ and hence have

$$\alpha[\mathbf{U}_k, [r]\mathbf{R}_{m,n}] = \text{casc}[\mathbf{hU}_k, [r]\mathbf{R}_{m,n}],$$

then we obtain the following corollary to Theorem 6.2:

Corollary 6.4. For any $r \geq 1$ and $\ell \geq 1$,

$$[\mathbf{U}_k, [r]\mathbf{R}_{m,n}] \xrightarrow{\tau_{r,\ell}\epsilon} (\tau_{r,\ell} \mathbf{V}_n)^{\epsilon_{r,\ell}}$$

for $\epsilon_{r,\ell} = \frac{1}{2}r^2\delta(\ell)$.

To obtain a practical instantiation, one can use the polynomial-based δ -AUH of the previous section and make use of Lemma 6.3.

Exercise 6.1. A possible drawback of this construction compared to the CBC-MAC is that it requires a secret key.⁹ An elegant solution for this problem, which the reader can explore as an exercise, is to “take” the k -bit key from $\mathbf{R}_{m,n}$, for example by reading out a few values of $\mathbf{R}_{m,n}$ (for fixed arguments).

⁹If $\mathbf{R}_{m,n}$ is replaced by a pseudo-random function, which requires a secret key of size k' , then this means that the total key size is $k + k'$.

Chapter 7

Constructing Shared Secret Keys

7.1 The Key Agreement Problem

Shared secret keys are a crucial resource in cryptography. Secret keys are not of primary interest in their own right, i.e., as the final “product” of a security protocol, but only as a resource used in cryptographic constructions. In a sense, a secret key is a form of security (e.g. secrecy and authenticity) that can be conserved and stored for future use. For example, if a secure channel is available at a certain time t one can use it to transport a secret key which can then later be used to construct a secure channel at a time $t' > t$ when only an insecure channel is available. In a sense this, is comparable to the conservation of solar energy in the form of ethanol by the use of biological systems (agriculture).

One is interested in constructing shared secret keys from weaker resources, for example from authenticated communication between A and B .

Definition 7.1. The construction of a shared secret key from authenticated communication by a protocol $\pi = (\pi_A, \pi_B)$, for simulator¹ σ and a suitable performance function f is called *key agreement*:

$$[\bullet \longrightarrow , \longleftarrow \bullet] \xrightarrow{\pi} (\sigma^E \bullet \bullet \bullet)^f.$$

For simplicity, we generally do not explicitly consider the bit-length of messages sent over the channels or of the generated secret key.

As discussed earlier, the Diffie-Hellman protocol $\text{DH} = (\text{dh}_A, \text{dh}_B)$ achieves this construction, where σ outputs two random group elements, where $f =$

¹Recall that here we understand σ as a converter with some computing power. We could also think of the computational aspects of σ as being a resource $\tilde{\sigma}_E$ available in the constructed resource specification. In this chapter we take the former viewpoint

$\overline{\text{DDH}}_\rho$ for a simple function ρ , and where DDH denotes the DDH problem. The proof of the following theorem was an exercise earlier in the course.

Theorem 7.1. $[\bullet \longrightarrow , \longleftarrow \bullet] \xrightarrow{\text{DH}} (\sigma^E \bullet \bullet \bullet)^f$.

7.2 Information-Theoretic Key-Agreement

A general question one asks in cryptography is: Which constructions can be achieved with information-theoretic (rather than only computational) security? Information-theoretic security means that the constructed resource is an ϵ -relaxation (for a small ϵ of a target resource specification, and that we do not care if Eve gets, as part of the constructed resource specification, an arbitrary resource in Φ (including an arbitrarily powerful computer). This is captured by including on the right side of a construction the resource specification Φ_E .

In Section 7.2.1 we prove that information-theoretically secure key agreement is impossible, which is the reason for resorting to computational security. However, we will mention in Section 7.2.2 that when the model is changed slightly and an additional resource is assumed, then information-theoretically secure key agreement is actually possible.

7.2.1 Impossibility

The following theorem follows from Theorem 7.3 below.

Theorem 7.2. *There exists no protocol $\pi = (\pi_A, \pi_B)$ and simulator σ such that*

$$[\bullet \longrightarrow , \longleftarrow \bullet] \xrightarrow{\pi} (\sigma^E \bullet \bullet \bullet)^\epsilon.$$

for $\epsilon \leq \frac{1}{4}$, even if arbitrarily many rounds of authenticated communication are allowed.

This theorem states, in other words, that information-theoretically secure public-key cryptography is impossible.

To be even more general, we also assume that A , B , and E hold initially some random variables X , Y , and Z , respectively, distributed according to some joint distribution P_{XYZ} .² If we denote such a resource providing X , Y , and Z as $[P_{XYZ}]$, then we consider the construction³

$$[\bullet \longrightarrow , \longleftarrow \bullet , [P_{XYZ}]] \xrightarrow{\pi} (\sigma^E \bullet \bullet \bullet)^\epsilon. \quad (7.1)$$

Note that if P_{XYZ} were defined directly as a secret key given to A and B (i.e., $X = Y$ is a uniform random string and Z is independent of X), then a simple

²The case where they hold no additional random variables is the trivial special case where X , Y , and Z are constants (and can be ignored).

³This statement can typically be strengthened by replacing Φ_E by a concrete simple simulator resource $\tilde{\sigma}_E$.

π would trivially achieve the construction. Of course, one is interested in non-trivial cases of this generalized key agreement problem.

We refer the reader to the Appendix where the basics of information theory are explained. It is sufficient to use the basic information-theoretic inequalities explained there; no further understanding of information theory is required.

A key-agreement protocol, where A and B initially hold X and Y , respectively, consists of a finite sequence of message exchanges between A and B , where each message depends on the sender's complete knowledge at that time. Let C denote the list of random variables corresponding to the entire communication between A and B . As the last step, A generates a key K_A from (X, C) by a (possibly probabilistic) function, and B generates a key K_B from (Y, C) . Of course, one wants that $K_A = K_B$ with very high probability.

The following theorem states that if $H(K_A)$ substantially exceeds $I(X; Y|Z)$, then it is unavoidable that either the amount of information the adversary learns about K_A (i.e., $I(K_A; ZC)$) is substantial, or $H(K_B|K_A)$ is substantial, which implies that $K_A \neq K_B$ with substantial probability.⁴

Theorem 7.3. *For any key-agreement protocol,*

$$I(K_A; ZC) + H(K_A|K_B) \geq H(K_A) - I(X; Y|Z).$$

Proof. Let X' , Y' , and Z' be the random variables held by A , B and E , respectively, at some stage of the protocol. We prove that for any next stage of the protocol,

$$I(X''; Y''|Z'') \leq I(X'; Y'|Z'),$$

where X'' , Y'' , and Z'' are the random variables held by the parties after the step is performed. In other words, we prove that the quantity $I(X'; Y'|Z')$ can not increase under any possible operations that can occur in a protocol. There are four such operations to consider:

- **Generation of local randomness** by A (and, analogously, by B). Let R be a random variable generated by A independently of all other random variables. Then $X'' = (X', R)$, $Y'' = Y'$, and $Z'' = Z'$, and hence

$$I(X''; Y''|Z'') = I(X'R; Y'|Z') = I(X'; Y'|Z')$$

This follows directly from the definition of $I(X'R; Y'|Z') = H(X'RZ') + H(Y'Z') - H(X'RY'Z') - H(Z')$ since the terms $H(X'Z')$ and $H(X'Y'Z')$ are both increased by exactly $H(R)$ when R is appended to X' .

⁴We refer to the Appendix for a brief summary of the basics of information theory. Here we only need a few very simple properties of the basic information-theoretic quantities, namely:

Fact 7.1. For any random variables X , Y , and Z , we have

1. $H(XY) \geq H(X)$ with equality if and only if Y is determined by X ,
2. $I(X; Y) \geq 0$,
3. $I(X; Y|Z) \geq 0$.

- **Local computation** by A (and, analogously, by B). Let U be computed from X' by some (deterministic) function. Then $H(X'U) = H(X')$ (or, equivalently, $H(U|X') = 0$). Therefore

$$I(X''; Y''|Z'') = I(X'U; Y'|Z') = I(X'; Y'|Z')$$

since all involved entropies remain unchanged.

- **Sending a message** from A to B (or, analogously, from B to A). Let V be a random variable seen by A (e.g. previously computed) which A communicates to B and hence also to E . Note that V is part of X' , i.e., $H(V|X' \dots) = 0$ and thus for any occurrence of $X'V$ we can equivalently write X' instead (used in the second step below). The following argument is best illustrated by a simple entropy diagram. We have⁵

$$\begin{aligned} I(X''; Y''|Z'') &= I(X'; Y'V|Z'V) \\ &= \underbrace{H(X'Z'V)}_{=H(X'Z')} + H(Y'Z'V) - \underbrace{H(X'Y'Z'V)}_{=H(X'Y'Z')} - H(Z'V) \\ &= H(X'Z') + H(Y'Z'V) - H(X'Y'Z') - H(Z'V) \\ &\leq H(X'Z') + H(Y'Z'V) - H(X'Y'Z') - H(Z'V) \\ &\quad + \underbrace{I(V; Y'|Z')}_{=H(VZ') + H(Y'Z') - H(VY'Z') - H(Z')} \\ &= H(X'Z') + H(Y'Z') - H(X'Y'Z') - H(Z') \\ &= I(X'; Y'|Z'). \end{aligned}$$

The inequality above follows from the fact that $I(V; Y'|Z') \geq 0$, according to Fact 7.1.

- **Deletion of information** by A (and, analogously, by B). Deleting a value by A (e.g. X' after having computed U) can not increase the conditional mutual information. The proof is left as an exercise and is similar to the proof of the previous statement.

It remains to prove the inequality of the theorem from the fact that $I(X'; Y'|Z')$ cannot increase. At the end of a protocol, A and B hold the keys K_A and K_B and have discarded the other information, i.e., we have $X'' = K_A$ and $Y'' = K_B$ and $Z'' = (Z, C)$. The proof is completed upon noting that

$$\begin{aligned} I(X; Y|Z) &\geq I(X''; Y''|Z'') \\ &= I(K_A; K_B|ZC) \\ &= H(K_A ZC) + H(K_B ZC) - H(K_A K_B ZC) - H(ZC) \\ &= H(K_A) - I(K_A; ZC) - H(K_A|K_B ZC) \\ &\geq H(K_A) - I(K_A; ZC) - H(K_A|K_B), \end{aligned}$$

⁵Recall that the entropy of several random variables does not depend on the order in which they are listed.

where we have used

$$H(K_A) - I(K_A; ZC) = H(K_A ZC) - H(ZC)$$

as well as

$$H(K_A K_B ZC) - H(K_B ZC) = H(K_A|K_B ZC) \leq H(K_A|K_B).$$

□

We state a simple corollary capturing the perfect case.

Corollary 7.4. *For any protocol for which $K_A = K_B (= K)$ with probability 1 and $I(K; ZC) = 0$, we have*

$$H(K) \leq \min(I(X; Y), I(X; Y|Z)).$$

Proof. $K_A = K_B$ implies $H(K_A|K_B) = 0$. Since $I(K_A; ZC) = I(K; ZC) = 0$, Theorem 7.3 implies that $I(X; Y|Z)$ is an upper bound on $H(K_A) = H(K)$. In a thought experiment where Eve computes $Z' = f(Z)$ for some function f and deletes Z (replacing it by Z'), the quantity $I(X; Y|Z')$ is also an upper bound on $H(K)$ in the original setting (with Z) because Eve could indeed do this. As a special case, Eve could delete Z entirely. This shows that $I(X; Y)$ is also an upper bound on $H(K)$, completing the proof. □

Proof of Theorem 7.2. We only give an impossibility proof for the perfect case, i.e., for $\epsilon = 0$. The proof for small ϵ is similar, but technically a bit more involved.

The security condition for simulator σ is

$$\pi_1^A \pi_2^B [\bullet \longrightarrow , \longleftarrow \bullet] \equiv \sigma^E \bullet \bullet \bullet . \quad (7.2)$$

The system

$$\pi_1^A \pi_2^B [\bullet \longrightarrow , \longleftarrow \bullet]$$

outputs the triple (K_A, K_B, C) of random variables. For any simulator σ , the system

$$\sigma^E \bullet \bullet \bullet$$

also outputs a triple (K'_A, K'_B, C') , where by definition of $\bullet \bullet \bullet$ we have $K'_A = K'_B (= K')$ and K' is uniform, and where C' and K' are independent (because the resource $\bullet \bullet \bullet$ provides no information about the key K' at the E -interface). Equation (7.2) means that these triples (K_A, K_B, C) and (K'_A, K'_B, C') must have the identical joint distribution. In particular, we must have $K_A = K_B (= K)$, and C and K must be independent, i.e., $I(K; C) = 0$.

For the special case where no additional information (X, Y, Z) is considered, Corollary 7.4 implies $H(K) = 0$ if $I(K; C) = 0$ is required. In other words, the key has no entropy and, in particular, cannot be a uniform key for any alphabet of size two or more. □

7.2.2 Possibility

Section 7.2.2 is *not* relevant for the exam.

In many realistic settings, the parties A , B , and E can access some information in their environment, which can be interpreted as the initial random variables X , Y , and Z discussed above. There are reasonable settings in which $I(X; Y|Z)$ is strictly positive and secret-key agreement by public discussion is indeed possible. One such setting is the so-called satellite model.

The satellite model: One assumes that a satellite broadcasts random bits that are received by A , B , and E over independent noisy channels, with bit error probabilities ϵ_A , ϵ_B , and ϵ_E , respectively. The proof of the following theorem is beyond the scope of this course.⁶

Theorem 7.5. *Secret key-agreement by public discussion is possible in the satellite model unless it is impossible by Corollary 7.4, i.e., unless either $\epsilon_E = 0$ or $\epsilon_A = 0.5$ or $\epsilon_B = 0.5$.*

In other words, if we denote the satellite setting by SAT, then the construction

$$[\bullet \longrightarrow, \longleftarrow \bullet, \text{SAT}] \xrightarrow{\pi} (\sigma^E \bullet \longleftarrow \bullet)^\epsilon.$$

is possible for arbitrarily small ϵ and for an adequate π and a simple σ .

We also briefly discuss two other approaches to information-theoretically secure key agreement:

- **Bounded-storage model.** In this model one assumes that Alice and Bob share a short secret key initially and that they want to expand it to a very long key, secure against an Eve with unbounded computing time, but with bounded memory. A very long bit-string (called randomizer) R is transmitted from Alice to Bob such that Eve's memory does not allow to store the entire value of R (but, say, only 90%). Alice and Bob extract a (long) secret key from the randomizer, using the initially shared secret key. Security is guaranteed even if Eve can store arbitrary information about the randomizer string, i.e., she can apply an arbitrary function mapping R to her memory size. The model has received considerable attention, but its practicality appears limited.
- **Quantum cryptography.** Alice and Bob exchange single photons with certain polarizations and then communicate over an authenticated channel to agree on a secret key. Assuming the laws of quantum physics are correct, one can prove that Eve, who has full control over the quantum channel between Alice and Bob, has a choice only between disrupting the process such that Alice and Bob notice this (and can abort), or receiving only a negligible amount of information about the secret key shared by Alice and Bob.

⁶see U. Maurer, Secret key agreement by public discussion, *IEEE Transactions on Information Theory*, vol. 39, no. 3, pp. 733–742, 1993.

7.2.3 Privacy Amplification

Privacy amplification refers to a setting, often arising in information-theoretic cryptography (including quantum cryptography), where some parties (usually Alice and Bob) share the value of a random variable W and some party (usually Eve) knows a random variable V correlated with W . Such a setting is typically reached after some protocol steps between Alice and Bob, including error correction which guarantees that the values they hold are equal, namely W .

The goal of privacy amplification is for Alice and Bob to generate an essentially uniformly random string K about which Eve has essentially no information, and which can serve as a key. They are allowed to communicate publicly (though authentically). Typically, privacy amplification is achieved by one party (say Alice) choosing a random value R , sending R to Bob, and then both Alice and Bob applying a deterministic publicly-known function φ to (W, R) :

$$K = \varphi(W, R).$$

An equivalent formulation, which is usually used, is that Alice selects a function G at random from a class of compression functions, and that Alice and Bob both compute $K = G(W)$.

It is assumed (e.g. known from the analysis of the previous protocol steps) that there is an upper bound on the uncertainty of W if one knows V . More precisely, the conditional probability distribution $P_{W|V}$ satisfies a certain condition. Such a condition can be stated in different ways, for example that a condition on $P_{W|V=v}$ holds with high probability (over values v), or that a condition holds on average (over values v). Privacy amplification must work even if Alice and Bob know nothing about $P_{W|V}$, except that it satisfies the assumed condition.

Example 7.1. Consider a setting where Alice and Bob know a uniformly random n -bit string W and where Eve is allowed, for a fixed $m < n$, to (secretly) choose an arbitrary function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ and to obtain the random variable $V = f(W)$. A numerical example is $n = 1000$ and $m = 800$, i.e., Eve can obtain arbitrary 80% of the information about W . Privacy amplification allows Alice and Bob to obtain a secret key of close to 200 bits (say 180 bits), despite the fact that which information Eve chose is unknown to Alice and Bob. In other words, they can essentially harvest all the remaining uncertainty of Eve about W , without knowing what this uncertainty is. Of course, Eve must choose f before execution of the privacy amplification protocol.

For simplicity we consider the following simplified description of the setting where Eve's knowledge $V = v$ is not made explicit. Instead, the probability distribution P_W can be understood as taking Eve's information already into account (and can be seen as the conditional distribution $P_{W|V=v}$). The two views are equivalent for the following treatment.

For a random variable X over a finite alphabet $\mathcal{X}$ we define the following

three quantities: The distance from uniform, denoted $d(X)$ and defined by

$$d(X) := \frac{1}{2} \sum_{x \in \mathcal{X}} \left| P_X(x) - \frac{1}{m} \right|,$$

the maximal probability, denoted $p_{\max}(X)$ and defined by

$$p_{\max}(X) := \max_{x \in \mathcal{X}} P_X(x),$$

and the *collision probability*, denoted $p_{\text{coll}}(X)$ and defined as the probability that two independent random variables X_1 and X_2 distributed according to P_X are equal, i.e., as $\Pr(X_1 = X_2)$, which is

$$p_{\text{coll}}(X) := \sum_{x \in \mathcal{X}} (P_X(x))^2.$$

One often defines $-\log_2 p_{\max}(X)$ as the so-called *min-entropy* of X , denoted $H_{\infty}(X)$, and $-\log_2 p_{\text{coll}}(X)$ as the so-called *Rényi entropy* of X , denoted $R(X)$. In privacy amplification statements one often considers that Eve's knowledge about W is bounded in terms of min-entropy (i.e. in terms of the maximal probability of W from Eve's viewpoint). The following lemma shows that an upper bound on $p_{\max}(X)$ is also an upper bound on $p_{\text{coll}}(X)$.

Lemma 7.6. $1/|\mathcal{X}| \leq p_{\text{coll}}(X) \leq p_{\max}(X)$.

Proof. Left as an exercise. $\square$

The following lemma states a relation between $d(X)$ and $p_{\text{coll}}(X)$ and will be needed in the analysis of the privacy amplification theorem stated below.

Lemma 7.7. $d(X) \leq \frac{1}{2} \sqrt{|\mathcal{X}| \cdot p_{\text{coll}}(X) - 1}$.

Proof. Left as an exercise. $\square$

We define a special case of the concept of δ -AUH functions introduced in Section 6.2.6.

Definition 7.2. A class $\mathcal{G}$ of functions $\mathcal{A} \rightarrow \mathcal{B}$ is *2-universal* ("universal" for short) if, for any distinct x_1 and x_2 in $\mathcal{A}$, the probability that $g(x_1) = g(x_2)$ is at most $1/|\mathcal{B}|$ when g is chosen at random from $\mathcal{G}$ according to the uniform distribution.⁷

When $\mathcal{A} = \mathcal{B}$, the class consisting only of the identity function is trivially a universal class, although there are others. In all cases, the class of all functions from $\mathcal{A}$ to $\mathcal{B}$ is universal, but it is not useful because there are too many functions in that class. A more useful universal class is that of all *linear* functions from $\{0, 1\}^n$ to $\{0, 1\}^r$. These functions can be described by $r \times n$ matrices M over $GF(2)$, i.e. by rn bits. Another universal class, requiring only n bits to specify a function, is given in the following lemma.

⁷This definition was stated in J. L. Carter and M. N. Wegman, "Universal classes of hash functions", *Journal of Computer and System Sciences*, Vol. 18, 1979, pp. 143–154.

Lemma 7.8. Let a be an element of $GF(2^n)$ and also interpret x as an element of $GF(2^n)$. Consider the function $\{0, 1\}^n \rightarrow \{0, 1\}^r$ assigning to an argument x the first r bits of the element ax of $GF(2^n)$. The class of all such functions for $a \in GF(2^n)$ is a universal class of functions for $1 \leq r \leq n$.

Proof. For any two $x_1, x_2 \in GF(2^n)$ with $x_1 \neq x_2$, and for any $a \in GF(2^n)$ we have $[ax_1]_r = [ax_2]_r$ if and only if $[a(x_1 - x_2)]_r = 0^r$, where $[y]_r$ for $y \in GF(2^n)$ denotes the first r bits of y . For each of the 2^{n-r} $y \in \{0, 1\}^n$ with $[y]_r = 0^r$, there is exactly one a with $a(x_1 - x_2) = y$, namely $a = y/(x_1 - x_2)$. (Here the field property is needed.) Hence, for any $x_1 \neq x_2$ there are exactly 2^{n-r} values a for which $[ax_1]_r = [ax_2]_r$. The probability of choosing such an a is $2^{n-r}/2^n = 2^{-r}$. $\square$

The following theorem implies that if one hashes W down to

$$r \ll -\log_2 p_{\text{coll}}(W)$$

bits, then not only $G(W)$, but even the pair $(G, G(W))$ is essentially uniformly distributed, where the distance from uniform decreases exponentially in $-\log_2 p_{\text{coll}}(W) - r$. This means that $G(W)$ is essentially uniform from Eve's viewpoint even if G is known to Eve. Therefore the choice of G needs to be communicated only authentically, but not secretly. This is known as *privacy amplification*.

Theorem 7.9. Let G be the random variable corresponding to the uniform choice of a function from a universal class $\mathcal{G}$. Then

$$d((G, G(W))) \leq \frac{1}{2} \sqrt{2^r p_{\text{coll}}(W)}. \quad (7.3)$$

Proof. Note that the cardinality of the alphabet of $(G, G(W))$ is $|\mathcal{G}| \cdot 2^r$. Using Lemma 7.7 we hence have

$$d((G, G(W))) \leq \frac{1}{2} \sqrt{|\mathcal{G}| \cdot 2^r \cdot p_{\text{coll}}((G, G(W))) - 1}. \quad (7.4)$$

To compute an upper bound on $p_{\text{coll}}((G, G(W)))$, we note that a collision between two values $(G_1, G_1(W_1))$ and $(G_2, G_2(W_2))$ occurs if and only if $G_1 = G_2$ (which happens with probability $1/|\mathcal{G}|$) and also $G_1(W_1) = G_2(W_2)$. Conditioned on $G_1 = G_2 = G$, the probability that $G_1(W_1) = G_2(W_2)$, i.e., that $G(W_1) = G(W_2)$, is⁸

$$\Pr(G(W_1) = G(W_2))$$

⁸The random experiment considered is that G is chosen uniformly and, independently, W_1 and W_2 are chosen (independently) according to P_W .

$$\begin{aligned}
&= \Pr(W_1 = W_2) + \underbrace{\Pr(G(W_1) = G(W_2) | W_1 \neq W_2)}_{= 2^{-r}} \cdot \Pr(W_1 \neq W_2) \\
&\leq p_{\text{coll}}(W) + 2^{-r}.
\end{aligned}$$

Hence

$$p_{\text{coll}}((G, G(W))) \leq \frac{1}{|\mathcal{G}|} (p_{\text{coll}}(W) + 2^{-r}).$$

Combining this with (7.4), we obtain

$$d((G, G(W))) \leq \frac{1}{2} \sqrt{|\mathcal{G}| \cdot 2^r \cdot \frac{1}{|\mathcal{G}|} (p_{\text{coll}}(W) + 2^{-r}) - 1} = \frac{1}{2} \sqrt{2^r p_{\text{coll}}(W)},$$

which concludes the proof. $\square$

Exercise 7.1. Prove that privacy amplification works in the setting of Example 7.1. More precisely, prove a bound on the average distance from uniform of the key generated by Alice and Bob, given Eve's information.

7.3 Key-Agreement in the Random Oracle Model

7.3.1 Definition of a Random Oracle

The so-called random oracle model is a useful paradigm for devising and analyzing cryptographic protocols.

Definition 7.3. The term *random oracle model (ROM)* refers to any cryptographic scenario in which one assumes the availability of an idealized resource, called a *public random oracle* (or simply *random function*), a uniform random function $\{0, 1\}^* \rightarrow \{0, 1\}^k$ (for some k) accessible⁹ to all the parties. Such a resource will be denoted as $\mathbf{PO}_k$.

The heuristic idea, proposed by Bellare and Rogaway in 1993,¹⁰ is to assume that a cryptographic hash function behaves like a random function. Since it can be computed by anyone locally, such a hash function can be used as a public random oracle.

⁹Accessible means that each party can evaluate the oracle function for any arguments of its choice, without leaking information about the arguments.

¹⁰M. Bellare and P. Rogaway, Random oracles are practical: a paradigm for designing efficient protocols, in V. Ashby, editor, *1st ACM Conference on Computer and Communications Security*, pages 62–73. ACM Press, 1993.

7.3.2 Key-Agreement Using A TOWP in the ROM

Recall the notion of a trapdoor one-way permutation (TOWP, see Definition 2.18) and RSA as a specific instantiation.

It appears natural to try to achieve the construction

$$[\bullet \longrightarrow, \longleftarrow \bullet] \xrightarrow{\pi} (\sigma^E \bullet \longleftarrow \bullet)^h$$

for adequate protocol $\pi = (\pi_1, \pi_2)$ and σ , using a TOWP (which means that $h = \overline{\mathbf{G}}\rho$, where ρ is a simple function and $\mathbf{G}$ is the inversion game for the underlying TOWP).

The basic idea is that B generates a parameter-trapdoor pair (p, t) according to the right distribution, using the corresponding algorithm. The parameter p is sent over the channel $\longleftarrow \bullet$. Then A chooses a random key k , computes $y = f(k, p)$, where f is the corresponding function of the TOWP, and sends y to B over the channel $\bullet \longrightarrow$. Using the trapdoor t , B can compute $k = g(y, t)$.

However, this naïve implementation is not secure in general since partial information about the key might leak to E . For example, for the RSA TOWP, the value $K^e \pmod{n}$ leaks the information bit whether K is a quadratic residue modulo n , since this property is preserved by the computation of the e -th power.

This problem can be solved in the ROM, i.e., assuming the availability of a public random oracle $\mathbf{PO}_k$. The reader can verify as an exercise that if A and B use as their key $\mathbf{PO}_k$ evaluated at K , instead of K , then the construction

$$[\bullet \longrightarrow, \longleftarrow \bullet, \mathbf{PO}_k] \xrightarrow{\pi} (\sigma^E \bullet \longleftarrow \bullet)^h$$

is achieved, for the described protocol π and a suitable simulator σ , where h is as explained above.

Informally, what is exploited here is that the application of a random oracle converts an unguessable value into a uniformly random value. This is a general principle, and the value of a security proof in the ROM is questionable since one more or less assumes to be true what one cannot prove. Substantial research efforts have recently been devoted to achieving this construction without random oracle, for a standard hardness assumption such as that for integer factorization.

7.3.3 Impossibility of Implementing a Random Oracle

While the ROM is attractive and allows for simple security proofs, one can show that a public random oracle cannot be constructed. In other words, there cannot exist a hash function, no matter how ingeniously designed, which behaves like a random oracle. More concretely, the following theorem was shown by Canetti et al.¹¹

¹¹R. Canetti, O. Goldreich, and S. Halevi, The random oracle methodology, revisited, in *Proceedings of the 30th Annual ACM Symposium on the Theory of Computing*, pages 209–218. ACM Press, 1998.

Theorem 7.10. *There exist cryptographic schemes (e.g. a digital signature scheme), making use of a hash function, which are secure in the ROM (based on some intractability assumption) but which are insecure for every instantiation of $\mathbf{FO}_k$ as a concrete hash function (or a hash function class with a public parameter).*

Proof. The following is only a proof sketch, simplifying the arguments of Canetti et al. Several technical details would have to be made more precise. The basic idea of the proof is as follows. Assume there exists a secure signature scheme $\tilde{S}$ (based on some intractability assumption). We now define a signature scheme S which has available a black-box B that contains either a random oracle or the implementation of a hash function. The message input to the signature scheme S is, before signing it with $\tilde{S}$, interpreted as a program of a hash function. If it is a syntactically correct program taking inputs from $\{0, 1\}^*$ and producing outputs in $\{0, 1\}^n$, then S tests whether the program's output and the black-box B 's output agree, for sufficiently many inputs. The number of test inputs is chosen such that a random oracle has negligible probability of producing these outputs for the given inputs. Hence the test is positive if B contains a hash function and the message is chosen as the program code of the hash function in B . In contrast, the test is negative (except with negligible probability) if B contains a random oracle.

The message is signed using the scheme $\tilde{S}$. If the test is positive, then S appends the secret key of $\tilde{S}$ to the signature, and hence S becomes completely insecure. In summary, if B contains a random oracle (i.e., in the ROM), then this is a secure signature scheme (it is essentially identical to $\tilde{S}$), but if B contains any hash function (with, of course, known program code), then the scheme is insecure since there exists a message that renders the scheme insecure. $\square$

Appendix A

A.1 Probability Theory Basics

A.1.1 Probability Spaces

Definition A.1. A *probability space* consists of a *sample space* Ω , which is the set of all possible *outcomes*, a set of subsets of Ω called *events* (which form a σ -algebra), and a function $\Pr$, called the *probability measure*, assigning to every event a probability in $[0, 1]$, such that $\Pr(\Omega) = 1$ and the probability of the union of disjoint events is the sum of their probabilities.¹

A probability space is *discrete* if Ω is finite or countably infinite. One also uses the term *discrete random experiment*. For a discrete probability space one can consider all subsets of Ω as events, and it is therefore completely described by the probabilities of the elementary events $\omega \in \Omega$, i.e., by the so-called *probability mass function* $p : \Omega \rightarrow [0, 1]$ satisfying

$$\sum_{\omega \in \Omega} p(\omega) = 1,$$

where $\Pr(\{\omega\}) = p(\omega)$ for all $\omega \in \Omega$ and for any event $\mathcal{A} \subseteq \Omega$ we have

$$\Pr(\mathcal{A}) = \sum_{\omega \in \mathcal{A}} p(\omega).$$

We will almost exclusively consider finite random experiments or infinite random experiments that can be understood as an infinite sequence of finite random experiments.

One sometimes considers more than one random experiment at the same time, and hence using $\Pr$ as the probability measure is ambiguous. In such a case we make the random experiment E explicit as a superscript: $\Pr^E(\cdot)$. If a random experiment E consists of selecting some random variables *independently*,

¹For a good introduction to probability theory we refer to W. Feller, *An Introduction to Probability Theory*, Wiley, 3rd edition, 1968. or to John A. Rice, *Mathematical Statistics and Data Analysis*, Duxbury Press, 2nd edition, 1995.

then we write the list of random variables as superscript. For example, if the random variables X , Y , and Z are chosen independently, we write $\Pr^{XYZ}(\mathcal{A})$ for the probability of the event $\mathcal{A}$ in this random experiment. If for example Y and Z are not independent, but the pair (Y, Z) is independent of X , then we write $\Pr^{X(Y,Z)}(\mathcal{A})$, making explicit that the pair (Y, Z) is considered as a single (dependent) random variable.

Definition A.2. Two events $\mathcal{A}$ and $\mathcal{B}$ are called (statistically) *independent* if

$$\Pr(\mathcal{A} \cap \mathcal{B}) = \Pr(\mathcal{A}) \cdot \Pr(\mathcal{B}).$$

Definition A.3. The *conditional probability* of an event $\mathcal{A}$, given an event $\mathcal{B}$, is defined as

$$\Pr(\mathcal{A}|\mathcal{B}) = \frac{\Pr(\mathcal{A} \cap \mathcal{B})}{\Pr(\mathcal{B})}$$

if $\Pr(\mathcal{B}) > 0$, and it is undefined if $\Pr(\mathcal{B}) = 0$.

A.1.2 Random Variables

Definition A.4. A *random variable* X is a function from Ω to some set $\mathcal{X}$. The function $\mathcal{X} \rightarrow [0, 1]$ assigning to $x \in \mathcal{X}$ the probability of the event² $X = x$ is denoted as P_X and is called the *probability mass function* of X or also often the *probability distribution* of X :

$$P_X(x) = \Pr(X = x) = \sum_{\omega \in \Omega: X(\omega)=x} p(\omega).$$

A list of random variables can be viewed as a single random variable whose values are tuples. For example, we can consider the pair (X, Y) whose probability distribution³

$$P_{XY} : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$$

is a function of two arguments defined by⁴

$$P_{XY}(x, y) = \Pr(X = x, Y = y).$$

The probability distributions of X and Y are determined uniquely by P_{XY} and are called *marginal distributions*:

$$P_X(x) = \sum_{y: P_{XY}(x, y) > 0} P_{XY}(x, y)$$

²Here $X = x$ is understood as an event, i.e., as $\{\omega \in \Omega : X(\omega) = x\}$.

³One writes P_{XY} instead of $P_{(X,Y)}$.

⁴Here $\Pr(X = x, Y = y)$ stands for $\Pr(\{\omega \in \Omega : X(\omega) = x \text{ and } Y(\omega) = y\})$.

(and analogously for $P_Y(y)$). More generally, for n random variables $X_1, \dots, X_n$ and for $m < n$ we have:

$$P_{X_1 \dots X_m}(x_1, \dots, x_m) = \sum_{(x_{m+1}, \dots, x_n): P_{X_1 \dots X_n}(x_1, \dots, x_n) > 0} P_{X_1 \dots X_n}(x_1, \dots, x_n).$$

The marginal distribution of any subset of the random variables is computed analogously.

Definition A.5. The *conditional probability distribution* of random variable X , given (or conditioned on) the event $\mathcal{A}$ with $\Pr(\mathcal{A}) > 0$, is defined as

$$P_{X|\mathcal{A}}(x) = \Pr(X = x|\mathcal{A}).$$

The *conditional probability distribution* of X , given Y is the partial function

$$P_{X|Y} : \mathcal{X} \times \mathcal{Y} \rightarrow [0, 1]$$

defined by

$$P_{X|Y}(x, y) := \Pr(X = x|Y = y) = \frac{\Pr(X = x, Y = y)}{\Pr(Y = y)}.$$

$P_{X|Y}(x, y)$ is not defined for values y with $P_Y(y) = 0$.

Note that $P_{X|Y}$ is a real-valued partial function with *two* arguments. Note also that for every y with $P_Y(y) > 0$ we have

$$\sum_x P_{X|Y}(x, y) = 1,$$

i.e., $P_{X|Y}(\cdot, y)$ is, when considered as a one-argument function, itself a probability distribution.

Definition A.6. Two random variables X and Y are *statistically independent* if (and only if)

$$P_{XY}(x, y) = P_X(x) \cdot P_Y(y)$$

for all $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. Analogously, a list of random variables $X_1, \dots, X_n$ are independent if the probability distribution factors:

$$P_{X_1 \dots X_n}(x_1, \dots, x_n) = P_{X_1}(x_1) \cdots P_{X_n}(x_n).$$

It follows from the definition that for statistically independent X and Y we have $P_{X|Y}(x, y) = P_X(x)$ for all $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ with $P_Y(y) \neq 0$.

A.1.3 Expected Value and Variance

For a random variable X whose range is (a subset of) the real numbers, the *expected value* or *expectation* $E[X]$ and the *variance* $\text{Var}[X]$ are defined as follows:

$$E[X] = \sum_x x \cdot P_X(x)$$

and

$$\text{Var}[X] = \sum_x (x - E[X])^2 P_X(x) = E[(X - E[X])^2].$$

It is easy to show that

$$E[X_1 + \dots + X_n] = E[X_1] + \dots + E[X_n].$$

If $X_1, \dots, X_n$ are statistically independent (actually, pairwise statistically independent suffices), then

$$\text{Var}[X_1 + \dots + X_n] = \text{Var}[X_1] + \dots + \text{Var}[X_n].$$

For a real-valued function f whose domain includes $\mathcal{X}$, we have

$$E[f(X)] = \sum_x f(x) P_X(x)$$

and

$$\text{Var}[f(X)] = \sum_x (f(x) - E[f(X)])^2 P_X(x).$$

A.2 Information Theory Basics

Information theory, invented by Claude Shannon in 1948, is a very powerful theory for analyzing many problems related to information transmission and storage. Information theory can be considered as a special branch of probability theory. Our purpose for using information theory is to prove results on information-theoretic security of cryptographic schemes, i.e., security against computationally unbounded adversaries.

The basic information-theoretic quantity is the entropy of a probability distribution or, equivalently, of a random variable (i.e., of its probability distribution). Entropy is a real-valued positive quantity that measures in some sense the uncertainty contained in a probability distribution. The larger the entropy, the larger the uncertainty about the outcome of a random experiment with the considered probability distribution.

Definition A.7. The *entropy* of a discrete random variable X (with probability distribution P_X) is defined as

$$H(X) := - \sum_{x \in \mathcal{X}: P_X(x) \neq 0} P_X(x) \log_2 P_X(x).$$

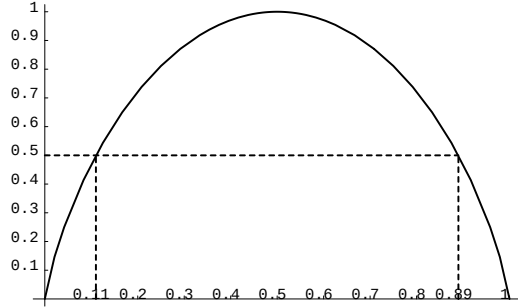


Figure A.1: The binary entropy function $h(p)$.

Entropy can also be defined as an expected value. Define the function $g: \mathcal{X} \rightarrow \mathbb{R}$ as $g(x) := -\log_2 P_X(x)$. Then

$$H(X) = E[g(X)] = E[-\log_2 P_X(X)]$$

We will not be interested in the particular formula, nor why this is the right measure of uncertainty for applications like data compression (source coding) or channel coding. We are only interested in certain properties of entropy which we discuss next. We do not prove the basic information-theoretic facts.

Example A.1. The probability distribution of a binary random variable X is characterized by the single parameter $P_X(0) = p$ (since $P_X(1) = 1 - p$). The entropy of X can be considered as a function of p ; this is called the *binary entropy function* h :

$$h(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

for $0 < p < 1$ and $h(0) = h(1) = 0$. The function $h(p)$ is strictly concave and takes its maximum at $p = 1/2$, where $h(1/2) = 1$ (see Figure A.1).

Theorem A.1.

$$0 \leq H(X) \leq \log_2 |\mathcal{X}|$$

with equality on the left side if and only if $P_X(x) = 1$ for one value x and with equality on the right side if and only if P_X is the uniform distribution over $\mathcal{X}$.

A pair $[X, Y]$, a triple $[X, Y, Z]$, or a list $[X_1, \dots, X_N]$ of random variables can be considered as a single random variable. Hence the entropy of several random variables is already defined. We write $H(XY)$ instead of $H([X, Y])$, and similarly for several random variables. For example, we have

$$H(XY) = - \sum_{(x,y)} P_{XY}(x,y) \log_2 P_{XY}(x,y) = E[-\log_2 P_{XY}(X,Y)].$$

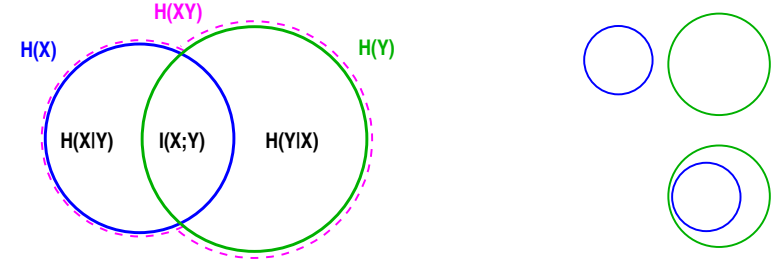


Figure A.2: Left: The relation between $H(X)$, $H(Y)$, and $H(XY)$. Right: Special cases of the diagram if X and Y are statistically independent (top) and if X is uniquely determined by Y (bottom).

One can prove:

Theorem A.2. We have

$$H(X) \leq H(XY)$$

with equality if and only if Y is uniquely determined by X , and

$$H(XY) \leq H(X) + H(Y)$$

with equality if and only if X and Y are statistically independent.

This motivates the definition of two derived quantities (see Figure A.2). It follows from the above theorem that both quantities are non-negative.

Definition A.8. The *conditional entropy of Y , given X* is defined as

$$H(Y|X) := H(XY) - H(X),$$

and the *mutual information between X and Y* is defined as

$$I(X;Y) := H(X) + H(Y) - H(XY).$$

Theorem A.2 can equivalently be stated as $H(Y|X) \geq 0$ and $I(X;Y) \geq 0$. The so-called chain rule follows by repeated application of this definition:

$$H(X_1 \cdots X_N) = \sum_{n=1}^N H(X_n | X_1 \cdots X_{n-1}).$$

There is one more basic quantity, now involving *three* random variables:

Definition A.9. The *conditional mutual information of X and Y , given Z* is defined as

$$I(X;Y|Z) := H(XZ) + H(YZ) - H(XYZ) - H(Z).$$

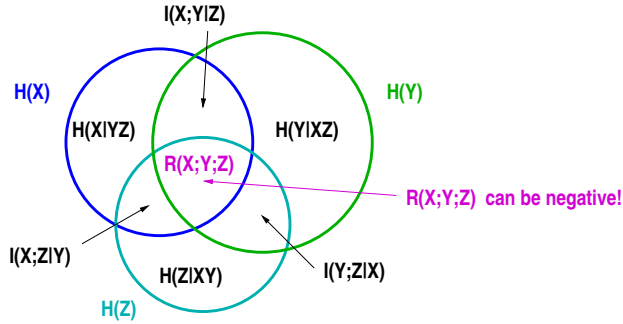


Figure A.3: Information-theoretic quantities for three random variables X , Y , and Z .

Theorem A.3. We have

$$I(X; Y|Z) \geq 0$$

with equality if and only if X and Y are statistically independent when given Z .

All these definitions and theorems hold when X , Y , and Z are replaced by lists of random variables.

Example A.2. Consider 5 random variables R, S, T, U, V . We have for example $H(RS) \leq H(RST)$ and $H(RSTUV) \leq H(RS) + H(TUV)$. Moreover, $H(RS|TUV) = H(RSTUV) - H(TUV)$ and $I(RS; T|UV) = H(RSUV) + H(TUV) - H(RSTUV) - H(UV)$.

When considering a list $X_1, \dots, X_n$ of n random variables, all information-theoretic quantities are determined by the $2^n - 1$ entropies of all (non-empty) subsets of the random variables. Figure A.3 shows the entropy diagram for three random variables X , Y , and Z . There are 7 regions, each of which corresponds to a certain combination (sum and differences) of the entropies $H(X)$, $H(Y)$, $H(Z)$, $H(XY)$, $H(XZ)$, $H(YZ)$, and $H(XYZ)$. The inner region, denoted $R(X; Y; Z)$, can be negative and does not seem to have a meaning in an application context. All other regions are non-negative.

Example A.3. Let X and Y be independent, unbiased random bits, and let $Z := X \oplus Y$. The entropy diagram of X , Y , and Z is shown in Figure A.4. Note that $I(X; Y) = 0$ and $I(X; Y|Z) = 1$.

A.3 Number Theory and Algebra Basics

We summarize a number of facts from number theory and algebra. Most proofs are omitted. A more detailed version of much of this material can be found in

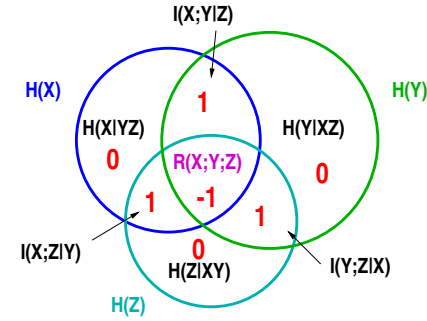


Figure A.4: Entropy diagram for Example A.3.

my lecture notes *Discrete Mathematics*.

A.3.1 Euclid's Extended GCD-Algorithm

Theorem A.4. The algorithm described in Figure A.5 computes, for given nonnegative integers a and b with $a \geq b$ (not both 0), the integers $d = \gcd(a, b)$, as well as u and v satisfying $ua + vb = \gcd(a, b)$.

```

 $\sigma_1 := a; \sigma_2 := b;$ 
 $u_1 := 1; u_2 := 0;$ 
 $v_1 := 0; v_2 := 1;$ 
while  $\sigma_2 > 0$  do begin
   $q := \sigma_1 \text{ div } \sigma_2;$ 
   $r := \sigma_1 - q\sigma_2;$ 
   $\sigma_1 := \sigma_2; \sigma_2 := r;$ 
   $t := u_2; u_2 := u_1 - qu_2; u_1 := t;$ 
   $t := v_2; v_2 := v_1 - qv_2; v_1 := t;$ 
end;
 $d := \sigma_1; u := u_1; v := v_1;$ 

```

Figure A.5: Extended gcd-algorithm (Euclid). For given nonnegative integers a and b with $a \geq b$, the algorithm computes $d = \gcd(a, b)$ as well as u and v satisfying $ua + vb = \gcd(a, b)$.

Proof. The algorithm terminates because the numbers (e.g. σ_1) decrease monotonically but remain nonnegative. The (well-known) proof that at the end of the algorithm we have $d = \gcd(a, b)$ is left as an exercise. We prove by induction

that at the beginning of the **while**-loop the two invariants

$$u_1 a + v_1 b = \sigma_1 \quad (\text{A.1})$$

and

$$u_2 a + v_2 b = \sigma_2 \quad (\text{A.2})$$

are always satisfied. After the initialization they are trivially satisfied. Equation (A.1) is satisfied at the end of the loop because the triple (σ_1, u_1, v_1) is assigned the value of the triple (σ_2, u_2, v_2) and therefore condition (A.1) at the end of the loop is equivalent to condition (A.2) at the beginning of the loop. Equation (A.2) holds at the end of the loop because

$$(u_1 - qu_2)a + (v_1 - qv_2)b = (u_1 a + v_1 b) - q(u_2 a + v_2 b) = r$$

holds at the beginning of the loop (by application of (A.1) and (A.2)) and because of the assignment of $(u_1 - qu_2)$, $(v_1 - qv_2)$ and r to v_1 , v_2 and σ_2 , respectively, within the loop. $\square$

Modular Congruences

Definition A.10. For $a, b, m \in \mathbb{Z}$ with $m \geq 1$, we say that a is congruent to b modulo m if m divides $a - b$. We write $a \equiv b \pmod{m}$ or simply $a \equiv_m b$:

$$a \equiv_m b \iff m \mid (a - b).$$

Example A.4. We have $23 \equiv_7 44$ and $54321 \equiv_{10} 1$. Note that $a \equiv_2 b$ means that a and b are either both even or both odd. It is easy to see that $a \equiv_1 b$ is satisfied for all a and b .

The following lemma shows that modular congruences are compatible with the arithmetic operations on $\mathbb{Z}$.

Lemma A.5. If $a \equiv_m b$ and $c \equiv_m d$, then $a + c \equiv_m b + d$ and $ac \equiv_m bd$.

Corollary A.6. Let $f(x_1, \dots, x_k)$ be a multi-variate polynomial in k variables with integer coefficients, and let $m \geq 1$. If $a_i \equiv_m b_i$ for $1 \leq i \leq k$, then $f(a_1, \dots, a_k) \equiv_m f(b_1, \dots, b_k)$.

There are m equivalence classes of the equivalence relation $\equiv_m$, namely $[0], [1], \dots, [m-1]$. Each equivalence class $[a]$ has a natural representative $R_m(a) \in [a]$ in the set

$$\mathbb{Z}_m := \{0, \dots, m-1\}$$

of remainders modulo m . Here $R_m(a)$ denotes the remainder when a is divided by m .

One is often interested only in the remainder of an integer (e.g. the result of a computation) modulo some modulus m . The following lemma establishes the simple connection between congruence modulo m and remainders modulo m . The proof is easy and left as an exercise.

Lemma A.7. For any $a, b, m \in \mathbb{Z}$ with $m \geq 1$,

- (i) $a \equiv_m R_m(a)$.
- (ii) $a \equiv_m b \iff R_m(a) = R_m(b)$.

The above lemma together with Lemma A.5 implies the following well-known fact: If in a computation involving addition and multiplication one is interested only in the remainder of the result modulo m , then one can compute remainders modulo m at any intermediate step (thus keeping the numbers small), without changing the result. This is referred to as *modular arithmetic*.

Lemma A.8. For any $a, b, m \in \mathbb{Z}$ with $m \geq 1$,

- (i) $R_m(a + b) = R_m(R_m(a) + R_m(b))$.
- (ii) $R_m(ab) = R_m(R_m(a) \cdot R_m(b))$.

A.3.2 Inverses Modulo m

Lemma A.9. If $\gcd(a, m) = 1$, there is a unique solution $x < m$ to the congruence equation

$$ax \equiv_m 1.$$

Proof. According to Theorem A.4 there exist integers u and v such that $ua + vm = \gcd(a, m) = 1$. Since $vm \equiv_m 0$ we have $ua \equiv_m 1$. Hence $R_m(u)$ is a solution. Suppose there is another solution u' . Then $ua - u'a \equiv_m 0$, thus $(u - u')a \equiv_m 0$ and hence m divides $(u - u')a$. Since $\gcd(a, m) = 1$, m must divide $(u - u')$, i.e., $R_m(u)$ is the unique solution in $\mathbb{Z}_m$. $\square$

Definition A.11. If $\gcd(a, m) = 1$, the unique solution $x \in \mathbb{Z}_m$ to the congruence equation $ax \equiv_m 1$ is called the *multiplicative inverse of a modulo m* . One also uses the notation $x \equiv_m a^{-1}$.

Example A.5. The multiplicative inverse of 5 modulo 13 is 8 since $5 \cdot 8 = 40 \equiv_{13} 1$.

The multiplicative inverse can efficiently be computed using the extended Euclidean algorithm of Section A.3.1 (setting $b = m$) which yields u (and v) such that $ua + vm = \gcd(a, m) = 1$. Since $vm \equiv_m 0$ we have $ua \equiv_m 1$. Hence the multiplicative inverse of a modulo m is $R_m(u)$.

A.3.3 The Chinese Remainder Theorem

We now consider a system of congruences for an integer x . The following theorem is known as the Chinese Remainder Theorem (CRT).

Theorem A.10. Let $m_1, m_2, \dots, m_r$ be pairwise relatively prime integers and let $M = \prod_{i=1}^r m_i$. For every list $a_1, \dots, a_r$ with $0 \leq a_i < m_i$ for $1 \leq i \leq r$, the system of congruence equations

$$\begin{aligned} x &\equiv_{m_1} a_1 \\ x &\equiv_{m_2} a_2 \\ &\dots \\ x &\equiv_{m_r} a_r \end{aligned}$$

for x has a unique solution x satisfying $0 \leq x < M$.

The proof shows that the solution x can actually be computed efficiently.

Proof. Let $M_i = M/m_i$. Hence $\gcd(M_i, m_i) = 1$ and thus there exists an N_i satisfying

$$M_i N_i \equiv_{m_i} 1.$$

Note that

$$M_i N_i \equiv_{m_j} 0$$

for all $j \neq i$. Hence the integer x defined by

$$x = R_M \left(\sum_{i=1}^r a_i M_i N_i \right)$$

satisfies all the congruences. In order to prove uniqueness, observe that for two solutions x' and x'' , $x' - x'' \equiv_{m_i} 0$ for all i , i.e., $x' - x''$ is a multiple of all the m_i and hence of M . Thus $x' \equiv_M x''$. $\square$

A.3.4 Groups

Definition A.12. A group $\langle G; *, e \rangle$ is a set G together with an operation $*$ on G and a special (neutral) element e such that

- (1) $*$ is associative.
- (2) e is a neutral element: $a * e = e * a = a$ for all $a \in G$.
- (3) Every $a \in G$ has an inverse element, denoted $\hat{a}$, i.e., $a * \hat{a} = \hat{a} * a = e$.

A group $\langle G; *, e \rangle$ is called *commutative* or *abelian* if $*$ commutes, i.e., if $a * b = b * a$ for all $a, b \in G$.

Some standard examples of groups are $\langle \mathbb{Z}; +, 0 \rangle$, $\langle \mathbb{Q}; +, 0 \rangle$, $\langle \mathbb{Q} \setminus \{0\}; \cdot, 1 \rangle$, $\langle \mathbb{R}; +, 0 \rangle$, $\langle \mathbb{R} \setminus \{0\}; \cdot, 1 \rangle$, and $\langle \mathbb{Z}_m; \oplus, 0 \rangle$.

Definition A.13. For a group $\langle G; *, e \rangle$ and a subset H of G , H (or, more precisely $\langle H; *, e \rangle$) is a *subgroup* of $\langle G; *, e \rangle$ if it is itself a group for the operation $*$ restricted to H .

In the remainder of this section we will use a multiplicative notation for groups, i.e., we denote the group operation as “ $\cdot$ ” (which can also be omitted). But it is important to point out that this is only a notational convention that entails no loss of generality of the kind of group operation. In many cases (but not always) we denote the neutral element of a multiplicatively written group as 1. The inverse of a is denoted a^{-1} or $1/a$, and a/b stands for ab^{-1} . Furthermore, we use the common notation a^n for the n th power of an element a . For all $m, n \in \mathbb{Z}$ we have

$$a^m \cdot a^n = a^{m+n} \quad \text{and} \quad (a^m)^n = a^{mn}.$$

Definition A.14. Let G be a group and let a be an element of G . The *order* of a , denoted $\text{ord}(a)$, is the least $m \geq 1$ such that $a^m = 1$, if such an m exists, and $\text{ord}(a) = \infty$ otherwise.

Example A.6. By definition, $\text{ord}(1) = 1$. If $\text{ord}(a) = 2$ for some a , then $a^{-1} = a$. The order of 6 in $\langle \mathbb{Z}_{20}; \oplus \rangle$ is 10.

Lemma A.11. In a finite group G , every element has a finite order.

Definition A.15. For a finite group G and $a \in G$, the *subgroup generated by a* is⁵

$$\langle a \rangle := \{1, a, a^2, \dots, a^{\text{ord}(a)-1}\}.$$

Definition A.16. A finite group $G = \langle g \rangle$ generated by an element $g \in G$ is called *cyclic*, and g is called a *generator* of G .

Example A.7. The group $\langle \mathbb{Z}_n, \oplus \rangle$ is cyclic for every n , where 1 is a generator. The generators of $\langle \mathbb{Z}_n, \oplus \rangle$ are all $g \in \mathbb{Z}_n$ for which $\gcd(g, n) = 1$.

Lemma A.12. A cyclic group of order n is isomorphic to $\langle \mathbb{Z}_n, \oplus \rangle$ (and hence abelian).

Definition A.17. For a finite cyclic group $G = \langle g \rangle$ and an element $a \in G$, the unique integer $x \in \{0, \dots, |G|-1\}$ satisfying $a = g^x$ is called the *discrete logarithm* of a to the base g .

Definition A.18. For a finite group G , the cardinality $|G|$ is called the *order* of G .

Theorem A.13 (Lagrange). Let G be a finite group and let H be a subgroup of G . Then $|H|$ divides $|G|$.

The following corollaries are direct applications of Lagrange’s theorem.

Corollary A.14. Let G be a finite group. Then $\text{ord}(a)$ divides $|G|$ for every $a \in G$.

Corollary A.15. Let G be a finite group. Then $a^{|G|} = e$ for every $a \in G$.

⁵The concept of being generated by an element can naturally be generalized to infinite groups. For example, the group $\langle \mathbb{Z}; +, 0 \rangle$, is generated by 1.

Corollary A.16. Every group of prime order⁶ is cyclic, and every element except the neutral element is a generator.

Definition A.19.

$$\mathbb{Z}_m^* := \{a \in \mathbb{Z}_m \mid \gcd(a, m) = 1\}.$$

The Euler function $\varphi : \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ is defined as the cardinality of $\mathbb{Z}_m^*$:

$$\varphi(m) = |\mathbb{Z}_m^*|.$$

Example A.8. $\mathbb{Z}_{18}^* = \{1, 5, 7, 11, 13, 17\}$. Hence $\varphi(18) = 6$.

Example A.9. If p is a prime, then $\mathbb{Z}_p^* = \{1, \dots, p-1\} = \mathbb{Z}_p - \{0\}$.

Lemma A.17. If $m = \prod_{i=1}^r p_i^{e_i}$, then

$$\varphi(m) = \prod_{i=1}^r (p_i - 1)p_i^{e_i-1}.$$

Lemma A.18. $\langle \mathbb{Z}_m^*; \odot \rangle$ is a group, where $\odot$ denotes multiplication modulo m .

Now we obtain the following simple but powerful corollary to Lagrange's theorem:

Corollary A.19 (Fermat, Euler). For all $m \geq 2$ and all a with $\gcd(a, m) = 1$,

$$a^{\varphi(m)} \equiv_m 1.$$

In particular, for every prime p and every a not divisible by p ,

$$a^{p-1} \equiv_p 1.$$

A.3.5 Fast Exponentiation

Consider a general group G , an element $g \in G$, and an exponent $e \in \mathbb{Z}$. We show how g^e can be computed efficiently. Let the binary representation of the n -bit exponent e be $e_{n-1} \dots e_1 e_0$, i.e.

$$e = \sum_{i=0}^{n-1} e_i 2^i.$$

Then g^e can be computed by the following simple algorithm often referred to as “square and multiply” algorithm or the “binary method”. The result is accumulated in the variable a .

```

a := 1;
for i := n - 1 downto 0 do begin
    a := a * a;
    if e_i = 1 then a := a * g;
end;
```

The number of actual⁷ multiplications is

$$n + w(e) - 2 \leq 2n - 2,$$

where $w(e)$ is the number of 1's in the binary representation of e . For most exponents there exist methods for exponentiation requiring fewer multiplications. For example, g^{23} is in the above procedure obtained by computing $g^2, g^4, g^5, g^{10}, g^{11}, g^{22}$ and g^{23} , but an alternative sequence of powers requiring one multiplication less is $g^2, g^3, g^5, g^{10}, g^{20}, g^{23}$.

⁶i.e., $|G| = p$ for some prime p . Groups of prime order play a very important role in cryptography.

⁷The algorithm, as described, performs also trivial multiplications (by 1).